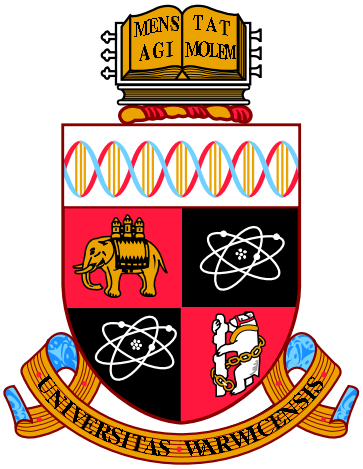


Deep Reinforcement Learning and Macroeconomic Modelling

by

Rui (Aruhan) Shi

Thesis

Submitted to the University of Warwick

for the degree of

Doctor of Philosophy

Department of Economics

August 2023

THE UNIVERSITY OF
WARWICK

Contents

List of Tables vi

List of Figures vii

Acknowledgments ix

Declarations xii

Abstract xiii

Abbreviations xiv

Chapter 1 Introduction 1

Chapter 2 Deep Reinforcement Learning: Emerging Trends in Macroeconomics and Future Prospects 4

2.1 Introduction 4

2.2 What is Reinforcement Learning and Deep Reinforcement Learning? 6

2.2.1 Brief History 6

2.2.2 Theory 9

2.2.3 RL Algorithms 17

2.2.4 Recent Applications 27

2.3 Deep Reinforcement Learning: Applications and Emerging Trends in Macroeconomics 29

2.3.1 Finding Optimal Policies 30

2.3.2 Bounded Rationality, Learning and Convergence 34

2.4 Deep Reinforcement Learning in Macroeconomics: Prospects and Issues 36

2.4.1 Prospects 37

2.4.2 Issues 44

2.5 Conclusion 48

Chapter 3 AI for Behavioural Economic Agents: Modeling Adaptability and Exploration in Dynamic Environments 49

3.1 Introduction 49

3.2 Related Literature 52

3.3 Methodology 54

3.3.1 The Model: an Economist's Approach 55

3.3.2 The Model: an AI Approach 58

3.3.3 Training Period Descriptions 68

3.3.4 Parameters and Learning Agent's Characteristics 69

3.4 Experiments and Results 70

3.4.1 An RE Agent and an AI Agent in a Stationary Environment with i.i.d Shocks 72

3.4.2 AI Agents with Different Exploration Levels in a Stationary Environment with i.i.d Shocks 78

3.4.3 AI Agents with Different Exploration Levels in an Environment with a Permanent Change 82

3.5 Summary 87

Chapter 4 Can an AI Agent Hit a Moving Target? 89

4.1 Introduction 89

4.2 The Model: an Economist's Approach 92

4.2.1 A Representative Household 93

4.2.2 Government: Fiscal and Monetary Policies 94

4.2.3 Household's First Order Conditions (FOCs) 95

4.2.4 Non-Stochastic Steady States and Determinancy 96

4.3 The Model: an AI Approach 97

4.3.1 Deep RL Components 98

4.3.2 Full Algorithm and Sequence of Events 100

4.4 Parameters and (Non-Stochastic) Steady State Values 104

4.5 Experiments and Results 105

4.5.1 Experiments 105

4.5.2 Inflation and Wealth 108

4.5.3 Consumption, Storage, and Liquidity Holding 110

4.5.4 AI vs RE agent(s) 115

4.5.5 Discussion 120

4.6 Summary 122

Appendix A Deep Reinforcement Learning: Emerging Trends in Macroeconomics and Future Prospects 124

A.1 How do ANNs learn? 125

A.2 Comparing RL with Other Solution Methods for MDPs 128

Appendix B AI for Economic Agent Behaviors: Modeling Adaptability and Exploration in Dynamic Environments 130

B.1 Derivation of the Analytical Solution to the Stochastic Optimal Growth Model 130

B.2 Neural Network Architectures and Parameters 133

B.3 Additional Results: Random Initial Conditions during Training . . . 135

B.4 Additional Results: Zero Mean for the Shock Process 136

List of Tables

2.1 Terminologies in Reinforcement Learning 11

3.1 RL Components and the Economic Environment 59

3.2 Main Parameters 70

4.1 RL Components and the Economic Environment 99

4.2 Baseline Parameters and Steady State Values 104

B.1 ANN Architectures and Parameters 134

List of Figures

2.1 A Markov Decision Process (A Reinforcement Learning Problem) . . 10

2.2 A Feedforward Deep ANN 23

2.3 Workflow of Deep RL Algorithms 25

2.4 Multiagent RL 26

3.1 Output Comparison 74

3.2 Consumption and Saving levels of an RE agent and an AI agent . . 75

3.3 Distance Metrics 77

3.4 Output Comparison 79

3.5 Consumption and Saving Levels Comparison 81

3.6 The Stochastic Process (in level and first difference) 83

3.7 Output (in level and first difference) 84

3.8 Consumption (in level and first difference) 85

3.9 Saving (in level and first difference) 86

4.1 Inflation 108

4.2 Wealth 109

4.3 Consumption of AI Agents during a Regime Change 111

4.4 Storage of AI Agents during a Regime Change 113

4.5 Real balance 115

4.6 AI vs RE Agents Before and After a Regime Change 117

4.7 AI vs RE Agents Before and After a Regime Change 118

A.1 A Feedfoward Network with One Hidden Layer 125

A.2 Learning Rates 127

A.3 Comparison of RL and Other Methods 129

B.1 Distance Metrics 136

B.2 The Stochastic Process (in level and first difference) 138

B.3 Output (in level and first difference) 139

B.4 Consumption (in level and first difference) 140

B.5 Savings (in level and first difference) 141

Acknowledgments

I would like to express my deepest gratitude to those who have supported and contributed to the completion of this thesis.

First and foremost, I would like to express my sincere gratitude to my supervisors, Roger Farmer and Herakles Polemarchakis for their invaluable guidance, unwavering support, and mentorship throughout the journey of completing my Ph.D. dissertation.

Professor Farmer's profound knowledge, and passion and dedication to the field, have been instrumental in shaping the direction of my research. I am sincerely thankful to him for consistently encouraging me to think beyond conventional boundaries, nurturing my independent thought process, and equipping me with the skills to effectively defend my diligent work. His meticulous attention to detail and constructive feedback have significantly enhanced the quality of my work. I am truly indebted to him for the invaluable knowledge and skills I have acquired under his insightful supervision.

I am equally grateful for the invaluable contributions of Professor Polemarchakis. His dedication to grounded and rigorous economic analysis has profoundly influenced my research journey. Through his emphasis on the immense value of the subject's resources, he has instilled in me a deep commitment to scholarly integrity and a comprehensive understanding of the subject matter. I am truly indebted to Professor Polemarchakis for enriching my research pursuits.

Beyond their academic guidance, I would like to acknowledge the personal support and mentorship provided by both supervisors. Their genuine care for my well-being, and their patience during challenging times have been truly invaluable.

I am thankful for the insightful comments and valuable suggestions made by my examiners, Professor Martin Ellison and Professor Pablo Beker. Their contributions have greatly enhanced the quality of this thesis.

I would like to express my gratitude to my colleagues at the IMF, especially Tohid Atashbar, Stephan Danninger, and Pawel Zabczyk for their valuable contributions to my research during my PhD internship. I am equally grateful for the insightful exchanges with the SPR/MP division, which influenced the first chapter of my thesis and enhanced my understanding of practical macroeconomics.

I appreciate the Rebuilding Macroeconomics research network and the "Deep Learning in a New Keynesian Model of Banking and Money Creation" project team, including Mingli Chen, Andreas Joseph, Michael Kumhof and Xinlei Pan, for their enriching dialogues and support. I am equally grateful to the Advanced Analytics Division of the Bank of England for their facilitation of my research visit and for their stimulating discussions.

I'm deeply thankful to the Department of Economics for their support and guidance, particularly to Jeremy Smith and Manuel Bagues. I am very grateful to James Fenske for his valuable comments and understanding. My thanks go to the Macro Research Group for providing an interactive platform for feedback and idea sharing, particularly to Christine Braun, Diego Calderon, Denis Novy, Roberto Pancrazi, Livia Silva Paranhos, Carlo Perroni, Giovanni Ricco, Federico Rossi, Alperen Tosun, Anshumaan Tuteja, Thijs Van Rens, Marija Vukotic, and Ivan Yotzov. My gratitude also extends to Luis Candelaria, Eric Renault, and Ao Wang for their comments and suggestions. I am fortunate to have been part of such an intellectually stimulating academic community. I am equally grateful to the administrative support and technical assistance that have been invaluable in facilitating the smooth progress of this study. In particular, I am thankful to Natalie Deven and Colin Ellis.

I am grateful for the financial support provided by the Department of Economics, which has allowed me to fully immerse myself in my academic pursuit.

I am grateful for the useful feedback and support from the Royal Economic Society Women's Committee Mentoring team - Friederike Mengel, Laura Muñoz Blanco, Karmini Sharma and Ines Vilela.

I am equally thankful for the useful comments from participants of the 2021 Chi-

nese Economist Society Conference, the 2021 Society for Computational Economics Conference, the 2021 CESifo Area Conference on Macro, Money, and International Finance, the 2021 Money, Macro and Finance Conference, the 2021 Bank of England Advanced Analytics: new methods and applications for macroeconomic policy Conference, the 2022 Society for Computational Economics Conference, and the 2022 Barcelona School of Economics PhD Workshop on Expectations in Macroeconomics.

Lastly but not least, I wish to convey my heartfelt gratitude to my dear ones who have been my steadfast source of love and comprehension on this journey.

Firstly, I thank my parents for their unyielding encouragement and faith in me, which have been a driving force in keeping me motivated to overcome challenges.

I extend my gratitude to my cherished friend, Ivy, for her unwavering friendship and support. Having her by my side is a true blessing.

I am fortunate to have shared many interesting and intellectually stimulating discussions, especially on reinforcement learning and programming with Alassane. These discussions enriched the early stages of this thesis.

I am grateful to be part of the "Asian Women in Macro" group alongside Amanah, Ivy and Vania.

I am blessed to have the camaraderie and support of my PhD colleagues and housemates: Alperen, Amedeo, Andreas, Anshumaan, Bruno, Carlos, Davide, Immanuel, Ivan, Miguel, Raghav and Theodor. Their presence has made the daunting moments of this journey enjoyable.

Rui (Aruhan) Shi

Declarations

The first chapter is co-authored with Tohid Atashbar (International Monertary Fund). This paper was written while I was a PhD intern at the IMF. I performed a survey of existing literature, and conducted a detailed analysis on the methodological framework of AI technologies, specifically focusing on deep reinforcement learning. I made partial contributions to the introduction and conclusion, as well as the section on prospects and issues of applying deep RL algorithms in macroeconomics.

Any views expressed in the first chapter are solely those of the authors and so cannot be taken to represent those of the IMF. This chapter should therefore not be reported as representing the views of the IMF.

The second and third chapters are entirely my own work.

I declare that the content presented in this thesis has not been previously published in academic journals and has not been submitted for another degree or at another university.

Rui (Aruhan) Shi

Abstract

I introduce a theory of bounded rationality, utilizing an AI framework to account for the inherent limitations and characteristics in human decision-making processes. The key focus of my contribution lies in implementing this framework within general equilibrium models to highlight the adaptive capabilities of AI agents. Additionally, I take into consideration potential genetic disparities among human actors in the decision-making process by integrating an exploration mechanism. Across three chapters, I examine the integration of AI representative agents in well-established general equilibrium models in macroeconomics.

Chapter 2 serves as an introductory section, delving into the methodological framework's historical development and theoretical foundations. I explore the applications of deep RL in economics, critically analyzing the framework's potential benefits and limitations when employed in macroeconomics.

In Chapters 3 and 4, I present the simulation experiments conducted in this thesis, aiming to assess the usefulness of the proposed AI framework in modelling adaptive behaviors. The first application focuses on a stochastic optimal growth environment with dynamic changes in income processes over time. The second application centers around a transaction cost of money demand model, involving a shift in the monetary policy regime from a constant to an increasing nominal money supply.

Through these simulation experiments, I demonstrate the versatility and advantages of the deep RL framework to model economic agents. Additionally, the results highlight the significant influence of exploration variations on distinct adaptive behaviors. Moreover, I discuss the limitations of the deep RL framework.

Abbreviations

ABMs Agent Based Models 37

AI Artificial Intelligence 4

ANN Artificial Neural Network 8

DL Deep Learning 4

DDPG Deep Deterministic Policy Gradients 16

DQN Deep Q Networks 16

MDP Markov Decision Process 9

ML Machine Learning 4

NLP Natural Language Processing 4

RL Reinforcement Learning 4

RE Rational Expectations 30

SARSA State-Action-Reward-State-Action 8

TD Temporal Difference 7

Chapter 1

Introduction

I present a theory of bounded rationality by employing a deep RL framework, which encompasses and accounts for the inherent limitations and characteristics in human decision-making processes. The primary focus is on integrating AI representative agents into widely-used general equilibrium models in macroeconomics to emphasize their adaptive capabilities.

The deep RL framework allows for capturing interactive dynamics between agents and the broader economy, considering their behavioral patterns and adaptive mechanisms. To acknowledge potential genetic disparities among human actors in decision-making, an exploration mechanism is integrated, representing varying propensities for experimentation or sticking to familiar options. The AI agents possess different levels of exploration, reflecting this inherent diversity.

To evaluate the usefulness of the proposed AI framework, two applications are performed. The first involves a stochastic optimal growth environment with dynamic income processes, while the second pertains to a transaction cost of money demand model, simulating a shift from zero growth of nominal money supply to a 1% growth

rate. Simulation experiments demonstrate the versatility and benefits of the deep RL framework for modeling economic agents. Notably, differences in exploration significantly influence distinct adaptive behaviors, leading to welfare implications.

Chapter 2[1] serves as an introduction to the methodological framework. Here, I present an overview of deep RL, a cutting-edge AI technology. I discuss its historical development, theoretical foundations, and explore various algorithms associated with it. Additionally, I examine the applications of deep RL in the field of economics. Lastly, I critically analyze the potential benefits and limitations of employing deep RL in macroeconomics, highlighting several key challenges that need to be addressed to foster its broader adoption in macroeconomic modeling.

In Chapter 3[2], I study the behaviors of an AI agent within a stochastic optimal growth environment through simulation experiments. I provide comprehensive analysis and attention to each component of the deep RL framework, which includes but is not limited to the decision-making strategy, subjective beliefs, and memory of the AI agent. Moreover, I emphasize the significance of exploration in influencing the AI agent's behaviors when making consumption-saving decisions in response to income shocks.

I find that AI agents start in a naive state, unaware of their action set, reward function, or state transition process. As they progress, their behaviors increasingly resemble those of rational agents, though their actions and beliefs will never be identical to those of an RE agent. This distinction is mainly due to the perpetual exploration characteristic inherent in AI agents. The extent of exploration enables AI agents to adjust and adapt to changes in the environment. Moreover, I have ob-

[1] A version of this chapter was published as IMF Working Paper No. 2022/259. I thank Stephan Danninger and Tohid Atashbar for permission to include it here.

[2] This chapter was circulated under the title "Learning from Zero: How to Make Consumption-Saving Decisions in a Stochastic Environment with an AI Algorithm" as CESifo working paper no.9255

served that AI agents who engage in more exploration tend to save more than those who explore less, contributing to an approximate 1.5% increase in the economy's output production.

In Chapter 4, I look into an examination of how an AI representative agent responds to a drastic environmental change. Specifically, I analyze how the agent reacts to changes in the stationarity properties of aggregate price sequences within a general equilibrium framework. This framework incorporates transaction costs of money demand, thus providing insights into the roles of inflation and monetary policy. Drawing upon the existing literature on the accelerationist debate, I conduct several simulation experiments, varying the growth rate of the nominal money supply set by the monetary authority. Through this regime change, I demonstrate how the AI agent adjusts its beliefs and behaviors in response to the altered stationarity characteristics observed in aggregate prices.

During the regime change experiments, I observe that the AI agent displays a capacity to adapt its beliefs in response to economic changes. This adaptability is evident in its adjustments to consumption and decisions regarding real balance and storage, indicating a sophisticated ability to respond to varying circumstances. This adaptability is mainly driven by the exploration feature, which also results in AI agents' behaviors not being identical to those of an RE agent.

Chapter 2

Deep Reinforcement Learning: Emerging Trends in Macroeconomics and Future Prospects

2.1 Introduction

Artificial Intelligence (AI) technologies are widely used in many fields. Deep Reinforcement Learning (RL) as a subset of AI has been widely used in many fields, including robotics, autonomous driving, computer vision, and Natural Language Processing (NLP), and has made significant advances in recent years. These advances are driven by the combination of various methods in Deep Learning (DL) and RL, a branch of Machine Learning (ML) that deals with how agents can learn from the environment through interactions to maximize their reward. This combination can

provide RL algorithms with more expressive power, making them more capable of learning and generalizing from data than conventional machine learning approaches.

Deep RL models can learn from multi-dimensional and continuous state-action spaces, and non-stationary complex and changing environments. These models also have the ability to improve their own learning algorithms through a process known as meta learning—a machine learning technique that is also known as "learning to learn", "learning from experience", or "learning by doing".

The application of RL and deep RL in economics has been limited until recently, mainly due to the difficulty in modeling the environment, designing optimal reward functions and solving the high dimensional economic models. There are, however been a surge of research including work by Chen et al. [2021], Ashwin et al. [2021], Hinterlang and Tänzer [2021], and Curry et al. [2022].

In this chapter, I discuss recent applications of deep reinforcement learning algorithms in economics while introducing the main theory of RL and deep RL— in relatively non-technical terms. Then, with a focus on macroeconomics, I explore the prospects of this class of algorithms and their open issues.

Through discussing several use cases of deep RL algorithms in economics[1], this chapter aims to shed light on this class of algorithms: how it can be used to answer existing questions and encourage a new venue of research.

[1]Mosavi et al. [2020] focus on the applications of neural networks rather than reinforcement learning algorithms. Charpentier et al. [2020] focus on economics and finance, however not many actual cases of applications are included in their paper. Athey [2018] discusses the impact of machine learning on economics and highlights the importance and caveats of using machine learning methods. She focuses on supervised and unsupervised machine learning methods without many discussions on reinforcement learning algorithms. Fischer [2018] provides a survey of applications of reinforcement learning techniques in financial markets.

2.2 What is Reinforcement Learning and Deep Reinforcement Learning?

RL is a type of machine learning that allows agents to learn in an interactive environment by taking actions and receiving rewards for their actions. Deep RL is a type of RL that uses deep neural networks to approximate its value or policy functions which are then used to guide the agent's decisions.

RL is different from both supervised learning and unsupervised learning in the literature of machine learning. Supervised learning requires a labelled training dataset so that it can be used to measure how well a supervised machine learns to predict the labelled output. Unsupervised learning methods are mainly for data without labels in order to classify or find a structure in the data. RL, however, does not require a training dataset as seen in supervised or unsupervised learning. It learns from the interactions with what is called an "environment". The agent learns through a reward function, which is a feedback mechanism that tells the algorithm if it is doing the right thing and tries to find the best possible response in that environment. RL algorithms generate simulation data from having AI agents[2] actively interact with the environment and the simulated data or learning experience is used to solve the RL problems.

2.2.1 Brief History

RL has its roots in behaviorism and was first proposed as a way to explain animal learning behavior. RL is heavily connected to psychology and neuroscience. The early development of RL involves the works of Alan Turing, Norbert Wiener, and

[2]I define the term "AI agents" as referring to agents that exhibit behavior based on deep RL algorithms.

Richard Bellman. However, it wasn't until the 1970s that reinforcement learning began to be studied extensively by computer scientists.

Two segments of literature have played a significant role in the development of RL dating back to its infancy: the first one is the development of Temporal Difference (TD) updating and trial-and-error learning. The second is the literature of optimal control, which involves dynamic programming [Sutton and Barto, 2018]. TD updating means learning from experience by using the difference between expected values and actual ones. Trial-and-error learning means that a learning agent tries different actions in order to find the best action or maximum reward for a given situation. Trial-and-error learning has close links to the psychology literature on animals learning.

In the early AI literature, researchers in engineering explored the idea of trial-and-error. Minsky [1954] and Farley and Clark [1954] were among the first two researchers who investigated the use of trial-and error computational techniques. Minsky [1961] started using the term "reinforcement" and "reinforcement learning". His paper discussed one central problem in reinforcement learning, the credit assignment problem, i.e., how do you distribute credit for success among the many decisions that may have been involved in producing it?

In the late 1950s and early 1960s, RL was further formalized in work by Bellman, which became the foundation of dynamic programming. Optimal control was first used to describe the problem of designing a controller to minimize a measure of the behavior of a dynamic system over time. Bellman [1957] was among the first to develop a solution to solve this problem, extending an early theory of Hamilton and Jacobi. Bellman's approach uses an optimal return function, known as the value function or Bellman equation; and optimal control problems were solved by solving this equation. **?** combined trial-and-error learning with TD learning, and he linked

trial-and-error learning to the psychology of animal learning. This was extended by Sutton and Barto [1981] who described learning rules as driven by changes in temporally successive predictions (hence temporal difference updating). They also developed a psychological model of classical conditioning based on TD learning.

In the 1970s and early 1980s, RL was combined with control theory and adaptive control to describe the behavior of agents. In the late 1980s, RL reached the machine learning community, where it was applied to problems such as simple mobile robots and chess. Watkins [1989] combined the TD learning and the optimal control literature in his development of Q-learning algorithm, which was the first modern RL algorithm.

In the early 1990s, RL was applied to simple problems such as balancing an inverted pendulum and playing backgammon. In 1995, Tesauro et al. [1995] proposed the use of TD learning for playing the game of backgammon and showed that it could achieve human-level performance. Schultz et al. [1997] explored the connections between TD learning and neuroscience. In the same year, Wiering and Schmidhuber [1997] proposed an algorithm for learning complex control policies for high-dimensional robot control tasks. In the same year, Barto and Sutton [1997] proposed an algorithm called State-Action-Reward-State-Action (SARSA).

In the mid-1990s, RL was combined with function approximation. In the late 1990s, RL was combined with Artificial Neural Network (ANN), which allowed it to be applied to problems such as learning how to play video games. In the early 2000s, RL was combined with evolutionary computation, which allowed it to be applied to problems such as learning how to control a robot arm.

And finally in the late 2000s, RL was combined with DL, which allowed it to be applied to problems such as learning how to play video games or navigate 3D envi-

ronments. Since then, deep RL has seen growing interest and has been applied to tasks in robotics, healthcare, finance, and many other fields. Recent applications in the area of economics are discussed in the remainder of the chapter.

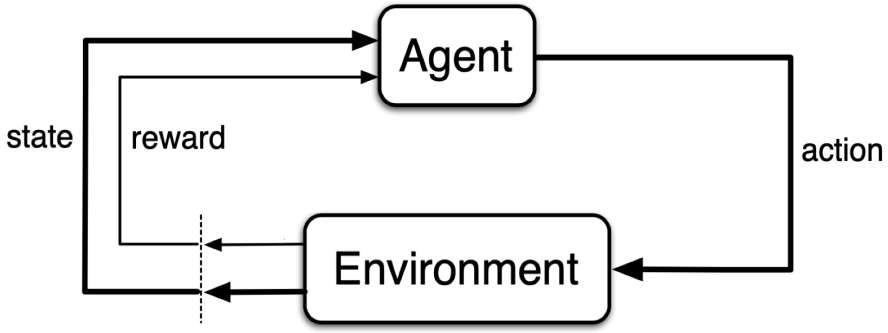
2.2.2 Theory

A Reinforcement Learning Problem and Terminologies

RL involves a class of algorithms that aims to solve an Markov Decision Process (MDP). An MDP is a mathematical framework for modeling decision-making in situations where outcomes are uncertain; partly random and partly under the control of a decision-maker. An MDP is a model that helps an agent take the best decision in any given state by evaluating the immediate and longer term rewards. The environment in RL is usually modeled as a Markov Chain, which means that the next state of the environment only depends on the current state and not on the past states. This makes the problem much simpler to solve at the cost of simplifying learning processes in real world situations.

Figure 2.1 shows an MDP. Given a state that represents the environment, an AI agent takes an action, and the state transitions to the next in part due to the agent's action. The agent then receives a reward.

Figure 2.1: A Markov Decision Process (A Reinforcement Learning Problem)



Source: Sutton and Barto [2018]

Table 2.1 summarizes the terminology used in RL. The state is a representation of an environment, drawn from a state space. An action denotes the agent's choices, drawn from an action space. The reward is a function that depends on the state and the action, which is a stimulus to guide the agent's decision and learning. A policy function, can be either stochastic or deterministic, is the decision-making strategy of the RL agent. A stochastic policy returns a probability distribution of actions given states. A deterministic policy returns a single action given states. Action-value function, taking the input of state-action pair, gives the cumulative rewards (in expectations).

Table 2.1: Terminologies in Reinforcement Learning

Terminologies	**Description**
State, s	A representation of an environment, a random variable drawn from a state space, $s \in \mathcal{S}$
Actions, a	Behavior of the AI agent, a random variable drawn from an action space, $a \in \mathcal{A}$,
Rewards, $R(s, a)$	A stimulus sent to the AI agent in part due to its action and the current state
Policy function, $\pi(s)$ **or** $\mu(s)$	Decision making strategy of the agent, a mapping from state to action (deterministic policy, $\mu(s)$) or a distribution of actions (stochastic policy, $\pi(s)$)
Value function, $Q(s, a)$	'Expected' (subjective belief) cumulative rewards, a mapping from a state-action pair to the expected value

At each time step t, an AI agent receives some representation of the environment's state (a random variable) out of a state space, $s_t \in \mathcal{S}$, and on that basis selects

an action out of an action space, $a_t \in \mathcal{A}(s_t)$. One time step later, partly as a consequence of its action, the agent receives a numerical reward based on a reward function, $r_t = R(s_t, a_t)$, and finds itself in a new state, s_{t+1}. The new state then feeds into another loop of the agent-environment interactive process. To describe how the state transits, a three-argument function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is defined as

$$p(s_{t+1}|s_t, a_t) \equiv Pr\{s_{t+1}|s_t, a_t\} \tag{2.1}$$

for all $s_t, s_{t+1} \in \mathcal{S}$ and $a_t \in \mathcal{A}(s_t)$. It shows the probability of transitioning to state s_{t+1} from state s_t, taking action a_t.

The AI agent's behaviours are defined by a policy, which can be either stochastic or deterministic. If an agent follows a stochastic policy π at time t, then $\pi(a_t|s_t)$ is the probability of choosing action a_t given a state s_t. If an agent is following a deterministic policy μ at time t, then $\mu(s_t)$ gives a deterministic action for an realised state s_t.

Expected returns are defined by a value function, which estimates how good it is for an agent to perform a given action in a given state [Sutton and Barto, 2018]. It is denoted as $Q^\mu(s_t, a_t)$, which shows the expected return after taking an action a_t in state s_t and thereafter following policy μ[3].

$$Q^\mu(s_t, a_t) \equiv \tilde{E}[G_t|s_t, a_t] \tag{2.2}$$

[3]It can also be defined in terms of a stochastic policy π. Given that the algorithm adopted later follows a deterministic policy, here only the value function in terms of a deterministic policy μ is introduced.

where $G_t = \sum_{k=0}^{\infty} \beta^k r_{t+k}$, and it is the sum of discounted future rewards. $\beta \in [0, 1]$ represents a discount factor. $\tilde{E}$ denotes the subjective expectation of this agent. Equation 2.2 also follows the Bellman recursive relationship,

$$Q^{\mu}(s_t, a_t) = r(s_t, a_t) + \beta \tilde{E} Q^{\mu}(s_{t+1}, a_{t+1}), \tag{2.3}$$

where $a_{t+1} = \mu(s_{t+1})$.

An optimal action-value function is defined as

$$Q^{*}(s_t, a_t) \equiv \max_{\mu} Q^{\mu}(s_t, a_t), \tag{2.4}$$

for all $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}(s_t)$. For the state - action pair (s_t, a_t), this function gives the expected return for taking action a_t in state s_t and thereafter following an optimal policy.

At the outset, an AI agent does not know anything about the environment. This means it does not know how a reward r_t is generated. The agent gets to know its own preferences by trying out different options and observing the rewards. The agent also does not know the true probabilities, i.e., $p(s_{t+1}|s_t, a_t)$ in its environment. It forms a subjective belief based on its past experience, and uses this belief to guide future decisions.

An AI agent's task is to make decisions, i.e., a policy π or μ, that maximises its expected returns. This expectation is a subjective belief of this agent and needs not be based on the true probabilities of the underlying processes.

Solving for RL rules can involve both tabular methods and approximation meth-

ods. When the policy or/and value functions become difficult to track with tabular methods, function approximators are usually applied. ANNs are examples of such approximations. The resulting class of algorithms are classified as deep RL.

Exploration vs Exploitation

An AI agent faces a trade-off between exploitation and exploration. Exploration refers to the process of trying out different actions in order to find the best possible action which allows the agent to sample the environment, thereby providing information that helps the agent to find the optimal policy. Exploitation on the other hand refers to the process of taking the best action that is known and allows the agent to reap the maximum possible reward assuming that the agent has found the optimal policy.

The trade-off between exploration and exploitation plays a critical role as it influences the speed at which an agent discovers the best action and learns the optimal strategy while maximizing rewards. If the agent leans towards excessive exploration, it may fail to utilize the knowledge it has acquired and thus struggle to gain useful insights. Conversely, insufficient exploration limits the agent's ability to uncover the optimal solution.

In this thesis, the interpretation of exploration diverges from the conventional wisdom found in RL literature. Rather than viewing it as a means to discover the optimal solution, exploration is perceived as an inherent characteristic of human beings. Some individuals possess a natural inclination towards exploring unknown spaces, while others exhibit a reluctance to do so. The agent's adaptability to a changing environment is significantly influenced by the balance between exploration and exploitation. The concept of exploration will be explored in greater detail in

There are several ways to balance exploration and exploitation. One simple way to address the exploration exploitation dilemma is to use an exploration strategy that encourages the agent to explore more in the beginning and then slows down the exploration as the agent learns more about the environment. This is known as an exploration schedule.

Similarly, one simple way to encourage exploration is to use an exploration bonus. This is an extra reward that the agent gets for exploring new states. This can be used to encourage the agent to try new things and to explore more of the state space.

Amin et al. [2021] survey different existing exploration strategies for solving RL problems. Two main strategies include ϵ- greedy and noise perturbation.

ϵ-greedy with probability ϵ, an AI agent chooses actions randomly (exploration), and with probability 1 - ϵ, the agent chooses actions greedily (exploitation). It is a simple and easy to implement method and used in algorithms such as Q-learning[4]. It, however, can be inefficient in complex problems that involves large state action spaces or continuous action spaces.

Noise perturbation: This involves adding a noise term to a deterministic policy, which could take the following form,

$$a_t = \mu(s_t) + \mathcal{N}_t \tag{2.5}$$

[4]Q-learning is an off-policy model-free learning algorithm, meaning that the agent does not need to follow the current policy in order to learn. This is advantageous because it allows the agent to explore different policies and find the one that leads to the highest rewards. The algorithm works by first observing the environment and then choosing an action based on a set of rules. The agent then receives a reward based on the action taken. The agent then uses this information to update the set of rules that it uses to choose actions.

Equation 2.5 means that the action of an AI agent is the output of its deterministic policy plus a noise term to explore the actions space. The noise can be sampled from a Gaussian process or an autoregressive process. This strategy can be used for RL problems with continuous action spaces.

Model-free vs Model-based Algorithms

Model-based algorithms are those that require a model of the environment. A classic example that is used in macroeconomics is dynamic programming, in which state transition probabilities are fully present. Model-free algorithms are those that do not require a model of the environment. they are also called learning from experience.

RL algorithms have the advantage of learning in a model free environment, which is for an environment with unknown transition probability distributions. Many currently applied RL algorithms follow a model-free approach, e.g., temporal-difference, policy gradient and actor-critic. Advanced deep RL applications commonly benefit from model-free algorithms, among them Deep Deterministic Policy Gradients (DDPG), Deep Q Networks (DQN), Proximal Policy Optimization, Trust Region Policy Optimization, Advantage Actor-Critic and Asynchronous Advantage Actor Critic.

The criteria that determine the reasons for using one or the other learning algorithms in RL could be related to a variety of factors including the size of the state space, the computational constraints, or the types of problems to be solved. For example, model-free RL algorithms are more easily used for problems with large state spaces. Other factors that could affect the choice of learning algorithm include the amount of prior knowledge about the environment, the types of feedback available, and the

types of reinforcement signals that are used. In the subsequent paragraphs, I present several classical algorithms in the field of RL.

2.2.3 RL Algorithms

Temporal Difference Updating: One Step Ahead

Temporal Difference updating is an important milestone in the field of RL. As Sutton and Barto [2018] argue, "if one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal difference (TD) learning". They also argue that TD learning is powerful because it can learn from raw experience without a model of the environment and can bootstrap, which means it can learn from incomplete sequences. TD learning also has close connections to biology; it has been argued that the brain may use TD learning to do credit assignment [O'reilly and Munakata, 2000].

TD learning combines both the ideas of Monte Carlo and dynamic programming. "Temporal" refers to the fact that the agent is learning from intermediate rewards at every time step, rather than waiting for a terminal reward. "Difference" refers to the fact that the agent updates its values by making an estimate of the value of a state by bootstrapping from the value of the next state using the difference between the expected reward and the actual reward. This way of updating allows an AI agent to learn and update at every period. The update is done through calculating a TD-error term. A one step TD-error term is defined as follows.

$$r_{t+1} + \beta Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \tag{2.6}$$

It shows that the TD-error term is composed of the reward r_{t+1} plus a discounted future value $\beta Q(s_{t+1}, a_{t+1})$, minus the current value $Q(s_t, a_t)$.

There are two main types of TD learning: State–action–reward–state–action (SARSA) and Q-learning. In SARSA, the agent learns by taking an action, observing the next state and reward, and then taking another action. In Q-learning, the agent learns by taking an action, observing the next state and reward, and then taking the best action given the learning stage.

Action-Value Approach: Q-learning

Action-value approach includes methods that focus on updating the action value function. AI agents in this approach first learn the values of actions and then select actions based on their estimated action values. By estimating the expected reward for each state-action pair, the agent can learn which actions are most likely to lead to the highest rewards, and then choose the best action accordingly. One prominent example of an action-value approach algorithm is Q-learning. Q refers to an action-value function that takes in a state and an action, and outputs a numeric reward. The goal of the algorithm is to find an optimal action-selection policy, i.e., a policy that maximizes the expected cumulative reward.

Q-learning algorithm was introduced by Watkins [1989]. It uses the TD-updating introduced in the previous section. The central updating/learning equation is defined as,

$$Q(s_{t+1}, a_{t+1}) = Q(s_t, a_t) + \nu[r_{t+1} + \beta \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \tag{2.7}$$

Equation 2.7 says that the updated Q function, i.e., $Q(s_{t+1}, a_{t+1})$, depends on the existing Q function $Q(s_t, a_t)$, plus a scaled (by a parameter ν) updating term, which is the TD-error term.

In this case, Q function is an approximate of the optimal action-value function, independent of the policy being followed. Policy function here influences which state-action pairs are visited and updated. The main procedure in this algorithm is to map different pairs of state-action with a value that approximates the expected return. For a learning agent to find optimal behaviors, it needs to compare Q values for different state-action pairs, which can be costly and inefficient in cases with a large state and action space. It also faces difficulty in learning when the action space is continuous.

When ANNs are used to approximate the action-value function, the resulting algorithm is called DQN. In fact, DQN is an algorithm that combines Q-learning with deep learning. In Q-learning, the action-value function is approximated using a table, meaning that it is essentially memorized and is represented as a set of values for every possible state-action pair. In contrast, in deep learning, the approximation is done using ANNs with many hidden layers in a neural network. DQN is an off-policy algorithm, meaning that it can learn from data that is not necessarily generated by the algorithm itself. DQN was first proposed and patented[5] by Google DeepMind researchers [Mnih et al., 2013]. The algorithm was used to successfully train an agent to play a variety of Atari games.

[5]https://patentimages.storage.googleapis.com/71/91/4a/c5cf4ffa56f705/US20150100530A1.pdf

Policy Gradient Approach

In a policy gradient approach, an AI agent learns the policy function parameters. The learning is based on the gradient of a performance measure, call it $J(\theta)$, where θ denotes the policy function parameter. To maximize performance measure $J(\theta)$, the updating of θ is done through gradient ascent on the measure J.

$$\theta_{t+1} = \theta_t + \xi \nabla \hat{J}(\theta_t) \tag{2.8}$$

In Equation 2.8, $\nabla \hat{J}(\theta_t) \in \mathcal{R}^d$ is a stochastic estimate whose expectation approximates the gradient of the performance measure with respect to its argument θ_t.

To put it simply, the agent starts with some initial policy function parameters. It then interacts with the environment, and at each step, it receives a state and chooses an action according to the policy function. After taking the action, it receives a reward and a new state. The agent then updates the policy function parameters using the gradient of the expected total rewards with respect to the policy function parameters. The process is repeated until the agent converges to an optimal policy function.

One example of a policy-gradient algorithm is REINFORCE. REINFORCE uses Monte Carlo sampling to estimate the gradient of the expected return with respect to the policy parameters. REINFORCE can be used with any parametric policy, such as a neural network.

REINFORCE has a number of advantages over other policy gradient algorithms. REINFORCE is relatively simple to implement and can be used with any parametric policy. REINFORCE is also efficient, as it only requires a single pass through

the data to estimate the gradient. However, REINFORCE also has a number of disadvantages. REINFORCE is biased, as the gradient is estimated using Monte Carlo sampling. This can lead to poor performance on complex tasks. REINFORCE is also slow, as it requires a full pass through the data to estimate the gradient. Castro et al. [2020] adopt the REINFORCE algorithm in searching for an optimal solution for a bank's participation in the high-value payment system. REINFORCE has been used in a variety of tasks, including robotic control, and natural language processing.

Actor-Critic Approach

Actor critic in reinforcement learning refer to methods that learn a value function and a policy function jointly, using the gradient of the value function to update the policy function. It combines the central updating equations for both an action value function, Equation 2.7, and a policy function, Equation 2.8. These methods are particularly effective in high dimensional environments. One advantage of using the value function gradient to directly update the policy is that it can reduce the variance of the gradient estimate, as compared to using the score function gradient. However, this may come at the expense of increased bias in the gradient estimate, mainly because the value function is being updated in the same direction as the current policy. Another advantage of critic algorithms is that they can learn faster than value-only methods, since they can make use of the information in the value function to guide the learning of the policy. A disadvantage of actor-critic algorithms is that they can be unstable, since the gradient of the value function can be very sensitive to small changes in the environment or the policy. Another disadvantage is that they require more computational resources than value-only methods since

they must solve an optimization problem at each step.

A popular actor-critic algorithm is the DDPG algorithm, first introduced by Lillicrap et al. [2015]. DDPG is an extension of the deterministic policy gradient algorithm, which is itself an extension of the policy gradient algorithm. DDPG is an off-policy algorithm that can be used to learn a continuous control task in an environment with high dimensional state space. In this algorithm, the policy and action-value functions are approximated by their respective ANNs. DDPG has been successful in a number of continuous control tasks, including robotics tasks such as manipulation and locomotion. However, like other actor-critic algorithms, it can be difficult to tune the parameters of the networks. Additionally, DDPG is not guaranteed to converge to the optimal policy. Other examples of actor critic algorithms include Advantage Actor-Critic and Asynchronous Advantage Actor Critic.

RL and the Use of ANNs

ANNs [6] are widely used as nonlinear function approximators. Many RL algorithms use ANN function approximation, which sometimes is called deep RL or neural RL and produce fruitful results in the AI literature.

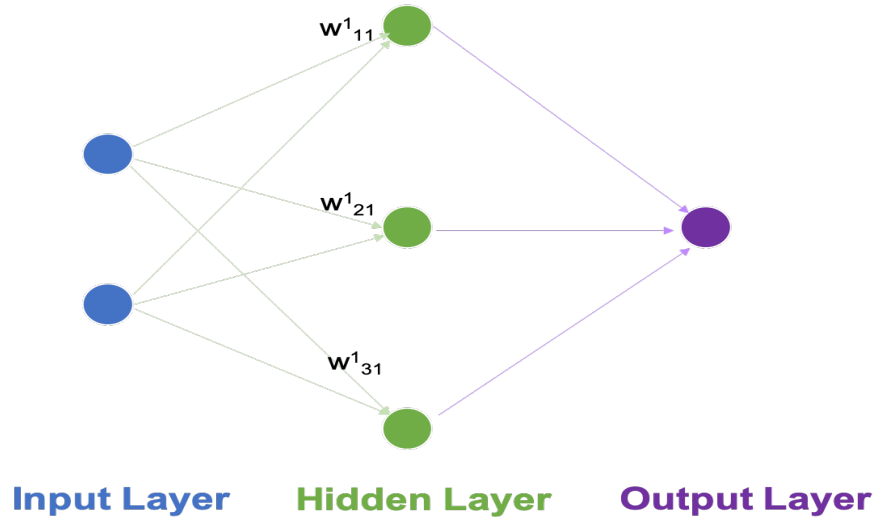
An ANN is a group of interconnected units that are named neurons, which resembles the components of nervous system and are used to process information.[7] The output of the network is determined by the weights of the connections between the units. The pattern of weights is often determined by a learning algorithm. The learning algorithm adjusts the weights of the links in order to minimize the error between the output of the network and the desired output. A simple feedforward neural network

[6] Goodfellow et al. [2016] have an in-depth analysis and discussions on different architectures of ANNs, which are not discussed further here.

[7] For more detailed information on the set up of an ANN, and how ANNs update parameters, please refer to Appendix A.1.

is shown in Figure 2.2.

Figure 2.2: A Feedforward Deep ANN



Source: Authors' construction.

Figure 2.2 network has three layers. It is also known as a deep network, because of the presence of a hidden layer. Information (or data) flows from the input nodes (the left two nodes) and feeds through the neurons in the hidden layer. The filtered information is then passed through the output layer and produce an output.

Deep RL Algorithms: Putting It All Together with Experience Replay and Core Structure

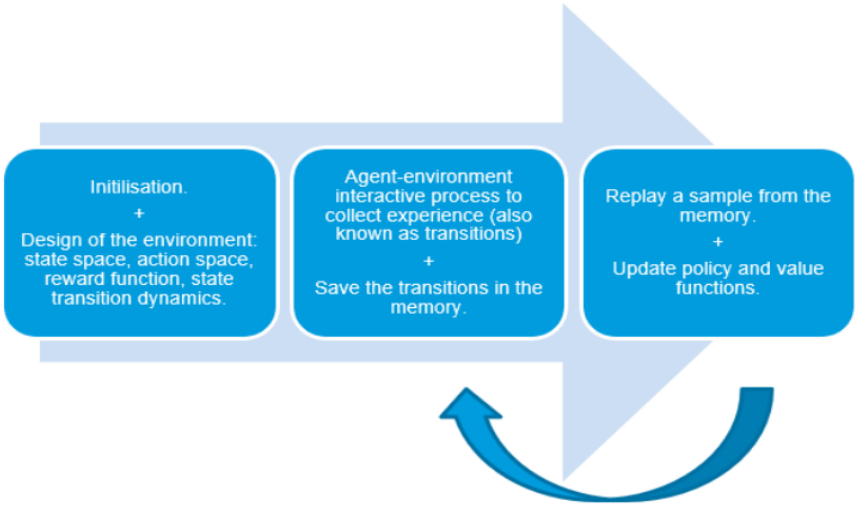
Most deep RL algorithms involve two main components: an agent-environment interactive process to collect experience, and a learning component to update policy and/or value functions. To update functions, a method called experience replay is

used. Experience replay is a sampling method that stores samples from the agent's experience while it is still actively learning and replays them later during the training process. The idea behind experience replay is that by replaying past experiences, the agent can learn from them without being influenced by the current state of the environment. Experience replay is also used to break the correlation between consecutive samples, which can help the agent learn faster and more effectively.

The flow of most deep RL algorithms is as follows:

- Initiate an empty memory
- Set up the environment; design the state and action space; specify the reward function with respect to state and action.
- Agent-environment interactive process to collect experience (also known as transitions) that involve state, action, next state, and reward.
- Save the transitions in the memory.
- Sample from the memory, and update either the action-value function or the policy function or both (updating usually follows Equation 2.7 and 2.8).
- The updated policy and value functions can be used to guide the AI agent's interactive process with the environment further.
- The agent-environment interactive process continues until the task is solved or a terminal state is reached, which is problem dependent.

Figure 2.3: Workflow of Deep RL Algorithms

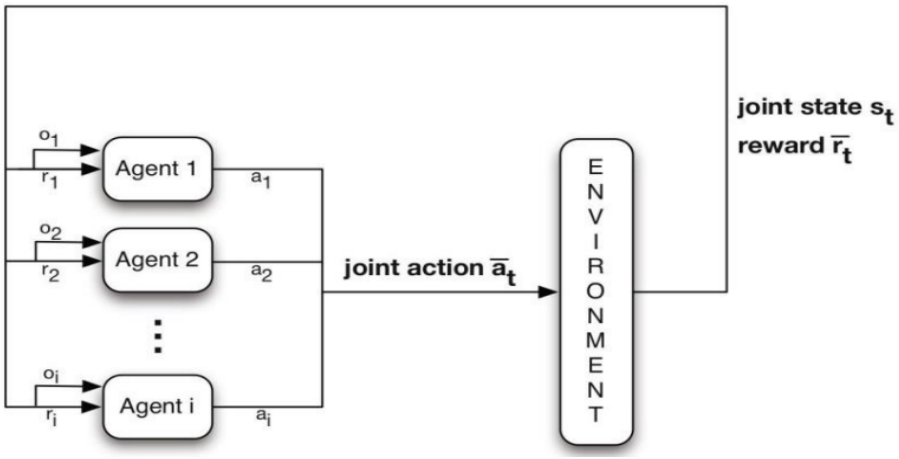


Source: Authors' construction.

Multiagent Deep Reinforcement Learning

Many recent studies have shifted their focus from single-agent RL problems to multi-agent learning scenarios. In contrast to a single-agent RL problem (i.e., Figure 2.1), Figure 2.4 shows a visualization of a multi-agent RL problem.

Figure 2.4: Multiagent RL



Source: Nowé et al. [2012]

In a learning scenario with multiple agents, the MDP is generalized to a stochastic game, or a Markov game. To provide a short description of a Markov game, denote n as the number of agents, $\mathcal{S}$ as a discrete set of environmental states, and A_t, where $t = 1, 2, ..., n$, as a set of actions for each agent i. Define the joint set of actions for all agents as $A = A_1 \times A_1 \times ... \times A_n$. The value function of each agent is dependent on the joint action and joint policy.

Multi-agent learning can be viewed as a distributed optimization problem that requires agents to cooperate with each other to achieve a common goal. The problem of multi-agent learning is further complicated by the presence of non-cooperative agents in the environment who seek to deviate from the commonly agreed upon optimal policy in order to maximize their own rewards.

The key question in multi-agent learning is how the agents interact with each other. The traditional approach to multi-agent learning is to use a centralized training algorithm with a centralized evaluation function. However, this approach does not

scale well to large numbers of agents and can be computationally expensive. An alternative approach is to use a decentralized training algorithm with a decentralized evaluation function.

Decentralized training algorithms are more scalable and can be more efficient, but they often require more communication between the agents. A recent trend in multi-agent learning is to use a hybrid approach that combines centralized and decentralized training. In this approach, the agents are trained using a centralized training algorithm, but the evaluation function is decentralized.

Multi-agent systems can be used to solve complex tasks through the cooperation of individual agents [Nguyen et al., 2020]. Zhang et al. [2021] has a selected overview of currently used algorithms and applications for multi-agent RL problems. Fudenberg and Levine [2007] discuss how multi-agent learning can be applied in game theory. Curry et al. [2022] look into how multiagent RL can be used in a real business cycle model with heterogenous agents categorized as 100 consumer-workers, 10 firms and a government. Hernandez-Leal et al. [2020] provide a survey of multiagent deep RL and its critiques.

2.2.4 Recent Applications

Recent applications of deep reinforcement algorithms range from game playing to robotics and from finance to healthcare and biology.

The DeepMind's AlphaGo program, which defeated a professional Go player in 2016, used a combination of Deep Learning and Monte Carlo Tree Search. DeepMind's DQN algorithm, an early version of which was used by AlphaGo, has been used to train artificial agents to play a wide variety of video games. DQN achieves human-level performance across many of these games.

The advancement of deep RL algorithms does not only mark a major advancement in the field of computer science, but also has significant implications in other disciplines and in the real world. For example, the AI system AlphaFold[8] predicts 3D protein structure, which accelerates the field of biology. Moreover, inspired by psychology and neural science, AI algorithms also help inform and better understand neural structure of human brains and the psychology of decision making. Relatedly, in the field of behavioral marketing, deep RL algorithms have helped to better understand consumer preferences and find the optimal time to place marketing ads. Personalized recommendations systems have benefited from deep RL [Cheremukhin, 2018]. In finance, deep RL algorithms have been used to improve trading strategies [Pendharkar and Cusatis, 2018]. In the medical field, RL algorithms have helped to develop agents for automated diagnosis and personalized therapy [Zhu et al., 2018].

Deep RL algorithms also help minimize energy consumption in the large autonomous data at Google[9]. This AI system helps identify actions that minimize energy consumption while ensure safety constraints are satisfied. The identified set of actions are then sent back to the data center and being implemented. WaveNet is another realworld use of an AI algorithm, and it improves the speech of Google Assistant on Android devices and makes sound more natural[10].

The applications of deep RL algorithms in economics and more specifically macroeconomics are fast growing and range from simple single-agent optimization to complex multi-agent settings, which will be discussed in detail in the following sections.

[8]See `https://www.deepmind.com/research/highlighted-research/alphafold`
[9]See Deep Mind
[10]See Deep Mind

2.3 Deep Reinforcement Learning: Applications and Emerging Trends in Macroeconomics

In this section, I discuss two applications of deep RL in the field of macroeconomics. The first application involves utilizing deep RL algorithms to discover optimal policies or solutions. The second application focuses on examining how deep RL can shed light on bounded rationality, learning, and convergence within macroeconomic frameworks.

RL and deep RL algorithms are relatively new to economics, however economists have been using comparable models for much longer. In Arthur [1991] or Barto and Singh [1991]—two reviews of reinforcement learning techniques in computational economics published thirty years ago—it is possible to find an old economic framework very similar to the one used in reinforcement learning (see, for example, the seminal thesis Hellwig (1973)), (in Charpentier et al. [2020]). Hughes [2014] also analyzed the applicability of reinforcement learning to specific economic topics.

The application of deep RL algorithms in economics has seen some traction in recent years. Charpentier et al. [2020] and Mosavi et al. [2020], reviewing the application of deep and reinforcement learning in finance and economics, have cited some recent use cases (including risk analysis, portfolio management, insurance, pricing and bidding optimization). Deep RL has also been applied to some topics in game theory (see Nowé et al. [2012] and Rajeswaran et al. [2020]), and mechanism design (Zhan and Zhang [2020] and Li et al. [2020]).

Applications of deep RL algorithms in the macroeconomics literature have focused on two areas. First, use of the algorithms as a solution method to find optimal policy or policy response function, such as Hinterlang and Tänzer [2021] and Covarrubias [2022]. This area also extends to solving general equilibrium models, such as Chen

et al. [2021], Hill et al. [2021], and Curry et al. [2022]. Moreover, Chen et al. [2021] also study the learnability of Rational Expectations (RE) solutions in a general equilibrium model with multiple equilibria. The second area of focus is on modelling bounded rationality and transition dynamics, such as Shi [2021a] and Shi [2021b]. This area treats households or consumers as boundedly rational, and studies agents' different behaviors facing shocks in the economy.

2.3.1 Finding Optimal Policies

Hinterlang and Tänzer [2021] make use of deep RL to find optimal interest rate reaction function that fulfils the inflation and output gap targets of central banks. They first use ANN to estimate transitions equations of an economy that is qualified by the three equation New Keynesian model in Rotemberg and Woodford [1997]. In particular, they estimate the aggregate demand equation (i.e., the dynamic investment-saving curve), and the aggregate supply equation. They then use the estimated transition equations as a representation of the RL environment and train the central bank AI agent to find the optimal interest rate reaction function.

They assume that the transition equations do not change over time while the AI agent learns[11]. The central bank's reward is determined by how much it deviates from the inflation and output targets. They show that the transition equations estimated with ANNs match empirical data better than VAR models, and attribute this to the flexibility of ANNs to approximate non-linear functions. They also show that the optimal reaction function reached through RL outperforms common rules used in the literature. They advocate that central bank should add RL algorithms to its toolkit in determining optimal monetary policy reaction functions.

[11]The ANNs used to estimate transition equations should be updated regularly with latest data to take into account the possible behavioral and structural changes in the economy.

Castro et al. [2020] use RL to approximate the policy rules of banks participating in a high-value payments system. The objective of the agents is to learn a policy function for the choice of amount of liquidity provided to the system at the beginning of the day.

They use a real-time gross settlement payments system environment and solve for the best initial liquidity demand of banks through the REINFORCE algorithm. Each day is an episode in their environment. And each day is further divided into T intraday periods. The bank agent starts the day with an initial decision on initial liquidity given the available collateral. The bank then goes through the day by making decisions on sending a payment or not. At the end of the day, the borrowing from central bank for remaining payments can be calculated, and its associated borrowing costs. They make two assumptions of their AI agent. First, agents are risk neutral and will choose their initial liquidity to minimize the total liquidity cost. Second, we assume that the payment demand profile is common knowledge. They train two AI agents concurrently in the same environment, and they make independent actions that are not related to each other. For each training episode, they simultaneously generate multiple trajectories using each agent's current policy, where each trajectory is a complete path of state, action and reward.

Using the experience gathered through multiple trajectories, they update the policy function at the end of each episode. Individual choices have complex strategic effects precluding a closed form solution of the optimal policy, except in simple cases. They show that in a simplified two-agent setting, agents using RL learn the optimal policy that minimises the cost of processing their individual payments. They also illustrate that in more complex settings, both agents learn to reduce their liquidity costs.

Curry et al. [2022] illustrate how deep RL with multiple agents can help discover stable solution that are ϵ-Nash equilibria for a meta-game over agent types. They apply

this multi-agent deep RL system in real-business cycle models, with 100 worker-consumers, 10 firms, and one government who taxes and redistributes. The learnt meta-game ϵ-Nash equilibria are evaluated through approximate best-response analyses. They also show that the resulting learnt policies align with economic intuitions.

They base their research on three main issues faced with existing solution methods of dynamic general equilibrium models. First, linearization is normally required to solve dynamic general equilibrium models through Taylor expansion. However, this approach might be misleading and provide undesirable properties (when nominal interest rate is zero) as argued by Atolia et al. [2010] and Boneva et al. [2016]. Second, formal solutions methods, such as backwards induction and dynamic programming methods, often cannot be solved explicitly and only characterize optimal policies implicitly. Third, curse of dimensionality is an often-encountered problem meaning that the number of states and actions increases exponentially as the number of dimensions increases. This occurs especially for models with a large number of agents. Their incentives are usually misaligned and pose a complex general-sum game, and it remains a challenge to select equilibria in general sum games [Bai et al., 2021].

One common challenge of multi-agent RL setting is that each agent faces a non-stationary environment. Agents are interdependent. Their actions have an impact on other agents. The second challenge is that economic agents typically have private information that is not observed by other agents. Therefore, Curry et al. [2022] state that joint learning can be very unstable in deep multiple agent RL, and moreover, individually trained policies easily get stuck in trivial equilibria where no labor or production occurs.

They develop multi-agent RL techniques that can find equilibria in both a closed and an open real business cycle models. The open RBC model involves a price-taking export market. The ϵ-Nash equilibria for the meta game over agent types are

defined as a set of agent policies such that no agent type can unilaterally improve its reward by more than ϵ. They also show how different factors contribute to which equilibria can be found. These factors include world dynamics, initial conditions and policies, learning algorithm and lastly policy model class.

The RBC model application is abstracted in terms of a normal-form meta game between three players representing all the agent types, i.e., the consumer-workers, the firm, and the government. Their policy evaluation, i.e., if a policy is ϵ-Nash equilibrium for the meta game, is defined as follows. They train each agent type separately for a significant number of RL iterations and holding the policies of other agent types fixed. This is to observe if the training improves the agent's reward by more than ϵ. If it does not, the policy is treated at least a local equilibrium. They also discover that a meta-game best response, as learnt in their setup, may not be the best response for an individual player. There may also be free-riders that benefit from the collective performance while not putting in any effort themselves.

Johnson et al. [2022] study multi-agent RL in a microeconomic environment. AI agents learn to produce resources in a spatially complex world, trade them with one another, and consume those that they prefer. They show that the emergent production, consumption and pricing behaviours can be explained by the supply and demand shifts. They find that when price disparities emerge, some agents learn to take arbitrage opportunities for profit. Their work is part of a research program that aims to build human-like artificial general intelligence through multi-agent interactions in simulated societies. Their model incorporates heterogeneous tastes and physical abilities, and agents negotiate with one another as a grounded form of communication.

Other similar applications include, for example, Jirnyi and Lepetyuk [2011]. They solve an incomplete market model with liquidity constraints through RL. Zheng

et al. [2020] study optimal tax policy in a gather-and-build economy using RL algorithm. Covarrubias [2022] studies the impact of oligopolistic competition on the transmission of monetary policy. He aims to solve for optimal strategies of firms using deep RL algorithms. Calvano et al. [2020] examine the likelihood that the pricing algorithms of multiple sellers trained with RL could result in collusion without the use of any coordination, a potentially challenging problem for competition policy. Igami [2020] examines the similarities and differences between structural estimation (as understood in econometrics) and dynamic programming and RL in the context of two-player, perfect-information, zero-sum games like chess and Go.

2.3.2 Bounded Rationality, Learning and Convergence

Learnability and convergence of the algorithm is studied by Chen et al. [2021], adopting a monetary model with a representative agent that exhibits multiple equilibria. The representative agent learns in the economy with the DDPG algorithm. They show that the AI agent can locally converge to all the steady states described by the monetary model.

In their work, Ashwin et al. [2021] explore the stability properties of multiple equilibria in the context of agents learning with flexible ANNs. They demonstrate that even when the set of rational expectations equilibria (REE) is extensive, there exists a learnable REE in such scenarios, contrary to the prevailing belief that the majority of these REE are not learnable.

Adaptive behaviours of AI agents, and especially the exploration feature of RL algorithms are studied in Chapter 3 and 4. In Chapter 3, I study the consumption-saving behaviors of AI agents in a stochastic optimal growth environment. In particular, I look into the discrepancies in learning behaviors when AI agents are different in

terms of their exploration levels, and how this impact convergence to the optimal policy. I also observe learning behaviors during a permanent income shock.

In Chapter 4, I study AI agent's ability in adapting a monetary policy regime change. I adopt the deep RL framework in a representative agent model with transaction cost of money demand. Through simulations, I observe that the AI agent displays a capacity to adapt its beliefs in response to economic changes, indicating a sophisticated ability to respond to varying circumstances. Although the AI agents exhibit adaptability in response to the structural change, it remains inconclusive whether they ultimately converge to the RE steady state in the new policy regime.

Hill et al. [2021] and Curry et al. [2022] look into cases of general equilibrium models with multiple learning agent. Hill et al. [2021] show how to solve three rational expectations equilibrium models with discrete heterogenous agents instead of a continuum of agents or a single representative agent. These models are precautionary saving model; the interaction between a pandemic and the macroeconomy (an 'epi-macro' model), with stochasticity in health statuses; and a macroeconomic model which has global stochasticity, i.e., where the background is changing in a way that the agents are unable to predict.

Kuriksha [2021] modifies a deep RL algorithm to make it applicable and easier to implement for a large number of households. He studies households' consumption-saving decisions.

2.4 Deep Reinforcement Learning in Macroeconomics: Prospects and Issues

Although deep RL algorithms have been applied to a variety of topics in macroeconomics in recent years, they have been utilized in a limited scope. However, considering the potential of deep RL to provide solutions to high-dimensional complex problems, it seems reasonable to expect their usage to grow exponentially in the coming years.

There are many areas that could be explored by taking inspirations from RL algorithms. This section discusses the protention and some prospects of deep RL in the field of macroeconomics, provides instances of how the existing work at the intersection of deep RL and macroeconomics could be extended, and offers a few suggestions for future research.

Subsequently, I will highlight some issues and challenges in the application of deep RL in macroeconomic settings. This will include a discussion of the issues related to training and choice of algorithms, computational scaling of deep RL models, the identifiability of heterogeneous agent models, the robustness of learning to changes in the environment, difficulty in modeling stochastic environments, and the curse of dimensionality. Other issues related to bias and inference also will be discussed in brief.

2.4.1 Prospects

Forecasting

The application of deep RL algorithms in the domain of forecasting holds great potential and is an area of significant interest. While ANNs have been widely employed for macroeconomic variable forecasting in the past, exploring the use of RL algorithms in this context opens up new avenues for research. Previous studies have already investigated the application of RL for predicting energy usage [Liu et al., 2020] and stock trading decisions [Li et al., 2020]. Considering that macro models typically comprise interconnected micro building blocks and involve interactions among various actors, it is worth exploring whether deep RL algorithms can be employed to predict strategic behaviors. This intriguing possibility, linked to game theory, holds implications for economic policy-making and warrants further investigation.

Macro Simulation

RL offers the possibility of simulating economies with numerous economic agents capable of interacting with each other, akin to the work discussed in Curry et al. [2022]. Through these simulation exercises, it becomes feasible to study agents' behaviors in terms of cooperation and competition. Additionally, there exists a theoretical potential for AI agents to share information with each other, leading to improved welfare through learning. An intriguing avenue to explore is the combination of Agent Based Models (ABMs) with RL. In Sert et al. [2020], the study examines social segregation behaviors using ABMs by modeling agents with deep Q-learning. By integrating RL with ABMs, a synthetic environment is created where policy makers can observe the behaviors of private agents and assess policy impacts. It is

worth noting that Song et al. [2021] also calibrates an ABM using RL algorithms.

Rational Expectations

The application of RL algorithms can contribute to the understanding of learnability and equilibrium selection within rational expectation general equilibrium models. In the study conducted by Chen et al. [2021], the local convergence properties of equilibria are explored. However, more evidence is needed to investigate the convergence property of RL algorithms. More specifically, a key question remains: does global convergence exist in models with AI agents? Answering this question holds important policy implications, as relying solely on analyzing the local properties and policy responses of a single equilibrium might not be sufficient. This becomes especially crucial in light of the growing aggregate uncertainties prevalent in current economic conditions.

Transfer Learning to Model Herding Behaviour, Spillovers or Technology Transfer

RL exhibits a close connection to the field of transfer learning, which aims to enhance the learning process by leveraging knowledge from external sources, as surveyed in Zhuang et al. [2020]. This concept is akin to real-life situation, where agents seek guidance and advice from others. In the context of macroeconomic models, which often focus on a limited number of countries or regions, the utilization of natural language processing can serve as a valuable resource for collecting knowledge and expert opinions. Deep learning models typically have high data requirements, but transfer learning can alleviate this limitation. By training deep learning models on data from one country or region, they can be applied to another, thereby lever-

aging the learned insights from agent-environment interactions. Consequently, the application of RL has the potential to mitigate data requirements. For instance, a deep RL model trained on data from region-I can be used to predict economic recessions in region-II by leveraging the knowledge acquired from the agent-environment interactions in the US data.

Meta Learning

In their work, Sutton and Barto [2018] highlighted the challenge of representation learning and meta-learning, emphasizing the importance of using experience not only to learn about a specific problem but also to acquire inductive biases that can benefit future learning endeavors. Addressing this challenge, Zhang et al. [2022] proposed a deep RL strategy based on meta-learning. Their approach involves initially training a meta-model, which is then fine-tuned through a few update steps to create submodels for associated subproblems. By constructing the Pareto front, their method offers a significant reduction in training time for multiple submodels compared to existing learning-based approaches. The meta-model's speed and flexibility enable the creation of new submodels, enhancing the quality and diversity of solutions. Meta-learning holds potential in specific economic planning scenarios where several subproblems arise within a broader optimization problem. Additionally, it proves beneficial in situations involving the need to learn numerous independent tasks within a short timeframe or where the number of tasks may increase over time due to a changing environment.

Automatic Hyperparameter Optimization and Automated Model Selection and Construction

For any given macroeconomic problem, there may be a range of different models that can be applied. Automatic model selection methods that use deep RL may be able to address this issue. Shang et al. [2020] propose an automatic model selection framework based on reinforcement learning. The framework can be divided into two phases, one is data preprocessing stage, which uses meta-learning to process; the other is model selection framework, including feature preprocessing, feature selection and model selection. These aspects are the environment of reinforcement learning, through which a model with the highest accuracy can be chosen as the predictive model output.

Deep learning models can be used to automatically search (even brute force) and tune the hyperparameters of a neural network in order to optimize its performance [Agrawal, 2021]. This could potentially be useful in situations where it is difficult or expensive to do manual tuning. Similarly, specific RL models could be designed to efficiently search or brute force the model space by trying different macro models on a changing environment or by adding and removing components and variables from models, keeping track of their performance, and automatically or semiautomatically selecting or constructing the most successful ones.

Barriga et al. [2018] present a prototype tool for automatic model repairing by using RL algorithms. They show that RL algorithms could potentially reach model repairing with human-quality without requiring supervision. Laredo et al. [2019] propose a modified micro-genetic deep learning algorithm that automatically and efficiently finds the most suitable neural network model for a given task. Zhu and Yuan [2007] use RL to automate recovery policy generation. In this sense, there's a high chance that RL could also be used for the calibration of macroeconomic models.

Quantum Reinforcement Learning

Quantum reinforcement learning is a relatively new field of study that uses quantum computing to allow RL algorithms to learn faster and more efficiently. Combination of RL with quantum computing (see Dunjko et al. [2017]; Wu et al. [2020]; Wittek [2014]; Biamonte et al. [2017]; Zhang and Ni [2020]) will help automated macro modeling (see above) and drastically improve this class of RL algorithms. Even if deep RL models are currently constrained in terms of processing resources, it might not be the case in the coming years due to the advances in quantum computing (e.g., see Madsen and Lita [2022])

Super Integrated Policy Frameworks

Macroeconomic models can tend to prescribe policies that are onesided, or that focus on one particular goal. Relatedly, macroeconomic models are typically complicated and detailed in one area but oversimplified in the other sectors. Macromodels also could suffer from Lucas Critique that patterns derived from historical relationship are assumed to be permanent – not capturing deeper underlying relationships. Modern macro models attempt to be microfounded and rely on the so-called "deep parameters" in order to circumvent the criticism. However, there is a significant chance that micro-foundations and utility functions in the models are not immune from Lucas Critique because even these deep parameters are not independent of the environments and institutional settings that themselves may be modified and influenced by policies or shocks in the models (Second round Lucan Critique).

RL approach to macro modeling could improve our understanding of what macroeconomic models can do as well as their role in the policy-making by tackling above mentioned problems. First, in the RL models, the environment is not an exogenous

object that is known to the model agents (or even model developers), but it is an object that is actively seen and learned by the agent, as well as something which could be modified by the agent. Second, RL models could bring an epistemic reform to macro understanding, taking into account not only goal-oriented or restrictive assumptions, but all factors that can influence individual economic decisions. Third, in addition to the previous two, there is no theoretical limitation on the agents' perception, except computational power, hence RL models could be highly granulated both in time and space and might serve as the foundation for future hyper interconnected and super integrated policy frameworks.

Interdisciplinary Integration (towards Social Science Integrated Policy Frameworks)

RL models for macroeconomic studies, coupled with significant developments in NLP, could create a ground for a new generation of integrated policy frameworks in which diverse factors from other disciplines could be introduced and studied as the scenario, and agent profiles in an economic model could be more complex, sophisticated and agents' perception more inclusive. This evolution might even lead to a horizon shift and turn the discipline of macroeconomics from the science of gross national product (GNP) and gross domestic product (GDP)— a reduced and one-dimensional understanding of human decision making—into the macro manifest of a broader event, and the science of human welfare, which is a more inclusive, deep and holistic view of human behavior and wellbeing more akin to the social science of a society.

The border between different branches of social science could be increasingly blurred, and ultimately, AI's push to use multiple disciplines to study social behavior could lead to a unified, integrated social science (and a likely integration plus stage after-

ward incorporating human and natural sciences), instead of currently near-separate islands of social studies. Integrated social science frameworks could help us to better identify which policies are more effective in different social, institutional, cultural, religious, and political contexts or even to develop new policies that are tailored to the specific needs of a particular community or region, considering various contextual factors.

Combined NLP-deep RL approaches

Recent advancements in NLP-deep RL (also called deep RL4NLP) aim to use the strengths of both deep RL and NLP for better text understanding. (See Wang et al. [2018]. For a survey see Uc-Cetina et al. [2022]). This could be useful also for macroeconomic modeling. For example, central banks release minutes of their meetings. This text can be scraped and an understanding of the minutes using deep RL4NLP can then be integrated into a macroeconomic model. The same could be done for scraped news articles to predict economic activity. For another example, a Transformer model pretrained on news or search data can be fine-tuned on economic time-series data to improve predictions of economic activity (e.g., predicting changes in the consumer price index using texts produced in the search engine or social media).

Quasi experiments using Causal Deep Reinforcement Learning

Causal RL is a new area of research which is the combination of RL and causal inference (See Gershman and Daw [2017]; Dasgupta et al. [2019]; Grimbly [2020]; Gasse et al. [2021]). In many cases, experiments are not possible to be conducted in macroeconomics or there is no possibility to conduct a counterfactual analysis mainly

due to the lack of data. For example, it is not possible to do a controlled experiment to test the counterfactual scenario of a change in a policy on economic activity. RL can be used in this context. For example, one can use an RL algorithm to simulate different scenarios of a fiscal policy and observe the corresponding macroeconomic outcomes, even in the absence of data regarding the hypothetical reality.

2.4.2 Issues

Deep reinforcement learning algorithms face a wide range of challenges and issues (See Ding and Dong [2020]; Dulac-Arnold et al. [2021] and Du and Ding [2021]), some of which are summarized as follows:

Training Costs and Scalability

Training with a deep RL algorithm requires a long computational time and high computational power. RL algorithms need to generate their own data through agent-environment interactive process, which means it requires a long simulation period to collect sufficient experience to solve a RL problem.

A related issue is scalability. Scalability is the ability of an algorithm to scale up to large environments. This is important in RL because many real-world environments are too large to be handled by a single AI agent. However, scalability is often a difficult problem because RL algorithms often require a lot of data and computational resources. This is in part due to ANNs. ANNs work in high dimensional state and action spaces, however it remains to be a slow learning device and requires lengthy training dataset. This means that for incremental online learning settings, ANNs would struggle to learn rapidly. This issue becomes more prominent for multiagent RL problems. Transfer learning is one way to go to improve learning efficiency.

Reward Function Design

The proper design of a reward function is crucial for successfully training a deep AI agent. An effective reward function should provide clear and consistent feedback to facilitate the agent's learning of the desired behavior. Additionally, it should address the issue of sparse rewards, which occur when rewards are given infrequently. Sparse rewards can hinder an AI agent's learning process by providing insufficient feedback.

Nevertheless, designing a reward function is often challenging and time-consuming. While it may be relatively straightforward to create a function that works for simple tasks, it becomes more complicated for complex tasks. Moreover, designing a reward function that accommodates all agents in a multiagent setting is often difficult, as it may result in rewarding actions that are detrimental to certain agents. Therefore, striking a balance and designing a reward function that works effectively for both simple and complex tasks, as well as in multiagent environments, poses significant challenges.[12]

Stability

Deep AI agents often suffer from instabilities during training. This is due to the fact that they are constantly trying to optimize a complex objective function and are often operating in highly stochastic environments. These instabilities can lead to the agent diverging from the optimal policy and never converging to a solution. Similarly, sensitivity to hyperparameters (e.g., batch size, exploration level, episode length) should also be addressed when applying these algorithms. Stability issues

[12] A related concept is credit assignment. Credit assignment is the process of determining which actions contribute to the goal and which actions do not. This can be a difficult task in complex environments where there may be many different ways to achieve the goal. In addition, credit assignment may be delayed in time, so that an action that contributes to the goal may not be apparent until after the goal is achieved. This can make it difficult for an AI agent to learn which actions are actually helpful in achieving the goal.

could be related to computational aspects of the algorithms as well as the design of the reward function or even the complexity of the environment.

Interpretability, Transparency and Choice of Algorithms

Deep RL algorithms leverage ANNs to approximate policy and value functions. However, the challenge of unpacking and interpreting ANNs remains an ongoing concern within the research community. Likewise, deep AI agents often exhibit opacity, making it challenging to comprehend their decision-making process. This lack of transparency becomes problematic when deploying these agents in real-world settings, where it is crucial to understand the rationale behind their decisions. For instance, policymakers may require insights into why an economic RL model recommends a specific policy.

There are many deep RL algorithms, and the total number is growing quickly. The choice of a particular algorithm depends on the question at hand, and its environment. This includes the design of the state and the action spaces, the reward function, as well as if one is interested in learning the parameters of an action-value function or a policy function or both.

It is also difficult to compare the performance of different deep RL algorithms because there is a lack of standardized benchmarks. Deep RL algorithms are often compared on a small number of tasks, which might not be representative of the algorithm's true performance. In addition, the performance of a deep RL algorithm can vary greatly depending on the specific details of the implementation, such as the choice of hyperparameters.

Ergodicity

Ergodicity in RL refers to the property of a MDP that determines whether all states and actions are reachable from the initial state. A process is said to be ergodic if all states and actions are reachable. A process is said to be non-ergodic if there are some states or actions that are not reachable. The challenge of ergodicity is important because it determines whether or not an AI agent can learn from all possible experiences. If a process is not ergodic, then the agent may never encounter some states or actions, and thus will never be able to learn about them.

Fairness and Safety

While deep AI agents strive to maximize rewards, issues of social fairness and safety arise. Fairness involves balancing individual and collective actions, treating all agents equally, and avoiding biases. AI agents undergo training using deep RL algorithms, where their primary objective is to maximize a given reward function. However, this pursuit of rewards can sometimes result in behavior that is deemed socially unfair, as agents may prioritize their own rewards at the expense of others.

Safety is another critical aspect of AI agents, ensuring that they refrain from taking actions that could lead to undesirable outcomes. Safety concerns are particularly relevant in industrial and health applications, where AI agents control physical actions. Nonetheless, safety considerations extend to other settings as well, where the actions or recommendations of AI agents may have indirect adverse consequences for humans or well-being, such as resource allocation in an economic plan.

Achieving fairness and safety in RL is complex due to the dynamic nature of interactions and inherent uncertainties in the learning process.

2.5 Conclusion

The field of economics has seen a rapid growth in the application of AI algorithms, particularly deep RL algorithms. This chapter provides a comprehensive review of the current state of deep RL in macroeconomics.

To begin, I introduce the fundamental concepts and methods of deep RL. Subsequently, I delve into the recent applications of deep RL in macroeconomic modeling. I find that deep RL models in macroeconomics predominantly focus on two main areas. The first area capitalizes on the model-free nature and flexibility of deep RL algorithms, utilizing them as solution methods for optimizing policies. The second area centers on employing RL algorithms to model bounded rationality, exploring topics such as transition dynamics, learnability of equilibria, and convergence properties.

Additionally, I provide an overview of the potential applications of deep RL models in macroeconomics. These include enhancing forecast accuracy, facilitating macroeconomic simulations, enabling transfer learning, automating hyperparameter optimization and model construction, fostering interdisciplinary macro models, and exploring the realm of causal deep reinforcement learning. Lastly, I discuss the challenges encountered when applying deep RL.

Overall, this chapter serves as a comprehensive exploration of the utilization of deep RL in macroeconomics, encompassing both current applications and future research directions.

Chapter 3

AI for Behavioural Economic Agents: Modeling Adaptability and Exploration in Dynamic Environments

3.1 Introduction

In light of recent advancements in AI technologies, there exist abundant opportunities for their application in the field of economics, as highlighted in Chapter 2. Building upon this foundation, the current chapter introduces a novel framework for modeling human behavior within economic models, employing a deep reinforcement learning approach. This framework serves as an alternative to the current RE assumption and adaptive expectations methodology.

My main contribution is to model economic agent's behaviors using a deep reinforcement learning framework. This framework showcases the interaction between AI agents and the economy, highlighting their ability to adapt their behaviors to maximise long-term rewards. Unlike the RE assumption, AI agents in this approach are not constrained by holding model-consistent beliefs. Furthermore, it diverges from the adaptive expectations assumption by allowing the policy and value functions of AI agents to be adaptive in both functional form and parameters.

As introduced in Section 2.2.3, deep RL contains a class of algorithms. In this chapter, I apply the structure of one particular algorithm, namely the DDPG algorithm that was introduced by Lillicrap et al. [2015] for several reasons. First, it can be applied to continuous action space problems, which is crucial for modeling macroeconomic variables. Second, it is one of the RL algorithms that can handle high dimensional state and action spaces, which are typical in macroeconomic models (e.g., the number of different economic sectors). Third, the actor-critic structure of the algorithm is intuitively appealing and moreover, separation of policy (i.e., actor) and value (i.e., critic) functions in the algorithm allows for analyzing each component independently during the learning process. Finally, The DDPG algorithm has been shown to perform well in a variety of challenging problems in the RL literature.

I create an economy similar to the stochastic optimal growth model, in which the representative agent deviates from the conventional RE assumption and instead follows the DDPG algorithm[1]. This approach requires the AI agent to learn from experience by interacting with the environment, without prior knowledge of the underlying economic structure or its own preferences. The agent makes consumption-saving decisions and receives a reward per period. The agent must try different alternatives

[1]A more interesting case to examine involves heterogeneous agents. However, my goal is to introduce applications of this framework in macroeconomic modeling. Therefore, I start with the simpler case of a representative agent. This approach allows me to illustrate each component of the framework and its implementation in a straightforward general equilibrium model in economics.

and learn from the outcomes, with some agents being more exploratory than others. The exploration mechanism generates different learning behaviors, providing adaptability in a changing environment. This method is intuitive and flexible as humans are different in terms of how exploratory they are with their environment. Some people prefer to try unknown options.

The utilization of a DDPG framework enables the incorporation of an exploration mechanism that closely resembles human behaviors and genetic variations. When individuals inhabit an environment and make decisions, certain individuals exhibit a more exploratory nature, willingly venturing into unexplored choices or decisions. Conversely, others may prefer to adhere to familiar options. Notably, as individuals progress through life, there is a tendency for reduced exploration compared to their younger years. Furthermore, inherent genetic differences exist, with some individuals naturally inclined towards greater exploratory tendencies. The ability to capture such a fundamental aspect of human decision-making is what renders the implementation of DDPG particularly appealing within economic models.

By conducting simulation experiments using the uncalibrated stochastic optimal growth model, I highlight three key findings:

1. AI agents begin in a naive state, unaware of their action set, reward function, or state transition process. As they advance, their behaviors increasingly resemble those of rational agents. However, attaining this close resemblance necessitates a substantial number of simulations.

2. Although the policy and value functions of AI agents differ from those of rational agents, this divergence primarily arises due to the perpetual exploration characteristic inherent in AI agents. By utilising a distance metric, I show that the disparity between the policy function of AI agents and that of RE agents can diminish to around 10% after 50 episodes of updating for the AI

agents in this chapter.

3. The extent of exploration significantly influences the behaviors of AI agents. In the specific example provided in this chapter, the high-exploration AI agent consumes approximately 54% of the total output and saves the remaining portion, in contrast to the low-exploration AI agent, which consumes around 53% of the total output. Despite the similar rates of consumption, the high-exploration AI agent maintains a higher level of savings. Consequently, this disparity in consumption and saving behavior leads to approximately 1.5% lower output production for the low-exploration AI agent.

It is crucial to acknowledge that while agents engaged in high exploration can discover improved strategies, there is an equal likelihood of encountering strategies that are worse than their current beliefs. Consequently, this can lead to welfare loss and heightened aggregate volatility.

The rest of the chapter is organized as follows. In Section 3.2, I discuss relevant literature. In Section 3.3, I introduce AI technologies, discuss the economic model, and the design of an AI agent. In Section 3.4, I present simulation experiments and results, and Section 3.5 is the summary.

3.2 Related Literature

Chapter 1 provides a comprehensive review of the current literature on the applications of deep RL algorithms in economics, along with an introduction to advanced AI algorithms. In this section, the emphasis is on establishing the connection between this chapter and the existing economic literature concerning the concept of bounded rationality.

In his work, Herbert Simon presents the notion of bounded rationality, which encompasses the inherent constraints on human knowledge and computational capabilities that impede economic actors from conforming to economic theories. These limitations encompass various factors, such as the lack of a comprehensive and consistent utility function for assessing choices, the inability to explore all feasible alternatives, and the incapacity to accurately predict the outcomes of chosen options [Simon, 2016]. Simon highlights the significance of the decision-making process itself, shifting attention from solely focusing on the final outcome. He proposes the application of AI to heuristic search and problem-solving by means of pattern recognition [Simon, 2016].

In his work, Sargent [1993] explores the integration of AI and macroeconomic modeling to establish a symmetry between econometricians and economic agents. The agent in the model possesses a learning capability based on a decision rule, eventually converging to a RE equilibrium. The agents' behavior resembles that of professional scientists or econometricians, employing scientific inference methods to gather information and form expectations. Sargent emphasizes the significance of this literature as it delves into the transitional behaviors and dynamics within a learning process

The integration of AI and economics in this study builds upon the perspectives of both Sargent and Simon. In addition to introducing a mechanism for modeling bounded rationality using AI technology, I investigate the role of exploration. Specifically, I analyze how exploration mechanisms impact the beliefs and welfare of economic agents in an economy when they make consumption-saving decisions. The combined use of AI technology and the inclusion of an exploration mechanism has not been explicitly studied in the economics literature. It serves as an important factor in modeling agents' behaviors during structural changes (as explored in Chapter 4) and holds potential for future studies on multi-agent learning systems.

3.3 Methodology

Deep RL algorithms exhibit appealing characteristics such as exploration mechanisms and the ability to learn from experience, making them suitable for modeling economic agents' decision-making processes. In Chapter 2, various deep RL algorithms were introduced, presenting a plethora of options. In this context, I opt to employ the structure of the DDPG algorithm for several compelling reasons[2].

Firstly, the DDPG algorithm is well-suited for addressing continuous action space problems[3], a crucial requirement when modeling macroeconomic variables. Its applicability in this domain holds significant importance. Secondly, it stands out as one of the reinforcement learning algorithms capable of effectively handling high-dimensional state and action spaces, a common characteristic in macroeconomic models (for instance, those encompassing multiple economic sectors). The actor-critic structure of the algorithm is not only intuitively appealing but also facilitates the independent analysis of its policy (actor) and value (critic) functions during the learning process.

[2]Chen et al. [2021] also adopts this algorithm in their study of learnability of the RE equilibrium in different policy regimes.

[3]The DDPG is specifically designed to handle continuous action spaces due to several key features of its algorithmic structure:

- Actor-Critic Architecture: DDPG utilizes an actor-critic approach, where the 'actor' proposes actions given the current state, and the 'critic' evaluates these actions based on the expected future rewards. This structure is particularly effective in continuous spaces because the actor can produce precise actions (as real-valued vectors) rather than having to select from a discrete set of actions.
- DDPG uses a deterministic policy gradient. This means that for any given state, the actor directly computes the optimal action, which can be any value within the action space's continuous range. This direct method is more efficient for continuous actions because it avoids the need to sample from a probability distribution.
- Use of Deep Learning: DDPG leverages deep neural networks for both the actor and the critic. These networks can approximate complex functions, enabling the model to learn and represent the policy and value functions accurately in high-dimensional, continuous spaces. The capability of deep learning models to handle complex, non-linear relationships is crucial for navigating continuous action spaces effectively.

Furthermore, the DDPG algorithm has demonstrated commendable performance across a wide range of challenging problems in the RL literature. However, it is important to note that, similar to other RL algorithms, the DDPG algorithm can exhibit instability in certain settings and may diverge, as evidenced in Atashbar and Shi [2023].

In the remaining part of this section, by providing an overview of the conventional approach employed by economists when modeling the environment. This traditional framework forms the basis for comprehending the adjustments required to tailor it to the context of an AI agent. Subsequently, I introduce the necessary modifications and the detailed DDPG framework.

3.3.1 The Model: an Economist's Approach

I consider a specific example of the stochastic optimal growth model, assuming logarithmic utility and full capital depreciation. It is important to note that this particular specification does not accurately represent the complexities of the real world, nor is it intended for policy experimentation. However, it encapsulates the fundamental decision-making problem in economics, which revolves around making consumption-investment decisions over an individual's lifetime. This model serves as a cornerstone in numerous prominent macroeconomic frameworks, including the real business cycle model and the incomplete market model, to name a few. Therefore, it serves as a natural starting point to showcase the implementation of AI in economic modeling.

Furthermore, this specific model specification possesses an analytical solution, which enables a comparison of the behaviors generated by an AI agent with those of its RE counterpart. By contrasting these outcomes, I can highlight the distinctive and

intriguing behaviors that emerge through the AI implementation, distinct from the patterns observed in the analytical solution derived from RE.

In a closed economy with one consumption/capital good, a representative consumer aims at maximising its lifetime utilities:

$$\max_{\{c_t, k_{t+1}\}_{t=0}^{\infty}} E_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \tag{3.1}$$

subject to the following constraints.

$$c_t + k_{t+1} = z_t y_t \tag{3.2}$$

$$y_t = k_t^{\alpha} \tag{3.3}$$

$$c_t \geq 0 \tag{3.4}$$

$$k_{t+1} \geq 0 \tag{3.5}$$

for all t.

c_t, k_t, y_t are consumption, capital investment, and output produced in period t. In this exercise, I use capital investment and saving interchangeably. Period utility $u(\cdot)$ is increasing and strictly concave, i.e., $u' > 0$, and $u'' < 0$. β is the discount factor. Disturbance to the output, z_t, is a stochastic random variable, and takes the following form

$$z_t = e^{\lambda + \rho ln(z_{t-1}) + \epsilon_t} \tag{3.6}$$

where ϵ_t takes a normal distribution with mean 0 and variance of 0.1, λ is a constant,

and ρ is an autoregressive parameter. One thing worth noting is that changes in aggregate productivity can lead to both an increase in consumption and an increase in capital investment. Since these effects cancel each other out, the autoregressive parameter ρ does not influence the RE solution.

Optimisation under RE

For reference purposes, I present the analytical solution of this model. It is important to note that the solution process under RE, although provided here, is not required for implementing the deep RL algorithm with an AI agent.

The Bellman equation for this problem is:

$$v(k_t, z_t) = \max_{k_{t+1} \in \Gamma(k_t)} \{log(z_t k_t^\alpha - k_{t+1}) + \beta E_t v(k_{t+1}, z_{t+1})\} \tag{3.7}$$

The solution[4] is:

$$k_{t+1} = \alpha\beta z_t k_t^\alpha \tag{3.8}$$

The value function following this policy is

$$\begin{aligned} v^*(k,z) = \frac{1}{1-\beta}\left[log(1-\alpha\beta) + \frac{\alpha\beta}{1-\alpha\beta} log\alpha\beta + \frac{\beta\lambda}{(1-\alpha\beta)(1-\beta\rho)}\right] \\ + \frac{\alpha}{1-\alpha\beta} logk + \frac{1}{(1-\alpha\beta)(1-\beta\rho)} logz \end{aligned} \tag{3.9}$$

[4]See Appendix B.1 for detailed derivation.

3.3.2 The Model: an AI Approach

I now proceed to explain and illustrate the prerequisites for deploying an AI agent within the stochastic optimal growth environment. I provide a comprehensive explanation of the various components involved, starting with the state representation, action space, and reward system. Following that, I delve into the agent's decision-making center, often referred to as its "brain", and emphasize the role of exploration in the agent's decision-making process. I then outline how all these components are interconnected, including the simulation mechanism through which the agent lives and acts in the economy. Additionally, I elaborate on how the agent processes information in order to maximize its long-term reward by employing an experience replay mechanism.

State, Action and Reward

To provide a comprehensive representation of the state, action, and reward aspects, I have augmented Table 2.1 in Chapter 2 with an additional column. The resulting table, Table 3.1, outlines the design of state, action, and reward in this particular economic context.

In this economic scenario, the AI agent initially lacks knowledge or information regarding optimal actions or decision-making processes to maximize both immediate and long-term cumulative rewards. It observes a state variable representing the total available resources at a given period. Based on this observation, the AI agent must decide the amount of resources it wishes to consume[5]. After making this decision

[5]I impose that the proportion of real resources the agent consumes has to lie between 0.005 and 0.995, thus preventing the case of zero consumption unless the total resource becomes 0. Additionally, I impose a rule that corrects any share value that becomes negative — which would lead to negative or zero consumption — to a very small positive value. This correction results in a lesser reward, serving as a deterrent and thereby teaching the AI agents to avoid such scenarios over time.

and engaging in consumption, the AI agent receives a reward.

Table 3.1: RL Components and the Economic Environment

Terminologies	**Description**	**Representation in the economic environment**
State, s_t	A random variable from a state space, $s_t \in \mathcal{S}$	total goods available to consume at period t, $z_t k_t^{\alpha}$
Actions, a_t	A random variable from an action space, $a_t \in \mathcal{A}$	proportion of the total goods that the agent is willing to consume at period t
Rewards, r_t	A function of state and action	utility at period t, $ln(c_t)$ where $c_t = a_t z_t k_t^{\alpha}$
Next State, s_{t+1}	A random variable from a state space	total goods available to consume at period $t+1$, where $k_{t+1} = (1-a_t) z_t k_t^{\alpha}$
Policy function, $\mu(s\|\theta^{\mu})$	A mapping from state to action, $\mu : \mathcal{S} \rightarrow \mathcal{A}$	Approximated by a neural network, ie., actor network; parameterised by θ^{μ} to be updated during learning
Value function, $Q(s, a\|\theta^{Q})$	the 'expected' (subjective belief) return of taking an action in a state	Approximated by a neural network, ie., critic network; parameterised by θ^{Q} to be updated during learning

The reward system is dependent on the level of consumption, with higher consumption resulting in higher rewards. However, maximizing long-term rewards requires finding a balance between present consumption and savings for the future. The income process is stochastic, and if the agent chooses to consume all available resources in the previous period, there is a risk of experiencing zero consumption in the current period if no income is received. This outcome would yield a minimal reward for the agent, as it would not be satisfied with going hungry.

Decision-making and Exploration

In line with the DDPG framework, I employ ANNs to approximate two key components:

- The decision-making process, denoted by $\mu(s|\theta^{\mu})$, which involves determining appropriate actions based on a given state realization.
- The assessment of the desirability of specific state-action pairs, denoted by $Q(s, a|\theta^{Q})$, which involves evaluating the potential long-term rewards associated with following a particular state-action pair.

The former function is analogous to the policy function in economics literature, while the latter is akin to the value function. I have previously introduced ANNs in detail in Chapter 2, Section 2.2.3, and therefore will not focus on their specifics here.

The ANNs' parameters are initially randomly initialized, meaning that given a state variable, the resulting decision or action would also be random. This simulates the process experienced by human beings when navigating an unfamiliar environment, where they initially explore and try things randomly to gain a better understanding

of their own preferences.

However, even after the agent acquires a better understanding of its own preferences, it continues to periodically try random actions as a result of its inherent exploration characteristic. This implies that even if the agent is aware that a particular decision may yield a satisfactory reward, it still adds noise to its decisions and, as a result, encounters options it has not encountered before. This sense of adventure and openness to new possibilities enables the agent to potentially discover better decisions than those it is already familiar with.

Varying levels of exploration imply that agents can possess distinct past experiences despite residing in the same environment. An AI agent can exhibit varying degrees of adventurousness in exploring available actions and their outcomes. A higher exploration level signifies the agent's willingness to engage in untested actions, thereby increasing the likelihood of discovering superior actions in terms of rewards. However, this also entails risks, as the agent may end up in a less favorable position than before. Conversely, a lower exploration level implies that the agent is less inclined to experiment with new approaches and may overlook state-action pairs that yield high rewards.

The exploration feature also enables the AI agent to stay vigilant regarding changes in its environment. Through exploration and experimentation with various actions, the agent has the opportunity to discover, for instance, whether an action leads to different rewards or state transitions from its initial belief. If this occurs, the AI agent can subsequently modify and adapt its beliefs and decision-making. For instance, if there is a change in the autoregressive parameter within z_t (where $z_t = e^{\lambda+\rho \ln(z_{t-1})+\epsilon_t}$), an AI agent equipped with exploration abilities will detect such alterations and adapt its future actions and policy function accordingly.

To incorporate the exploration mechanism, I adopt the same exploration strategy as used in the DDPG algorithm for the reason that it has been applied to a number of problems in the computer science literature, e.g., Lillicrap et al. [2015]. In particular, an exploration policy μ' is created by introducing a noise process $\mathcal{N}$ to the ANN that approximates the policy function. The exploration policy is defined as follows:

$$\mu'(s_t) = \mu(s_t|\theta^\mu) + \sigma_t \mathcal{N}_t. \tag{3.10}$$

Here, $\mathcal{N}_t$ is a sample from a autoregressive process of order 1, i.e., AR(1). It is worth noting that while the noise could be sampled from an uncorrelated Gaussian process, a correlated AR(1) process is typically preferred as it better aligns with the learning agent's tendency to explore its environment in a correlated manner [Lillicrap et al., 2015].

The parameter σ_t in Equation 3.10 corresponds to the scale of the noise $\mathcal{N}_t$ and reflects the level of exploratory behavior exhibited by the agent. It can assume values between 0 and 1, and is capable of changing over time. Initially, during the early stages of the learning process, the exploration level tends to be high as the agent needs to explore a wider range of options in order to gather valuable information. As the agent learns and gains insights into actions that can yield higher rewards, it may gradually decrease its level of exploration over time.

I set the value of σ_t to be greater than 0 for all time steps t, indicating that the agent consistently engages in exploration. This choice is made to capture the inherent exploratory nature of human decision-making in real-life scenarios. In a non-stationary environment where conditions are constantly changing, the agent is capable of recognizing when a previously rewarding state-action pair no longer generates high rewards and can adapt its behavior accordingly. By maintaining

a level of exploration, the agent remains flexible and responsive to changes in the environment, allowing for continued adaptation.

The agent assesses the quality of its decisions based on two criteria: immediate reward and long-term cumulative rewards. It considers a decision-making strategy to be effective if it not only results in a high immediate reward but also leads to a significant increase in long-term cumulative rewards. To quantitatively evaluate the long-term cumulative reward, the agent relies on a second ANN, analogous to a value function.

Let's assume there are two policies, μ_1 and μ_2. The policy μ_1 is considered more desirable if the expected return, as measured by the value function Q, is higher for μ_1 compared to μ_2, denoted as $Q^{\mu_1}(s, a|\theta^Q) > Q^{\mu_2}(s, a|\theta^Q)$. In other words, when faced with the same realized state and action pair, the superior policy generates a higher expected return. The parameters θ^Q encapsulate the agent's subjective beliefs, which evolve over time as the agent learns more about the environment. By continuously updating its beliefs, the agent strives to improve its decision-making strategy and maximize long-term cumulative rewards.

Memory and Experience Replay

The process in which the AI agent observes a state variable, makes a decision, receives a reward, and transitions to the next state forms an experience. The agent logs these experiences by storing the variables (s_t, a_t, r_t, s_{t+1}). It is assumed that the agent never forgets and retains all the experiences encountered.

To enhance its decision-making strategy, the AI agent adjusts and adapts its parameters using a technique known as experience replay. This involves utilizing past experiences as data for parameter updates within two ANNs. The agent employs

random sampling from its memory during experience replay, using these sampled experiences to update its policy and value functions. Depending on the specific learning requirements, alternative sampling methods like selecting from recent or impactful experiences can also be incorporated.

Experience replay can be likened to humans gaining insights from a diverse range of past events, as it enables AI agents to glean knowledge from a broader scope of scenarios. By revisiting previous experiences, AI agents acquire the ability to make more robust and generalized decisions.

This approach to information collection and processing is consistent with empirical evidence on learning from experience and use-dependent brain by Malmendier and Nagel [2016], D'Acunto et al. [2021] and Malmendier [2021].

Full Algorithm and Sequence of Events

In Chapter 2, Section 2.2.3, I provided an overview of the general structure of most deep RL algorithms[6]. Now, I will concentrate on the DDPG algorithm and its application to an AI agent operating within a stochastic optimal growth environment. The DDPG application follows a sequence of three primary steps: initialization, interaction with the economy, and the subsequent updating and adjustment of its decision-making strategy and beliefs.

Step I: Initialisation

- The environment setup involves the definition of several crucial components.

[6]The inclusion of the DDPG algorithm is based on discussions with colleagues at the Bank of England.

These components encompass a state space characterized by bounded and compact properties, an action space that similarly possesses bounded and compact attributes, a reward function that generates rewards based on the combination of state and action variables, and state transition dynamics that govern the evolutionary process from the current state to the subsequent state when an action is undertaken.

- The establishment of two neural networks is integral to the framework.[7] The first network, denoted as $\mu(s|\theta^{\mu})$, assumes the role of a policy function and takes the state as input, generating corresponding actions. The second network, assumes the role of a value function, symbolized by $Q(s, a|\theta^{Q})$, accepts a state-action pair as input and predicts the anticipated return. The parameters of both networks, represented by θ^{μ} and θ^{Q}, are initially assigned random values. During the learning process, these parameters undergo updates to facilitate the convergence of the networks towards optimal policy and value functions.

- A replay buffer denoted as $\mathcal{B}$ is introduced to serve as a memory repository. This buffer efficiently stores information, termed transitions, acquired by the AI agent during its interactive engagements with the environment. A transition is defined as a sequential arrangement of variables encompassing the state at time t, the corresponding action taken (a_t), the obtained reward (r_t), and the resulting state at time $t + 1$ (s_{t+1}).

- The length of a mini-batch, denoted by N, is stipulated to indicate the number of samples extracted from the replay buffer. These samples are employed for

[7] Approximating a function using a neural network requires the following steps: 1. Determine the size of the input and output variables of the function, which serve as the inputs and outputs of the neural network. 2. Specify the neural network's architecture by choosing a structure, such as a feedforward neural network in my case, and selecting the number of layers and nodes in each layer. 3. Initialization: Establish the method for initializing the neural network's weights, which in this thesis involves random initialization. The detailed steps for how the neural network updates its parameters are provided in Appendix A.1.

training purposes.

- The total number of episodes, denoted as E, is explicitly defined. Each episode signifies a distinct simulation period during which the AI agent engages with the environment and acquires knowledge. It is important to note that in contrast to conventional deep RL scenarios, where episodes are characterized by explicit terminal states, economic environments often lack such clearly defined endpoints. Consequently, in the context of this framework, episodes represent the duration of the agent's learning process.

- Lastly, a simulation period of T is established for each episode, with T exceeding the value of N.

For each episode, the following steps are repeated:

Step II: Interaction of the AI agent with the environment for the period $t \leq T$.

- The agent observes the current state, denoted as $s_t = z_t k_t^{\alpha}$, representing the total available goods for consumption at period t. It selects an action, $a_t = \mu(s_t|\theta^{\mu}) + \mathcal{N}_t$, which corresponds to the proportion of goods the agent intends to consume. This action is determined by the current policy defined by the actor network, with an additional exploration noise component.

- The selected action, a_t, is executed, and a reward is obtained based on the utility function, given by $r_t = \ln(c_t)$, where c_t represents consumption. The subsequent state, $s_{t+1} = z_{t+1} k_{t+1}^{\alpha}$, is observed.

- The transition, (s_t, a_t, r_t, s_{t+1}), is stored in the memory buffer, $\mathcal{B}$.

Step III: Training of the AI agent for the period $N \leq t \leq T$.

- A random mini-batch of N transitions, (s_i, a_i, r_i, s_{i+1}), is sampled from the memory buffer, $\mathcal{B}$.

- For each transition i, a value TD_i is calculated as follows:

$$TD_i = r_i + \beta Q^{\mu}(s_{i+1}, \mu(s_{i+1}|\theta^{\mu})|\theta^{Q}), \tag{3.11}$$

  where $Q^{\mu}(s_{i+1}, \mu(s_{i+1}|\theta^{\mu})|\theta^{Q})$ represents the prediction of the critic network for the state-action pair $(s_{i+1}, \mu(s_{i+1}|\theta^{\mu}))$, and $\mu(s_{i+1}|\theta^{\mu})$ represents the prediction of the actor network for the state s_{i+1}.

- The value $Q(s_i, a_i|\theta^{Q})$ is obtained from the value function network, using the state-action pair (s_i, a_i).

- The average loss for the mini-batch of N transitions is calculated as:

$$L = \frac{1}{N} \sum_{i} \left(TD_i - Q(s_i, a_i|\theta^{Q})\right)^2. \tag{3.12}$$

- The critic network is updated with the objective of minimizing the loss function L, utilizing backpropagation and gradient descent techniques.

- For the policy function, represented by the actor network, the objective is to maximize its corresponding value function. This value function, denoted as $Q(s, a|\theta^{Q})$, is aligned with a specific policy, where the action a is obtained from the policy μ as $a = \mu(s|\theta^{\mu})$. The objective function is defined as:

$$J(\theta^{\mu}) = Q^{\mu}(s_i, \mu(s_i|\theta^{\mu})|\theta^{Q}). \tag{3.13}$$

- Alternatively, the objective function can be expressed as minimizing $-J(\theta^{\mu})$. The actor network parameters are updated with the objective of minimizing $-J(\theta^{\mu})$, employing backpropagation and gradient descent techniques.

3.3.3 Training Period Descriptions

In this example, I demonstrate the operation of the algorithm during its initial phase. Initially, the parameters within the neural networks, specifically denoted as θ^{μ} and θ^{Q}, are initialized randomly. This initialization follows the default methods provided by the PyTorch library in Python, which vary based on the type of neural network layers; in this case, I predominantly use linear layers typically initialized based on a uniform distribution.[8]

Starting with an endowment level of 1, this variable serves as the input to the policy neural network. This network, functioning as a non-linear function, outputs a deterministic value within a bounded range of $[0.005, 0.995]$. After incorporating an exploration value, the agent's final action in the initial period is determined to be 0.453. This value represents the proportion of the endowment the agent decides to consume, thus making the consumption amount 0.453. The remaining amount, 1–0.453 = 0.547, is saved. The agent then receives a reward, calculated as $\ln(0.453) = -0.792$. The subsequent state is determined by $0.547^{0.4} \times \text{shock} = 0.758$, with the shock, drawn from a normal distribution of mean 0 and variance 0.1, having a realization of 0.9475 in this period.

This process of recording state, action, reward, and subsequent state sequences, denoted as $(1, 0.453, -0.792, 0.758)$, is stored in an initially empty memory. This interactive process continues over several time periods (in this case, 128[9].), ensuring a substantial historical sequence in the memory for updating the neural networks.

From period $t = 129$ onwards, while continuing the same interactive process and storing experiences in memory, the agent also begins updating the neural networks.

[8] PyTorch offers different default initializations depending on the type of neural network layers. In this thesis, linear layers, which are commonly initialized based on a uniform distribution, are primarily used.

[9] This is the size of a mini-batch introduced in the full algorithm in Section 3.3.2

This process involves:

- Drawing a random sample of 128 sequences from the memory.[10]
- Utilizing each sequence from the sample to calculate the TD error, defined in Equation 3.11 as $TD_i = r_i + \beta Q^{\mu}(s_{i+1}, \mu(s_{i+1}|\theta^{\mu})|\theta^{Q})$.
- Calculating the average loss for this sample as $L = \frac{1}{N}\sum_i \left(TD_i - Q(s_i, a_i|\theta^{Q})\right)^2$, where $i \in [1, 128]$, $N = 128$, and $Q^{\mu}(s, \mu(s|\theta^{\mu})|\theta^{Q})$ represents the output from the value function network for a given state, and $\mu(s|\theta^{\mu})$ represents the output from the policy function network.
- Updating θ^{Q} with the goal of minimizing the calculated loss L, and θ^{μ} is updated aiming to maximize Q. The specifics of gradient descent and back-propagation used for these updates are detailed in Appendix A.1.

3.3.4 Parameters and Learning Agent's Characteristics

The parameters are presented in Table 3.2.

[10] Over time, the agent accumulates more memories, thus I initialize the memory to hold up to 1,000,000 sequences of experiences.

Table 3.2: Main Parameters

Parameters	Values
Output elasticity of capital α	0.4
Shock location parameter λ	0.1
Autoregressive factor ρ	0
Discount Factor β	0.99
Exploration Level σ_t	low - 0.1; high - 0.3

In this economic environment, the parameter values are uncalibrated: the output elasticity of capital is 0.4, the shock location parameter is 0.1, and the discount factor is 0.99.[11] In the baseline environment, the autoregressive factor is set to 0.

Additionally, the exploration level is a characteristic parameter unique to each AI agent. For the AI agent with a low level of exploration, I set the exploration parameter to be 0.1, which is quantified by the noise scale σ_t in equation (3.10). For a high exploration AI agent, the exploration parameter is 0.3.

3.4 Experiments and Results

I conduct a series of simulation experiments to explore the behavior and decision-making processes of AI agents within the stochastic optimal growth model.[12]

[11] I provide the results for a zero shock location parameter in Appendix B.4

[12] The code, written in Python and applicable to the experiments in this chapter as well as in Chapter 4, is available upon request. The general structure of the code is similar to `https://github.com/ikostrikov/pytorch-ddpg-naf/tree/master`.

Initially, I introduce a representative AI agent programmed to engage in a predefined low level of exploration. This setting is subjected to shocks that are i.i.d.. Despite the environment's stationary nature, the AI agent's constant exploration leads to behavior and beliefs that diverge from those of an RE agent. The outcomes of this initial experiment are detailed in Section 3.4.1. Further exploration of agent behavior is pursued through experiments that contrast agents with higher and lower exploration levels, with findings documented in Section 3.4.2.

Subsequent experiments shift focus to the impact of altering the nature of shocks from i.i.d to an autoregressive process of order 1 (AR(1)), transitioning the stochastic process from $z_t = e^{0.1+\epsilon_t}$ to $z_t = e^{0.1+0.7\ln(z_{t-1})+\epsilon_t}$, thereby introducing a persistent change. The insights gleaned from these experiments are presented in Section 3.4.3.

Two aspects are noteworthy. First, all experiments are conducted 50 times, with the average outcome reported for each experiment. Second, the simulation plots are generated for the testing periods only. This approach implies that, during the training periods, AI agents engage in exploration, accumulating interactive experiences and updating their beliefs based on past experiences. However, during the testing periods, the agents neither explore nor update their beliefs. The rationale for focusing on the testing periods in our plots is to facilitate comparisons without the confounding effects of continuous exploration. This allows for a clearer illustration of the AI agents' learned behaviors through gathering interactive experience, especially when comparing these aspects among AI agents of different exploration levels and against the RE agent.

Through the experiments, I present three main findings:

- AI agents begin in a naive state, unaware of their action set, reward function, or state transition process. As they advance, their behaviors increasingly

resemble those of rational agents. However, attaining this close resemblance necessitates a substantial number of simulations.

- Although the policy and value functions of AI agents differ from those of rational agents, this divergence primarily arises due to the perpetual exploration characteristic inherent in AI agents. By utilising a distance metric, I show that the disparity between the policy function of AI agents and that of RE agents can diminish to around 10% after 50 episodes of updating for the AI agents in this chapter.

- The extent of exploration significantly influences the behaviors of AI agents. In the specific example provided in this chapter, the high-exploration AI agent consumes approximately 54% of the total output and saves the remaining portion, in contrast to the low-exploration AI agent, which consumes around 53% of the total output. Despite the similar rates of consumption, the high-exploration AI agent maintains a higher level of savings. Consequently, this disparity in consumption and saving behavior leads to approximately 1.5% lower output production for the low-exploration AI agent.

3.4.1 An RE Agent and an AI Agent in a Stationary Environment with i.i.d Shocks

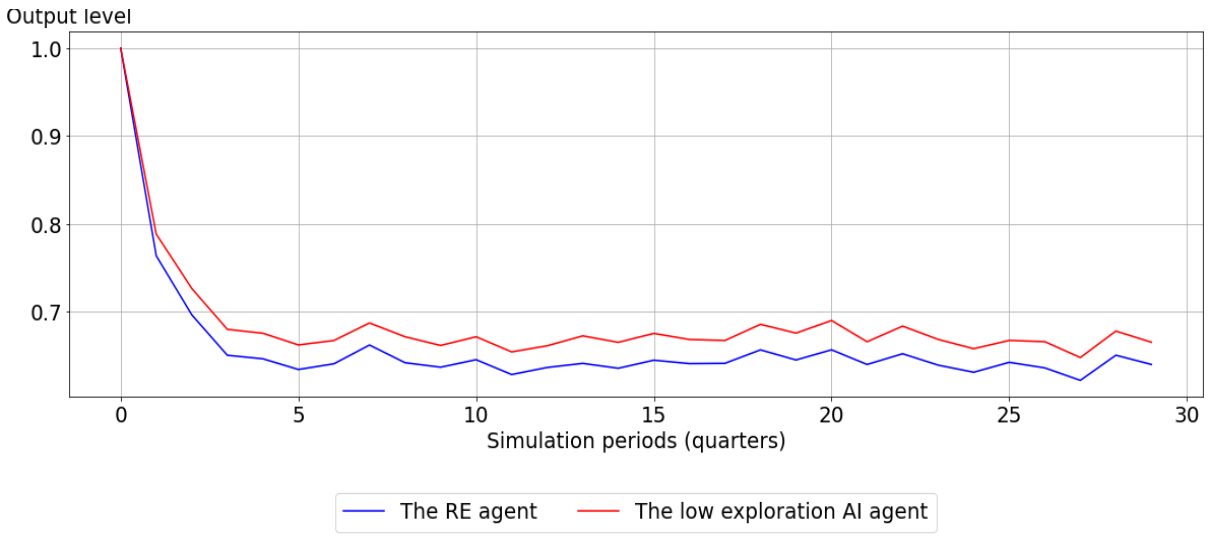
AI agents demonstrate distinctive characteristics compared to an RE agent. While an RE agent is assumed to form model-consistent beliefs instantly, AI agents acquire a profound understanding of the economy through taking explorative actions and accumulating interactive experiences. To explore the implications of these disparities, simulation experiments are conducted, where the behaviors of an AI agent and an RE agent are compared within an economy subject to the same initial condition and identical stochastic shocks.

Specifically, the examination focuses on comparing the consumption paths of both agent types, analysing their respective policy functions, and quantifying the differences between the policy function of an AI agent and that of an RE agent using a distance metric. This investigation aims to shed light on how the two types of agents diverge in their decision-making and how the AI agent's learning process influences its policy behavior in contrast to the immediate model-consistent behavior of the RE agent.

At simulation period 0, I set an identical initial condition[13] so that both the AI agent and the RE agent demonstrate the same initial output of the economy, as illustrated in Figure 3.1. However, a noteworthy disparity emerges in their consumption-saving behavior. In the initial periods, the AI agent exhibits lower consumption than the RE agent while saving more than the RE agent, as depicted in Figure 3.2. Consequently, this results in higher output in subsequent periods.

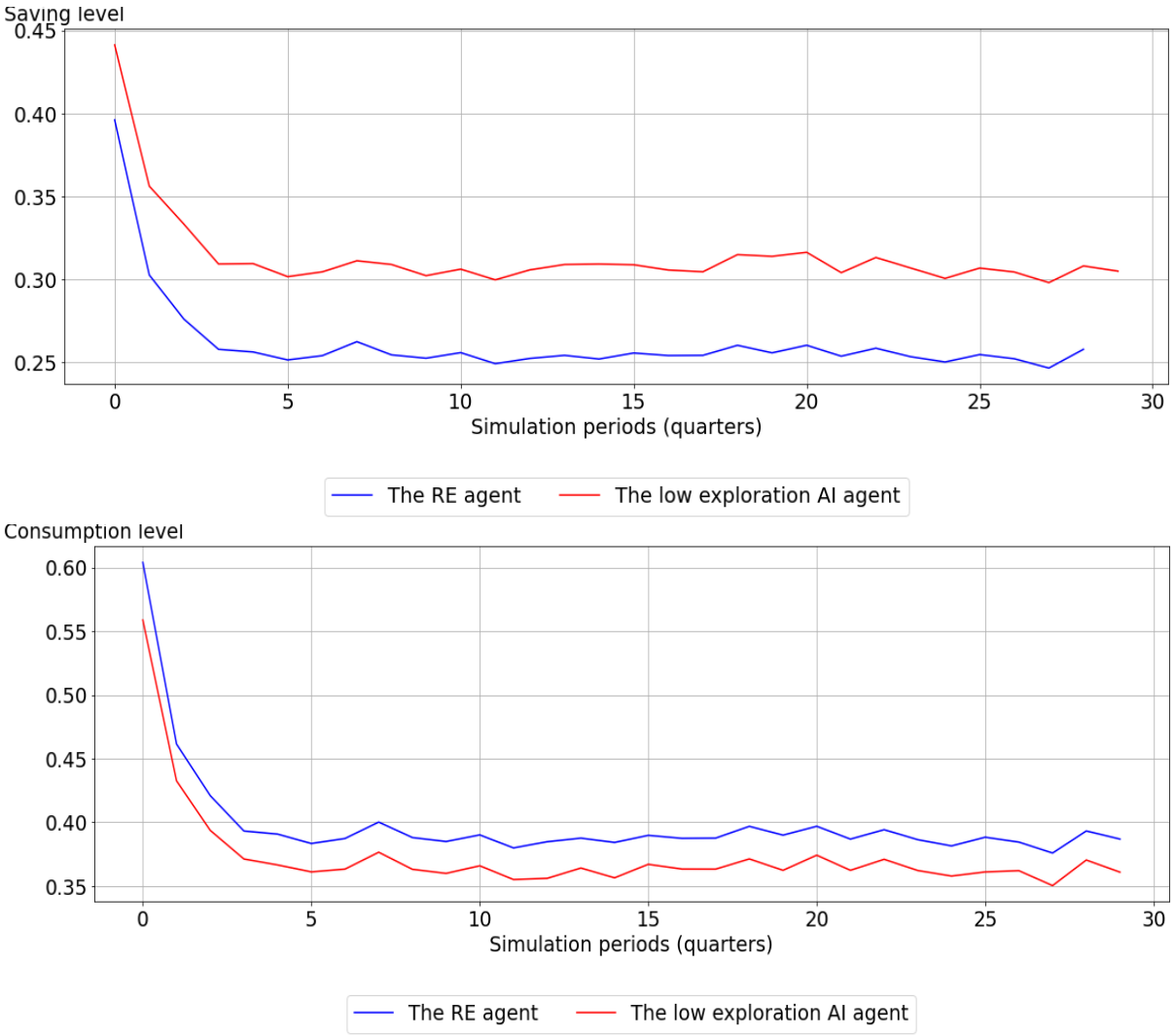
[13]In this exercise, I set the initial condition to be a fixed value of 1. However, I also provide additional simulations where the initial conditions are randomly drawn from a normal distribution. These additional simulations are presented in Appendix B.3.

Figure 3.1: Output Comparison



Notes:
1. x-axis represents simulation periods in quarters.
2. y-axis represents the quantity of production output taking into account the exogenous shock, i.e., total production is $z_t k_t^{\alpha}$, where k_t is the saving/investment of the representative agent.
3. The blue line represents variables of interest for the representative agent RE agent; the red line represents variables of interest for the AI agent who engages in low level exploration.

Figure 3.2: Consumption and Saving levels of an RE agent and an AI agent



Notes:
1. x-axis represents simulation periods in quarters.
2. y-axis represents saving or investment, and consumption levels.
3. The blue line represents variables of interest for the RE agent; the red line represents variables of interest for the AI agent.

The initial findings reveal differences in the behaviors of the AI agent compared to the RE agent. Particularly noteworthy is that the AI agent consumes approximately 5% less of the total output in the initial periods than the RE agent, in contrast to

the RE agent's consumption of 60%.

One possible explanation for the AI agent's higher saving behavior is that it recognizes the need to save more buffer in order to guard against future shocks. Therefore, I observe a higher saving and lower consumption behavior.

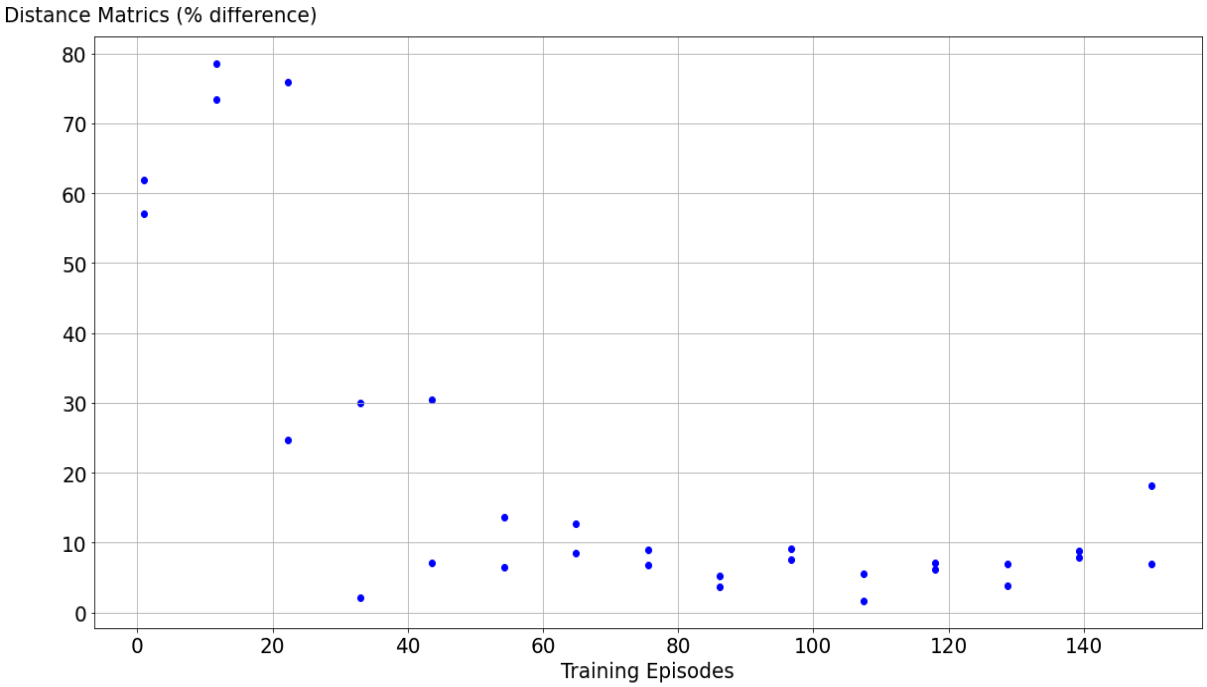
Given the exploratory and adaptive nature of an AI agent, it can be conjectured that its policy function would not be identical to that of an RE agent. However, one should expect to see its policy becoming closer to an RE agent's policy as the AI agent collects more interactive experience and learns how to act in a given environment.

To demonstrate this, I need a metric to quantify the distance between the AI agent's and the RE agent's policy functions. I compute a distance metric for each episode of updating using Equation 4.14. In the equation, k_g^* represents the value of the analytical solution policy function at a specific grid point g, while k_g corresponds to the approximated policy function at the same grid point. The total number of grid points is denoted by G, and d_e represents the distance between the analytical solution and the approximated policy function at episode number e. This distance metric allows us to measure the convergence of the AI agent's policy function to the analytical solution over the course of its updating episodes.

$$d_e = \frac{1}{G} \sum_{g=1}^{G} \frac{|k_g^* - k_g|}{k_g^*} \tag{4.14}$$

Next, I generate a plot illustrating the calculated distance across training episodes, as depicted in Figure 3.3.

Figure 3.3: Distance Metrics



Notes:
1. y-axis represents the percentage distance according to Equation (4.14).
2. x-axis represents how long the AI agent has been living and updating its policy function. Each episode encompasses 5000 steps of updating policy and value functions.
3. A downward trend can be observed as the AI agent learns more in the economy.
4. Reaching a stage where the AI agent's behaviors closely resemble those of an RE agent requires a substantial number of simulation periods.

In Figure 3.3, training episodes are indicative of the AI agent's duration of living and updating its policy and value functions. As the number of training episodes increases, the AI agent accumulates more interactive experiences and gains additional opportunities to update its policy and value functions. The y-axis represents the calculated distance, denoted as d_e in Equation 4.14.

As the AI agent gains more experience, and updates its policy function, the agent's decision gets similar to an RE agent's decision. For example, as Figure 3.3 shows, at training episode 50, the AI agent's decision is around 10% difference from the RE

agent's decision, reducing from around 60% at the initial periods.

After comparing the behaviors of the AI agent with the RE agent in the same economy, I find that:

- The AI agent, due to its explorative nature, does not reach a consumption level as illustrated by an RE agent, and may never do so. However, it saves more than the RE agent, which can be a useful feature to guard against further shocks to its endowment.
- The simulation results are uncalibrated, and it requires a substantial number of periods for the AI agent to behave similarly to the RE agent, as illustrated in Figure 3.3.

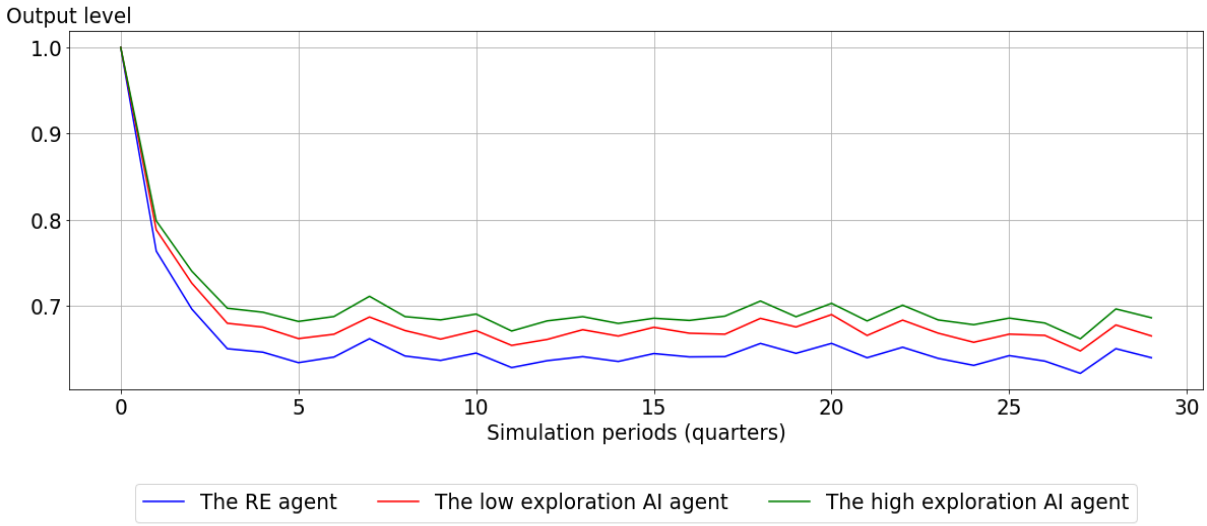
3.4.2 AI Agents with Different Exploration Levels in a Stationary Environment with i.i.d Shocks

AI agents display unique behaviors in comparison to an RE agent, and their behaviors can also vary amongst themselves. Some AI agents tend to explore less and stick to what they already know, while others opt for more exploration of the environment, increasing their chances of discovering actions that lead to higher rewards. To further investigate the implications of different levels of exploration, this subsection includes the simulation results of an AI agent who explores more than the one examined in the previous subsection. This additional analysis aims to understand the consequences of low and high levels of exploration and how they influence the AI agents' decision-making and overall performance.

In the same environment as the previous experiment, where the same stochastic shocks are applied, I introduce an additional simulation path to Figure 3.1 and 3.2

representing an AI agent with a high level of exploration. The corresponding plots are displayed in Figure 3.4 and 3.5. This new simulation allows us to observe and compare the behaviors and outcomes of an AI agent exploring more extensively with those of the previously studied agents.

Figure 3.4: Output Comparison



Notes:
1. x-axis represents simulation periods in quarters.
2. y-axis represents the quantity of total output production in the economy with an RE agent (blue line), a low exploration AI agent (red line), and a high exploration AI agent (green line).
3. The total production output starts at the same level for all agents at period 0.

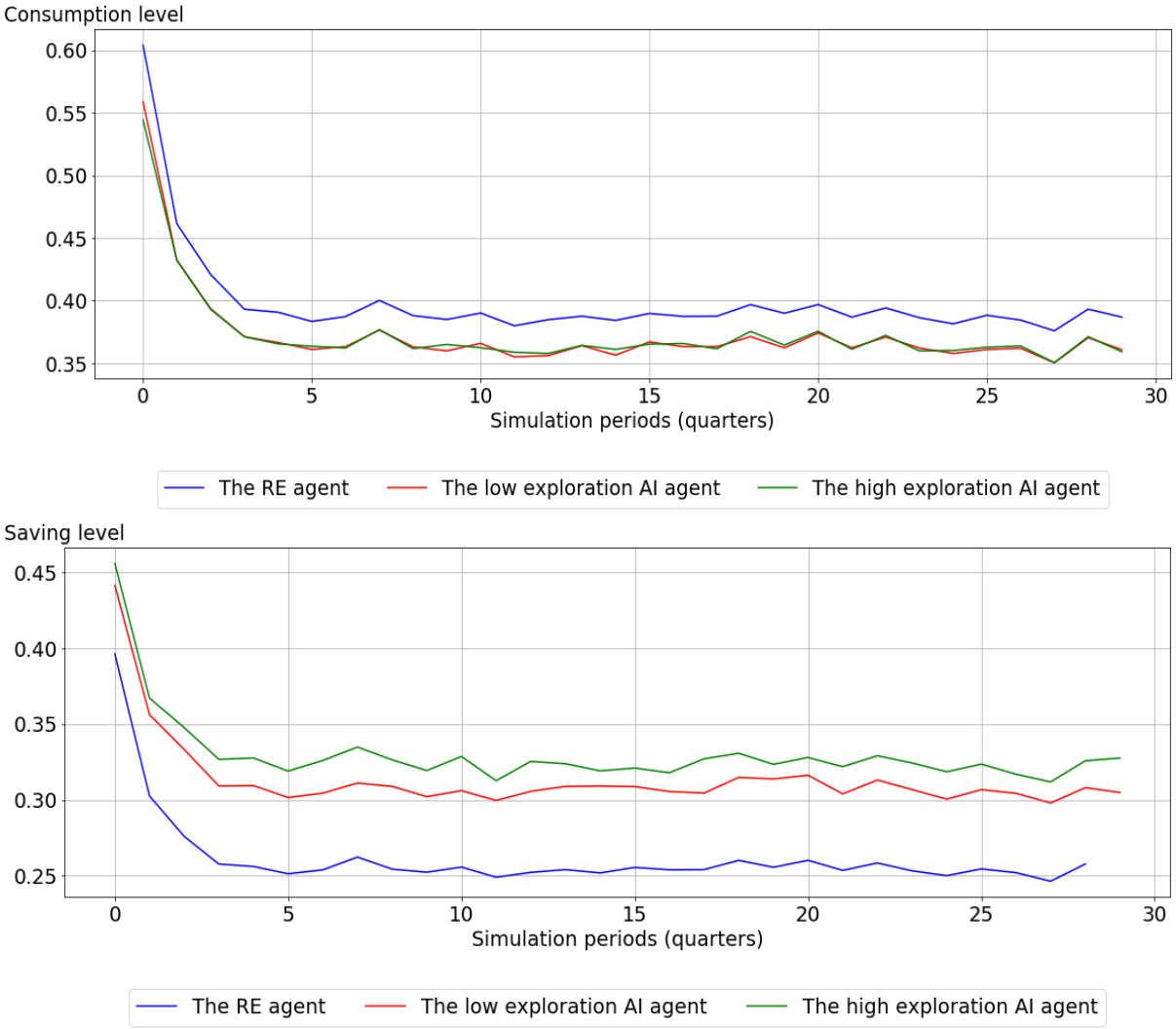
I observe that the AI agent who explores more (green line in Figure 3.5) consume around similar level to the AI agent who explores less. It saves around 46% of the total output compare to 44% for a low exploration agent. Its consumption level gradually stabilizes at approximately 0.37 (equivalent to 54% of the total output) after 5 quarters.

After introducing an additional AI agent who engages in more extensive exploration of the environment, I observe that its behavior exhibits similarities to that of the

RE agent but is not entirely identical. This divergence can be attributed to the persistent explorative nature of the AI agent, which causes it to deviate from its established beliefs and introduce noise to its behaviors. The exploration-driven behavior underscores the agent's adaptability and willingness in pursuit of potentially more rewarding strategies.

The agent's openness to exploration adds a dynamic element to its decision-making process, enabling it to respond flexibly to new information and opportunities. This adaptability allows the AI agent to evolve its strategies over time, making it more resilient and capable of adjusting to changing circumstances. While the AI agent's behavior may not perfectly align with that of the RE agent due to its exploratory nature, this characteristic provides valuable insights into the agent's ability to learn, adapt, and potentially discover more favorable courses of action.

Figure 3.5: Consumption and Saving Levels Comparison



Notes:

1. x-axis represents simulation periods in quarters.
2. y-axis represents the quantity of consumption per simulation period for the RE agent (blue line), the low exploration AI agent (red line), and the high exploration AI agent (green line).

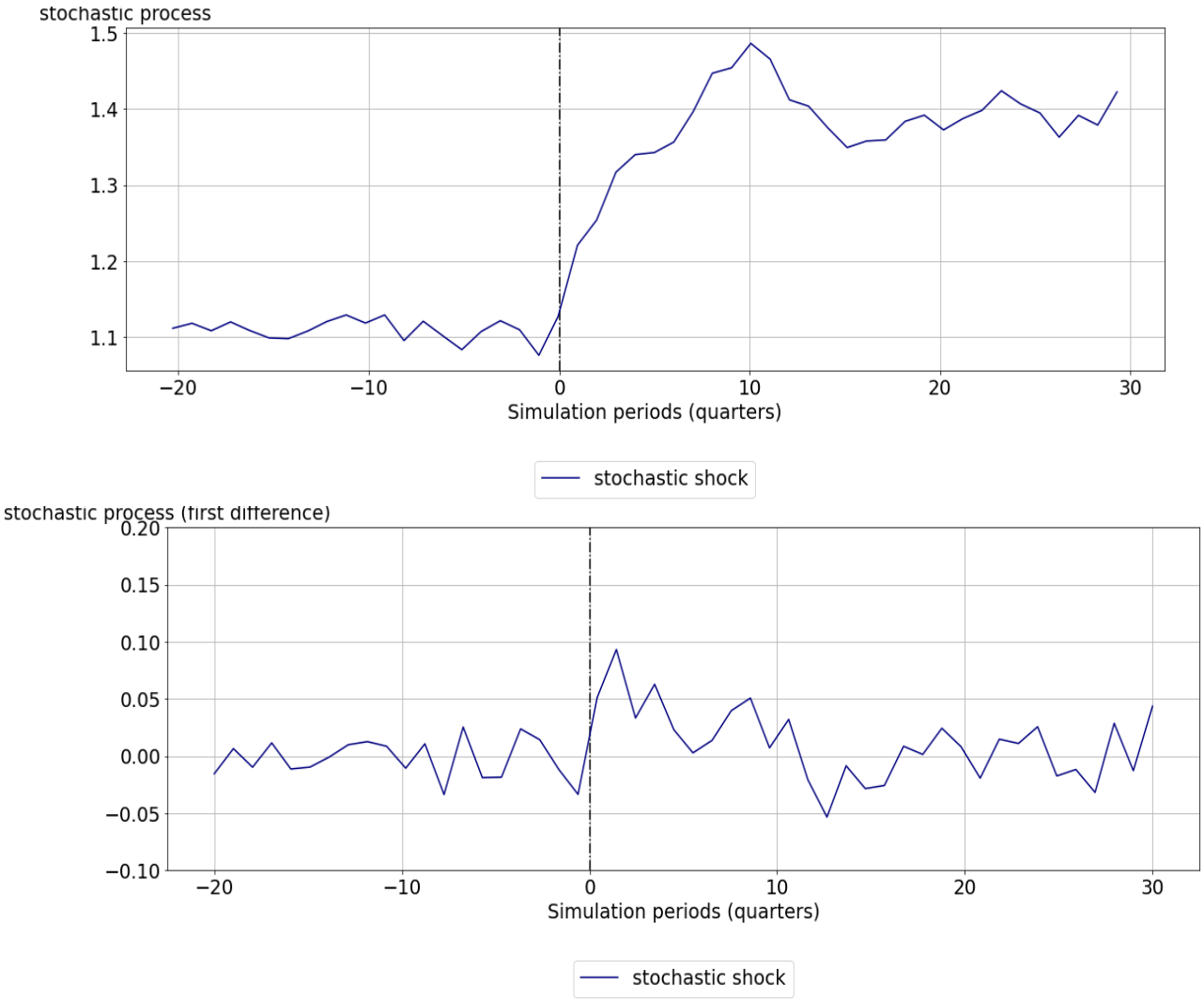
3.4.3 AI Agents with Different Exploration Levels in an Environment with a Permanent Change

I modify the environment to transform the stochastic process from being i.i.d to an autoregressive process with an order of 1, denoted as AR(1). An example of this altered stochastic process is illustrated in Figure 3.6, with period 0 representing the timing when the transition takes place.

I plot the output, consumption, and savings in both levels and first differences, as shown in Figures 3.7, 3.8, and 3.9. I find that, both before and after the change in the stochastic process, the high-exploration AI agent saves a larger portion of its resources compared to the low-exploration agent, as depicted in the top panel of Figure 3.9.

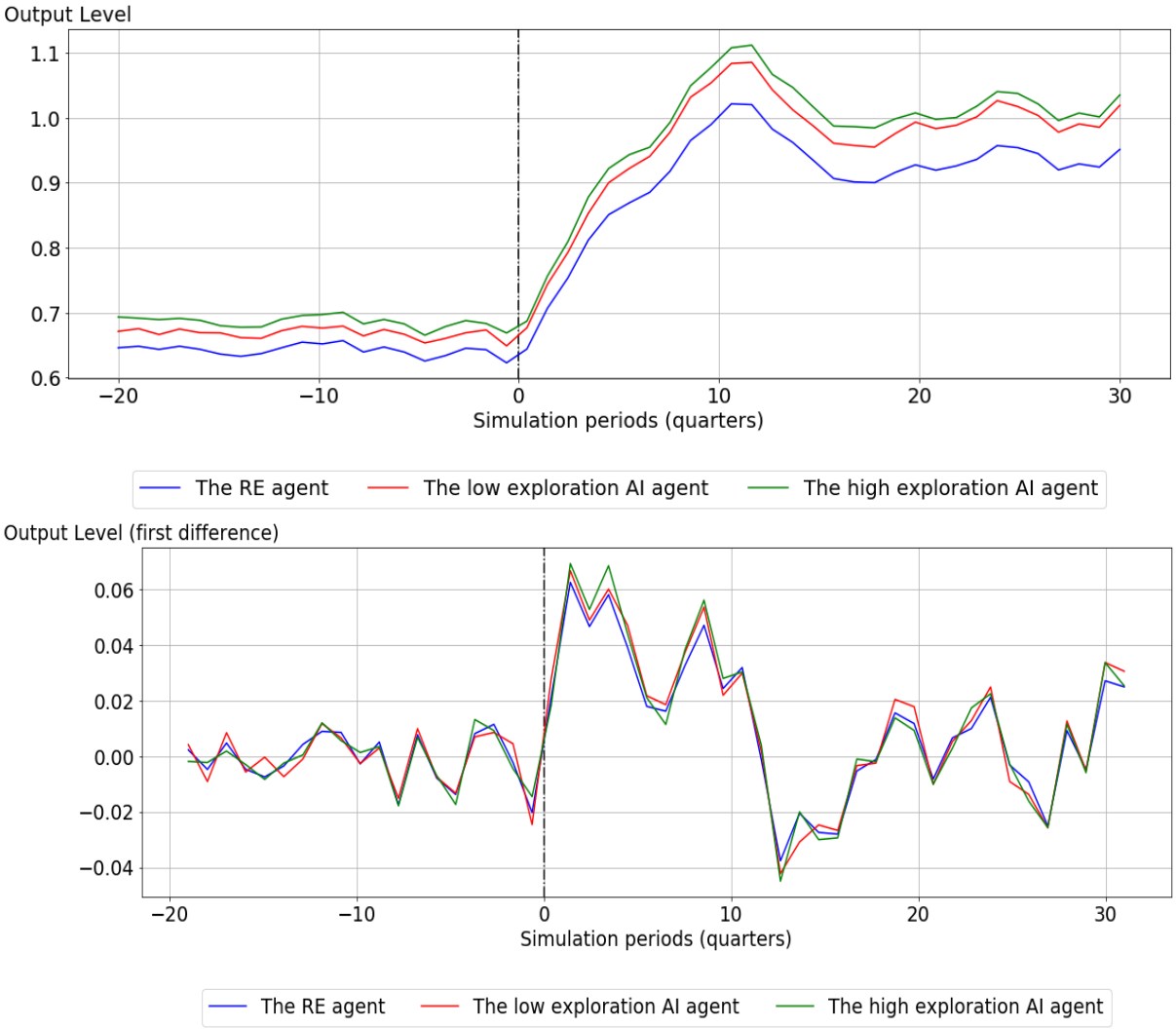
Moreover, the saving level of the high-exploration AI agent is consistently higher than that of the low-exploration AI agent. Consequently, the economy associated with the high-exploration agent experiences higher total production. However, after removing the level effects, the simulation paths of the three agents become similar, as depicted in the bottom panels of Figures 3.7, 3.8, and 3.9. These results corroborate the findings presented in Section 3.4.2, further demonstrating the impact of exploration levels on AI agents' decision-making and economic performance, even during a permanent change in the environment.

Figure 3.6: The Stochastic Process (in level and first difference)



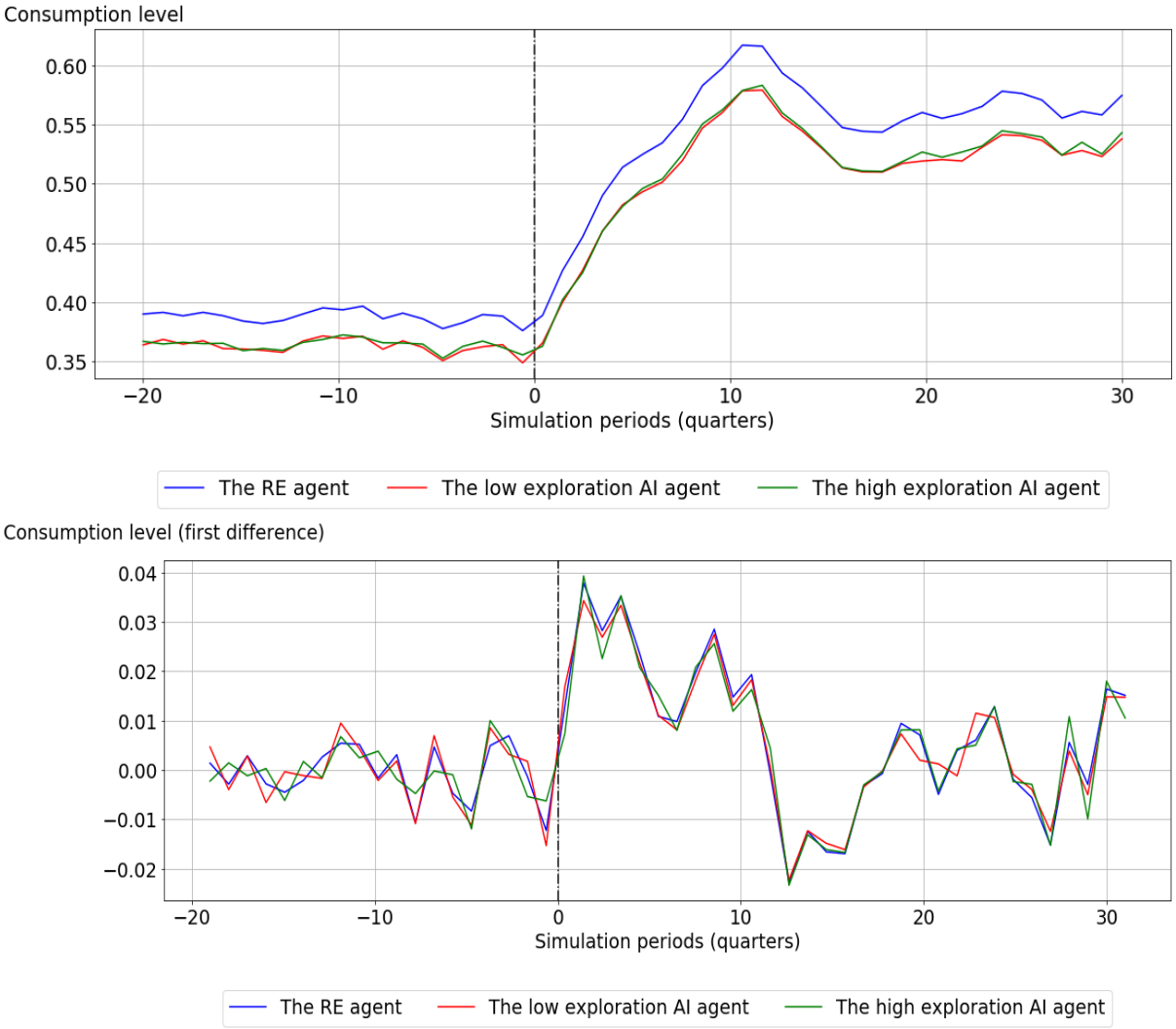
Notes:
1. The x-axis depicts simulation periods in quarters.
2. At marked period 0, a stochastic shock transition occurs from drawing from an i.i.d process to an AR(1) process.
3. The i.i.d process is represented as follows: $z_t = e^{0.1+\epsilon_t}$
4. The AR(1) process is represented as follows: $z_t = e^{0.1+0.7 \ln z_{t-1} \epsilon_t}$
5. The y-axis represents the stochastic process on the top panel and the first difference transformation of the same stochastic process on the bottom panel.

Figure 3.7: Output (in level and first difference)



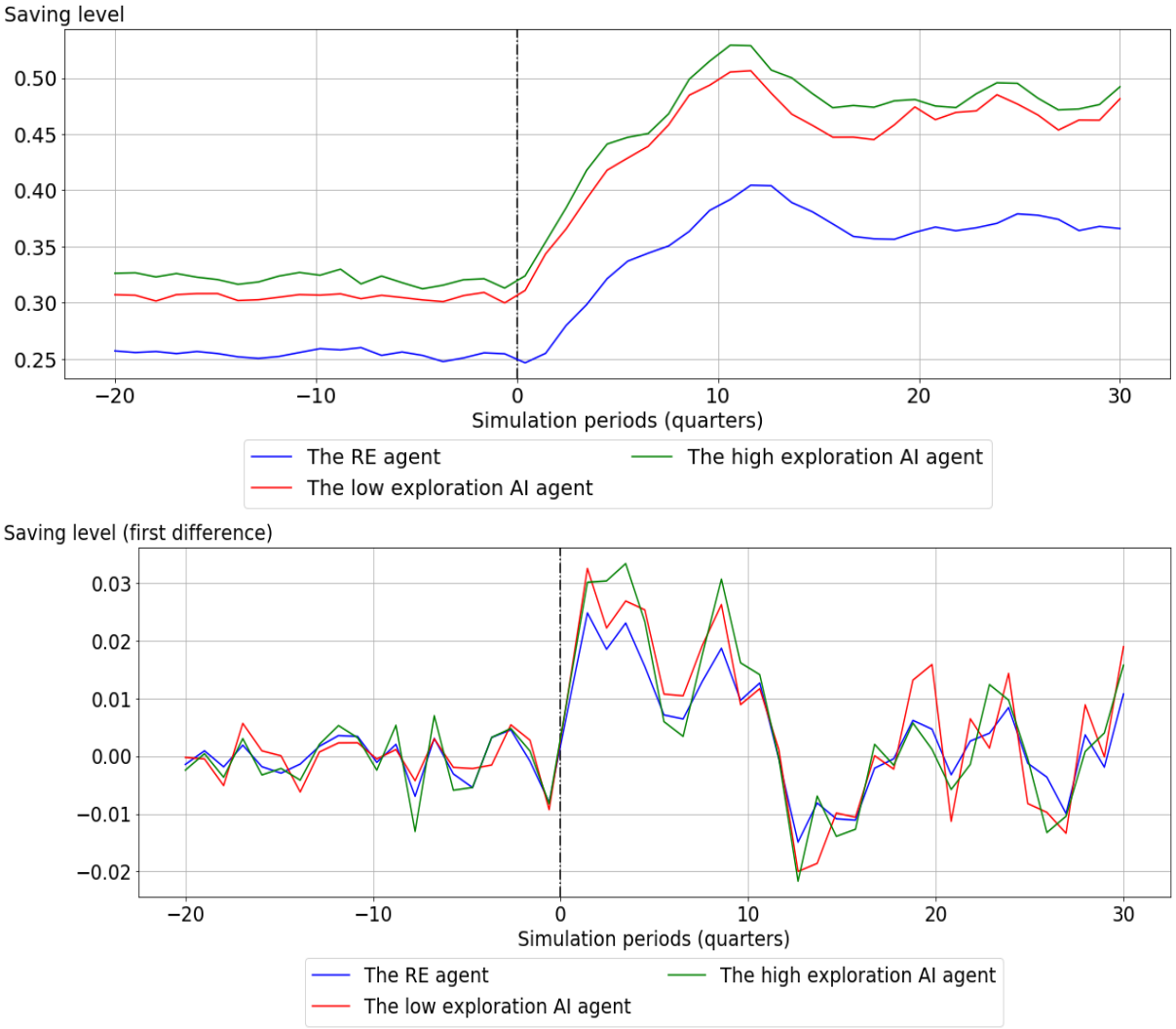
Notes:
1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the output level on the top panel and its first difference transformation on the bottom panel.
3. The red line represents the low exploration AI agent, while the green line represents the high exploration AI agent.

Figure 3.8: Consumption (in level and first difference)



Notes:
1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the consumption level on the top panel and its first difference transformation on the bottom panel.
3. The red line represents the low exploration AI agent, while the green line represents the high exploration AI agent.

Figure 3.9: Saving (in level and first difference)



Notes:
1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the savings level on the top panel and its first difference transformation on the bottom panel.
3. The red line represents the low exploration AI agent, while the green line represents the high exploration AI agent.

However, it is essential to acknowledge that high exploration can also entail negative consequences. While the AI agent may discover strategies that yield higher rewards, it is equally possible for it to stumble upon strategies that underperform

its established beliefs, leading to lower utility and potentially volatile aggregate conditions. This highlights the trade-off associated with high exploration, where the agent's adaptive and exploratory nature can lead to both advantageous and disadvantageous outcomes.

3.5 Summary

In conclusion, this chapter has introduced a novel framework for modeling human behavior within economic models by leveraging advancements in AI technologies, specifically deep reinforcement learning. This framework stands as an alternative to the traditional Rational Expectations assumption and the adaptive expectations methodology, offering a more dynamic and responsive approach to modeling economic agents.

Employing a deep RL algorithm capable of handling continuous action spaces and high-dimensional state and action spaces, this framework facilitates the interaction between AI agents and the economy. Contrary to the RE assumption, AI agents within this framework are not constrained to maintain model-consistent beliefs. Furthermore, both the policy and value functions of AI agents are adaptable in terms of their functional form and parameters, moving beyond the simple adaptive expectations assumption.

The exploration feature of AI agents ensures that their behaviors, even in a stationary environment, are distinct from those predicted by the RE model. Through simulation experiments in a stochastic optimal growth model, I have shown that AI agents can evolve from a state of ignorance to exhibit behaviors similar but not identical to rational agents. This comparison underscores the AI agents' ability to adjust their actions to maximize long-term rewards while maintaining the capacity

for exploration, a feature absent in RE agents.

Moreover, the degree of exploration significantly influences AI agent behaviors. Agents with higher levels of exploration tend to save more, and contribute to greater overall economic output compared to both their less exploratory counterparts and traditional RE agents. These results highlight the unique behaviors of exploratory AI agents and their effect on decision-making and economic outcomes.

Incorporating an exploration mechanism that mirrors the exploratory nature of human decision-making, the DDPG framework presents a valuable asset for economic behavior modeling. It captures the variation in individuals' propensity to explore new options and their tendency to decrease exploration over time, offering a more nuanced and realistic depiction of human decision-making within economic models.

This study proposes a flexible framework to model bounded rationality in economic agents. It can be customized for different economic environments, memory sampling strategies, and sample sizes. The framework supports research on important economic questions such as structural breaks, regime changes, and multi-agent learning. By using ANNs, it allows agents to make decisions without predetermined functional forms and adapt to their evolving subjective beliefs. This aligns with empirical evidence on belief formation with a use-dependent brain by Malmendier [2021]. Overall, the framework offers valuable insights into economic behaviors and facilitates a deeper understanding of AI agents' decision-making processes.

Chapter 4

Can an AI Agent Hit a Moving Target?

4.1 Introduction

The modeling of adaptive economic agents during periods of regime changes has been a subject of extensive discussion. This discussion can be traced back to the accelerationist debate [Sargent, 1971]. The accelerationist argument, characterized by a backward-looking Phillips curve coupled with adaptive expectations, suggests a plausible trade-off between inflation rates and unemployment levels. This theoretical construct promotes the idea that a government could maintain low unemployment rates by accelerating the money supply process. It raises the fundamental question of whether policymakers can exploit agents with naive adaptive expectations who consistently make errors.

In response to this, the RE hypothesis was put forth as a solution, drawing on the works of Lucas [1972, 1976]; Sargent [1971]; Sargent and Wallace [1973]. This

hypothesis assumes that agents hold model-consistent beliefs and possess a profound understanding of the economy. The evolution from naive to intelligent agents initiated a revolution in economic modeling, bringing considerable advantages, particularly while executing policy experiments within relatively stable environments. However, RE agents lack adaptability in dynamic environments, where structural changes occur. Therefore, exploring the modeling of adaptive agents and studying their behaviors during such changes remains an important and contentious question.

Economic agents, or in reality, human beings, exhibit a level of intelligence that falls between that of RE agents and naive adaptive expectations agents. Their expectations and behaviors are intricately nuanced. They engage in exploration of different options, evaluate their actions, draw connections between the present and the past, consider long-term satisfaction, and may also prioritize immediate gratification. When faced with changes in their environment, they demonstrate the ability to adapt, albeit at varying speeds.

I propose to model the adaptive behaviors of an economic agent during a regime change using the deep RL framework. Through the previous two chapters, I have demonstrated that the deep RL framework offers a natural and practical approach to capturing human decision-making processes. It involves training AI agents to interact with the economy and make exploratory actions, allowing them to learn from these interactions and adjust their policy and value functions over time.

My objective is to examine the ability of an AI agent to adapt and modify its behaviors when the environment undergoes a significant change. This change is characterized by a shift in the aggregate price sequence from a stationary to a non-stationary state. I focus on the extent to which the AI agent can adjust its beliefs and decisions, and how the level of exploration impacts its adaptive behavior during such a drastic economic shift. These inquiries are essential in demonstrating the

real-world applicability of AI algorithms in modeling agents with limited rationality within an economy experiencing structural transformations.

To conduct the experiment on regime change, I employ a general equilibrium model that features transaction costs of money demand, and utilize a representative AI agent operating within the deep RL framework. Taking inspiration from the accelerationist debate, I design the monetary authority to raise the daily growth rate of nominal money supply from 0% to 1%. The representative AI agent makes two decisions in each period: the storage decision, determining the quantity of real resources to store (and to consume), and the liquidity holding decision, which influences the desired level of real balance in order to minimize transaction costs. Within the deep RL framework, the agent's policy function and value function are approximated using deep ANNs, enabling adaptive adjustment of both functional form and parameters.

Through simulation experiments, I find that:

- The AI agent's capability to adapt its beliefs in response to economic changes is evident in its adjustments to consumption, storage, and demand for real balance decisions.
- Exploration not only impacts the levels of different decisions made by AI agents but also influences which decisions the agent adjusts during a regime shift, given that the AI agent has two decisions to make each period. I find that both agents maintain similar levels of consumption in both regimes. However, the high-exploration agent's liquidity holding level is consistently higher than that of the low-exploration agent. During the regime change, I find that the high-exploration agent mainly adjusts its liquidity holding level, whereas the low-exploration agent reduces both its liquidity holding level and storage level. This difference in adjustment results in 0.6% less wealth accumulated

by the low-exploration agent compared to the high-exploration agent in the new regime.

- Because of the exploration feature, the agents' beliefs do not align with those of an RE agent; however, they possess the ability to adjust and adapt to a regime change.

The rest of the paper is organized as follows. Section 4.2 describes the economic environment. Section 4.3 introduces and discusses the AI framework of bounded rationality. Section 4.5 discuss the experiments conducted, and the results. The last section concludes.

4.2 The Model: an Economist's Approach

I employ a transaction cost of money demand model to illustrate the role of money in the economy, akin to the approach modelled by Sims [1994]. In this model, the preference of individuals for holding money stems from its capacity to diminish transaction costs. Although there are alternative methods to incorporate money into economic models, such as direct integration into utility functions, I opt for the transaction cost approach due to its intuitive appeal. It is important to recognise that money itself does not inherently provide utility; rather, its value derives from its ability to facilitate the purchase of goods and services and its convenience in enabling transactions.

In this model, an AI agent is required to determine a policy within a framework that encompasses decisions on both storage and real balance in each period. This represents a departure from the previous chapter, where the agent's decisions were

confined to savings each period.

By integrating money into the model, the monetary authority acquires policy leverage to influence the decisions of private agents through various policy instruments, including the control over the growth rate of the nominal money supply. Consequently, this model offers a platform to conduct experiments to explore the behavioral shifts of AI agents in response to a regime change in monetary policy, contrasting with the previous chapter, which focused solely on the real sector of the economy.

I first elucidate how the model is conventionally presented in economics as a benchmark.

4.2.1 A Representative Household

A representative household aims to maximise its lifetime utility, as outlined in Equation 2.1.

$$E_0 \sum_{t=0}^{\infty} \beta^t u(c_t), \tag{2.1}$$

subject to the constraint,

$$c_t(1 + \frac{1}{\kappa} f(v_t)) + s_{t+1} + \frac{M_{t+1} - M_t}{P_t} \leq y_t + s_t^{\alpha} + \tau_t \tag{2.2}$$

where $\beta \in (0, 1)$, P_t is the price level at period t, c_t denotes the consumption at t, M_t is nominal money balance, s_t is the saved stock of goods that a household enters period t with, and there is a storage technology that takes the Cobb-Douglas form.

y_t is the endowment or income of the agent. τ_t is the government transfer at t.

$\frac{1}{\kappa}f(v)$ represents transactions costs per unit of consumption. $f(v_t) = v_t$, and velocity is $v_t \equiv \frac{c_t P_t}{M_{t+1}}$. Real balance is defined as $m_{t+1} = \frac{M_{t+1}}{P_t}$. Demanding or holding real balance reduces transaction cost.

The endowment depends on a constant $\bar{y}$ and an exogenous process ϵ_t^y, where ϵ_t^y is drawn randomly from a normal distribution with zero mean and variance of 0.01.

$$y_t = \bar{y} + \epsilon_t^y \tag{2.3}$$

4.2.2 Government: Fiscal and Monetary Policies

The government conducts an active monetary policy and a passive fiscal policy. The monetary authority follows a money growth rule:

$$M_{t+1} = \delta^M M_t, \tag{2.4}$$

where δ^M is a policy variable that determines the speed of money supply. In real terms, this money supply rule becomes,

$$m_{t+1} = \delta^M \frac{m_t}{\pi_t}, \tag{2.5}$$

where real balance $m_t \equiv \frac{M_t}{P_{t-1}}$.

Government budget constraint is,

$$g + \tau_t = \frac{M_{t+1} - M_t}{P_t}. \tag{2.6}$$

The government sets initial nominal money supply and a policy variable δ^M. It also determines a constant government spending g.

Combine the household and the government budget constraints equation 2.2 and 2.6, the consolidated budget constraint is

$$c_t(1 + \frac{1}{\kappa}v_t) + s_{t+1} = y_t + s_t^\alpha - g. \tag{2.7}$$

4.2.3 Household's First Order Conditions (FOCs)

The household's Lagrangian is specified as

$$\mathcal{L} = E_t \sum_{t=0}^{\infty} \beta^t \left\{ ln(c_t) + \lambda_t \left[y_t + s_t^\alpha + \tau_t - c_t(1 + \frac{1}{\kappa}v_t) - s_{t+1} - \frac{M_{t+1} - M_t}{P_t} \right] \right\} \tag{2.8}$$

where $v_t = \frac{c_t P_t}{M_{t+1}}$.

The FOC with respect to consumption c_t is ,

$$\frac{1}{c_t} = \lambda_t(1 + \frac{2v_t}{\kappa}). \tag{2.9}$$

The FOC with respect to M_{t+1} is,

$$\frac{\lambda_t}{P_t}(1-\frac{v_t^2}{\kappa}) = \beta E_t \frac{\lambda_{t+1}}{P_{t+1}}. \tag{2.10}$$

The FOC with respect to s_{t+1} is,

$$\lambda_t = \alpha\beta s_{t+1}^{\alpha-1} E_t \lambda_{t+1}. \tag{2.11}$$

4.2.4 Non-Stochastic Steady States and Determinancy

The equations governing the dynamic system of this model consist of the first-order conditions, specifically Equations 2.9, 2.10, and 2.11, alongside the consolidated budget constraint, the rule governing the money supply, and the process determining the endowment. Additionally, I incorporate the definition of velocity directly into these equations and use inflation instead of price levels. Inflation is defined as $\pi_t = \frac{P_t}{P_{t-1}}$.

$$\left[\frac{1/c_t}{1+\frac{2c_t}{\kappa m_{t+1}}}\right]\left[1-\frac{c_t^2}{\kappa m_{t+1}^2}\right] = \beta E_t \frac{\frac{1/c_{t+1}}{1+\frac{2c_{t+1}}{\kappa m_{t+2}}}}{\pi_{t+1}} \tag{2.12}$$

$$\left[\frac{1/c_t}{1+\frac{2c_t}{\kappa m_{t+1}}}\right] = \alpha\beta s_{t+1}^{\alpha-1} E_t \left[\frac{1/c_{t+1}}{1+\frac{2c_{t+1}}{\kappa m_{t+2}}}\right] \tag{2.13}$$

$$m_{t+1} = \delta^M \frac{m_t}{\pi_t} \tag{2.14}$$

$$c_t(1+\frac{c_t}{\kappa m_{t+1}}) + s_{t+1} = y_t + s_t^\alpha - g \tag{2.15}$$

$$y_t = \bar{y} + \epsilon_t^y \tag{2.16}$$

To solve for the non-stochastic steady state, I assume zero shock and constant real variables. The non-stochastic steady state values are presented in Table 4.4, given

the specified parameters.

I arrange the equations in the form of Equation 2.17.

$$E_t f\left\{Y_{t+1}, Y_t, X_{t+1}, X_t\right\} = 0 \tag{2.17}$$

where E_t denotes the expectations operator conditional on information available at time t. The state vector is $X_t = [s_t, y_t, m_t]'$. The co-state vector is $Y_t = [c_t, \pi_t]'$.

I check whether the non-stochastic steady state is locally determinant by deriving the Jacobians of this system, as written in the form of Equation 2.17. I calculate generalized eigenvalues using QZ decomposition, all in accordance with the parameters stated in Section 4.4.

I calculate the list of eigenvalues to be $[0, 1, 0.18, 5.48, 0]$. There are three stable roots, two unstable ones, and two co-states are solved from three state variables. Therefore, I conclude that the steady state is unique and locally determinate. The same exercise is repeated for the second regime with an increasing supply of nominal balance, leading to the same conclusion: the steady state is locally determinate and unique.

4.3 The Model: an AI Approach

In the context of the specified economic environment, I present my methodology for designing the AI agent to carry out experiments. In Chapter 3.3, I have provided detailed specifications for the state, action, reward, and decision-making center of the AI agent. Therefore, in this section, I focus on designing these components based on the specified economic model without revisiting the definitions.

4.3.1 Deep RL Components

To apply and implement the AI framework, the economic model is first translated into components of a MDP, including state, action, reward, and next state[1]. These components are summarized in Table 4.1.

In the transaction cost of money demand model, the state variables encompass current period income and storage technology. The agent is also aware of past period inflation, real balance, and storage level.

The actions available to the AI agent consist of determining the proportion of real resources to store and the desired level of real balance to demand. Based on its storage decision, the agent allocates resources for consumption. Likewise, its decision on real balance demand affects transaction costs and aggregate inflation dynamics. In this representative agent model, the AI agent's demand for real balance represents the aggregate money demand, although in multi-AI agent studies, aggregate money demand would differ from individual demands for real balance.

[1]To avoid confusion with the storage variable s_t, I use capital letters to denote deep RL-related terms: state (S_t), action (A_t), and reward (R_t).

Table 4.1: RL Components and the Economic Environment

Terminologies	**Description**	**Representation in the economic environment**
State, S_t	A random variable from a state space, $S_t \in \mathcal{S}$	$S_t = \{y_t, \pi_{t-1}, m_t, s_t\}$
Actions, A_t	A random variable from an action space, $A_t \in \mathcal{A}$	$A_t = \{\lambda_t^s, m_{t+1}\}$
Rewards, R_t	A function of state and action	$R_t = ln(c_t)$
Policy function, $\mu(S\|\theta^\mu)$	A mapping from state to action, $\mu : \mathcal{S} \rightarrow \mathcal{A}$	Approximated by a neural network, i.e., actor network; parameterised by θ^μ to be updated during learning
Value function, $Q(S, A\|\theta^Q)$	The 'expected' (subjective belief) return from taking an action in a state	Approximated by a neural network, i.e., critic network; parameterised by θ^Q to be updated during learning

The reward function is influenced by the agent's level of consumption. Higher consumption leads to greater rewards per period. Policy and value functions are approximated using two deep neural networks. The AI agent aims to develop a decision-making strategy that maximizes its performance not only in each period but also over the long run, as measured by cumulative rewards. The agent seeks consistent high-level rewards rather than focusing solely on maximizing rewards in a single period.

Consequently, the agent faces two critical decisions: 1) determining the balance between storage and consumption, and 2) managing transaction costs through the allocation of resources to hold money. Higher storage allows for increased future consumption when adverse income or endowment shocks occur, though it means consuming less in the current period. The second decision involves finding the optimal allocation between holding money and real goods, striking a balance between lower transaction costs and resource allocation for cash demand, which reduces overall rewards.

4.3.2 Full Algorithm and Sequence of Events

The complete algorithm comprises three main steps: initialization, interaction, and learning. Steps I and III closely resemble those presented in Chapter 3 and are included here to provide a comprehensive overview of the algorithm. Step II outlines the sequence of events specific to the current model, as specified in Section 4.2.

Step I: Initialisation

- In a given environment, design a state space $\mathcal{S}$, a continuous bounded and compact set for random variables specified in Table 4.1; design an action space $\mathcal{A}$, a continuous bounded and compact set for the action (random) variables.
- Set up two deep ANNs: an actor network $\mu(S|\theta^{\mu})$ takes the argument of a state from the state space and generates an action within the action space; a critic network $Q(S, A|\theta^{Q})$ takes the argument of a realised state-action pair and generates a value. Setting up two ANNs involves determining the input and output dimensions, the specific architectures, the number of layers, the number of nodes per layer, and how nodes are connected.

- θ^{μ} represents the parameters of the actor network, and θ^{Q} represents the parameters of the critic network.

- Define a replay buffer $\mathcal{B}$ (called transitions in the deep RL literature), which is a memory that stores information that is collected by a deep AI agent during the agent-environment interactive process. A transition is characterised by a sequence of variables (S_t, A_t, R_t, S_{t+1}).

- Define a length of N, which is the size of a mini-batch. A mini-batch refers to a sample drawn from the memory, $\mathcal{B}$.

- Define the total number of episodes E and simulation periods per episode. The higher the episodes, the longer the learning periods.[2]

- Lastly, a simulation period of T is established for each episode, with T exceeding the value of N.

For each episode, define the initial state,[3] and loop Steps II and III.

Step II: The AI agent starts to interact with its environment for $t \leq T$. This step involves how agents' actions are chosen and how their actions impact the aggregate economy.

- Assume $\delta^M = 1.00$ (δ^M=1.01 in the new regime) and other variables that are known to the agent at period t: y_t, π_{t-1}, m_t, s_t.

[2]In the deep RL literature, the AI agent is usually set to learn a particular task or an Atari game. An episode thus means re-starting the game or task, and it ends with a terminal state (i.e., the end result of a game). In an economic environment, however, a clear terminal state can be difficult to specify. Therefore, the concept of episodes only correlates to how long an agent has been learning.

[3]Initial state variables can also be randomly drawn from the state space.

- The agent selects a random (based on the randomly initialised actor network) of action variables, $A_t = \mu(S_t|\theta^\mu) + \mathcal{N}_t$, and A_t contains λ_t^s and m_{t+1}, and $\mathcal{N}_t$ is a noise attached to the action to ensure exploration, which is sampled from an AR(1) process.

- Given the exogenous (to the AI agent) nominal money supply from the government, $M_{t+1} = \delta^M M_t$, with aggregate money demand equal to aggregate money supply, price level or inflation can be derived as $\pi_t = \frac{M_{t+1}}{M_t}\frac{m_t}{m_{t+1}} = \delta^M \frac{m_t}{m_{t+1}}$. In a one-agent case, aggregate money demand at period t is m_{t+1}.

- The amount stored is $s_{t+1} = \lambda_t^s(y_t + \frac{m_t}{\pi_t} + s_t^\alpha + \tau_t - m_{t+1})$.

- c_t is reached from the budget constraint.

- The new state variables are $S_{t+1} = \{y_{t+1}, \pi_t, m_{t+1}, s_{t+1}\}$, where $y_{t+1} = \bar{y} + \epsilon_{t+1}^y$, and ϵ_{t+1}^y is sampled from a normal distribution $N(0, 0.01)$, $\bar{y} = 1$.

- The reward the agent receives is, $R_t = u(c_t)$.

- Store a transition (S_t, A_t, R_t, S_{t+1}) in the memory $\mathcal{B}$.

Step III: Training the AI agent (when the AI agent starts to learn) for period $N \leq t \leq T$.[4]

- Sample a random mini-batch of N transitions (S_i, A_i, R_i, S_{i+1}) from the memory $\mathcal{B}$.

- Calculate the TD-target values TD_i for each transition $i \in N$ following

$$TD_i = R_i + \beta Q^\mu(S_{i+1}, \mu(S_{i+1}|\theta^\mu)|\theta^Q) \tag{3.18}$$

[4]The training period descriptions are analogous to those described in Chapter 3, Section 3.3.3, except for the specific parameter values and the action and state spaces; thus, they are not repeated here.

where $Q^{\mu}(S_{i+1}, \mu(S_{i+1}|\theta^{\mu})|\theta^{Q})$ is a prediction made by the critic network with state-action pair $(S_{i+1}, \mu(S_{i+1}|\theta^{\mu}))$, and $\mu(S_{i+1}|\theta^{\mu})$ is a prediction made by the actor network with input S_{i+1}.

- Obtain $Q(S_i, A_i|\theta^{Q})$ from the critic network with input state-action pair (S_i, A_i)
- Calculate the average loss for this sample of N transitions

$$L = \frac{1}{N} \sum_{i} \left(TD_i - Q(S_i, A_i|\theta^{Q})\right)^2 \tag{3.19}$$

- Update the critic network with the objective of minimising the loss function L.[5]
- For the policy function, i.e., the actor network, the objective is to maximise the value function predictions. Define the objective function as,

$$J(\theta^{\mu}) = Q^{\mu}(S_i, \mu(S_i|\theta^{\mu})|\theta^{Q}). \tag{3.20}$$

- Maximising this objective function is equivalent to minimising $-J(\theta^{\mu})$. Update the actor network parameters θ^{μ} with the objective of minimising $-J(\theta^{\mu})$.[6]

[5] This involves applying backpropagation and gradient descent procedures.

[6] Similar to the critic network, the specific steps of updating ANN's parameters by minimising an objective function involve backpropagation and gradient descent.

4.4 Parameters and (Non-Stochastic) Steady State Values

Table 4.2: Baseline Parameters and Steady State Values

	Regime I	**Regime II**
Storage Technology Parameter, α	0.4	
Discount Factor, β	0.99999	
Endowment, $\bar{y}$	1	
Government Spending, g	0	
κ	1000	
Exploration	Low - 0.008; High - 0.02	
Monetary Policy parameter, δ^M	1.0	1.01
RE Velocity, v	0.1	3.148
Consumption, c	1.326	1.322
Storage, s	0.217	0.217
Real Balance, m	13.256	0.420

I choose the discount factor to be 0.99999 so that each simulation period corresponds to a day. The rest of the parameters are uncalibrated. Under the baseline regime,

the annual growth rate of the nominal money supply is assumed to be zero, and government spending is also set to zero. The parameter for the transaction cost function was chosen to be 1000.

In the scenario of a monetary regime change, there is a shift in the daily growth rate of the nominal money supply from 0 to 1%. This shift does not represent a realistic change in real-world regimes. The purpose of this drastic shift is to facilitate comparison of steady-state values across the two regimes, enabling an AI agent to detect differences.[7]

In this chapter, the exploration level is lowered to 0.008 compared to the previous chapter.[8] This adjustment ensures that the exploration variation does not excessively hinder the AI agent's capacity to learn from changes in the environment. Specifically, the challenge arises from the subtle nature of the policy variable change, potentially complicating detection and response activities for an AI agent without a sufficiently low exploration level.

4.5 Experiments and Results

4.5.1 Experiments

In this chapter, I study the adaptive behaviors of a representative AI agent when the environment undergoes a drastic change. I investigate the role of exploration in facilitating the adaptive behaviors of AI agents, and how different levels of ex-

[7] I find that when the two steady states are very similar, especially regarding their corresponding reward values, it becomes difficult for the AI agent to discern changes. This observation warrants further studies and experimentation to ascertain the precise relationship between the magnitude of shifts and the AI agents' ability to detect changes and adapt accordingly.

[8] Reflecting the minor magnitude of policy change, the learning rates for the ANNs in this chapter are also reduced, facilitating more precise updates each period. Detailed ANN specifications are available in Appendix B.2.

ploration may shape the beliefs and behaviors of these agents. Following the full algorithm outlined in Section 4.3.1, I conduct the following experiment.

A representative AI agent is introduced into an economy where the monetary authority maintains a constant nominal money supply. The AI agent operates under this policy regime for approximately 27 years. Then, without prior announcement, there is a shift to a new regime where the nominal money supply increases daily at a rate of 1%.[9] The AI agents live in the new regime for around 60 years. Designing the regime change as a modification in the money supply process draws inspiration from early literature on the accelerationist controversy.

Given its adaptive and explorative nature, the AI agent theoretically should adjust to a monetary policy regime change by adapting its consumption and liquidity holding decisions. Through this experiment, I aim to examine the agent's adaptability, the extent of its adjustments in beliefs and decisions, and how the level of exploration affects its adaptive behavior during such a significant economic shift.

To examine the role of exploration, I set up two AI agents, one with a high level of exploration and the other with a low level, to start in the same economy under identical initial conditions. I then subject both agents to the same regime change experiment to observe and compare their responses. Additionally, I compare the simulation paths of the AI agents against those of an RE agent in both regimes.

Similar to the experiments described in Chapter 3, I replicate the simulation experiment 50 times and analyze the median results. These results, representing the median performance across the repetitions, provide a stable basis for comparison.

Moreover, the plots, derived from simulations conducted during the testing period in

[9]To reiterate, this shift does not represent a realistic change in real-world regimes. The purpose of this drastic shift is to facilitate comparison of steady-state values across the two regimes, enabling an AI agent to detect differences.

which agents operate without further updates or exploration, illustrate what the AI agents would have done after experiencing a regime change and living under the new regime for 60 years. This means that the beliefs have already been updated after experiencing a regime change. Therefore, direct changes in behaviour or aggregate variables can be observed as a result of regime changes in the simulation paths. These plots aim to facilitate the comparison of different agents' behaviors and their comparison with the path of the RE agent.

Through this experiment, I highlight three main findings:

1. The AI agent's capability to adapt its beliefs in response to economic changes is evident in its adjustments to consumption, storage, and demand for real balance decisions.

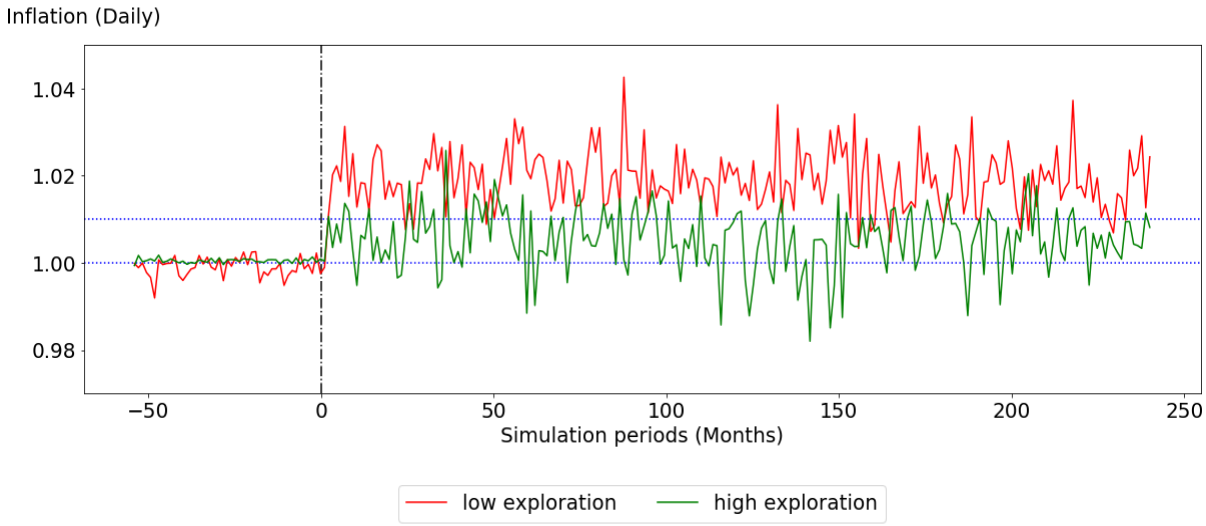
2. Exploration not only impacts the levels of different decisions made by AI agents but also influences which decisions the agent adjusts during a regime shift, given that the AI agent has two decisions to make each period. I find that both agents maintain similar levels of consumption in both regimes. However, the high-exploration agent's liquidity holding level is consistently higher than that of the low-exploration agent. During the regime change, I find that the high-exploration agent mainly adjusts its liquidity holding level, whereas the low-exploration agent reduces both its liquidity holding level and storage level. This difference in adjustment results in 0.6% less wealth accumulated by the low-exploration agent compared to the high-exploration agent in the new regime.

3. Because of the exploration feature, the agents' beliefs do not align with those of an RE agent; however, they possess the ability to adjust and adapt to a regime change.

4.5.2 Inflation and Wealth

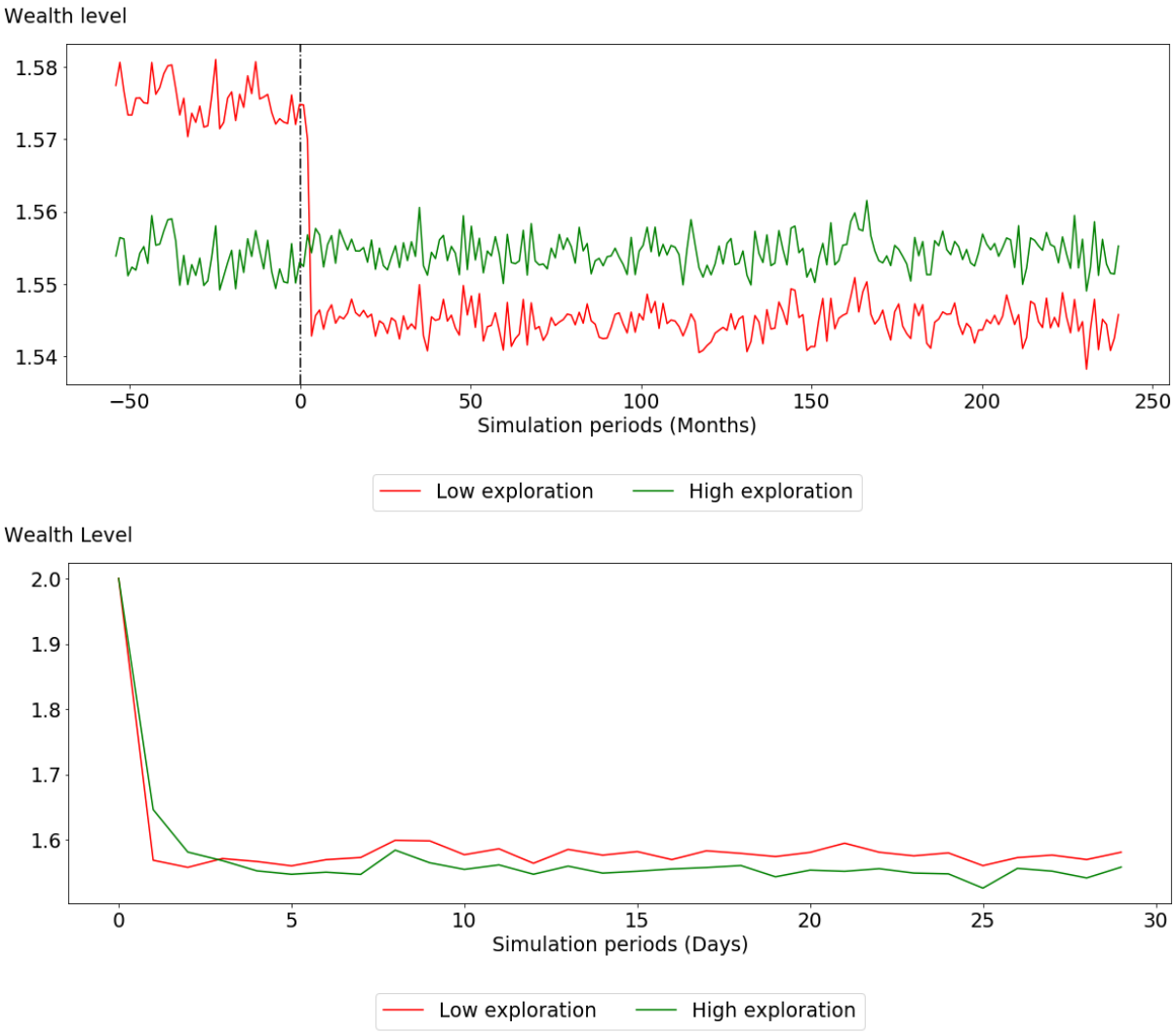
I first examine the aggregate of the economy by plotting inflation and wealth. In Figure 4.1, I plot the inflation series for two economies. The red line represents the economy where the low-exploration AI agent resides, and the green line represents the economy where the high-exploration agent resides. Prior to the regime change, inflation for both economies was around 0%. Once the regime shifts, I observe that inflation in the economy where the high-exploration agent resides becomes volatile, stabilising around 1%, while in the economy with the low-exploration agent, it stabilises around 1.5%.

Figure 4.1: Inflation



Notes:
1. The x-axis represents simulation periods in months.
2. The y-axis plots daily inflation, where inflation is defined as current period price level divided by the previous period price level.
3. A vertical dashed line at period 0 represents the occurrence of the regime change.
4. The red line represents the variable of interest for the low-exploration AI agent, and the green line represents that for the high-exploration AI agent.

Figure 4.2: Wealth



Notes:
1. The x-axis represents simulation periods in months in the top panel, and days in the bottom panel.
2. The top panel displays the level of wealth in the economy during a regime change, where wealth at period t is defined as $y_t + s_t^{\alpha}$.
3. The initial level of wealth, shown in the bottom panel figure, is identical for both economies, set at an arbitrary value of 2.
4. The red line represents the variable of interest for the low-exploration AI agent, while the green line depicts the same variable for the high-exploration AI agent.
5. A vertical dashed line at period 0 signifies the occurrence of the regime change on the top panel. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% daily growth rate.
6. In the bottom panel, Period 0 represents the initial period of the first regime.

Figure 4.2 displays the wealth levels for AI agents of different exploration levels. The measure of wealth at period t is defined as $y_t + s_t^{\alpha}$, where a lower storage position indicates reduced wealth for the same income, y_t. I find that in the initial regime, the high-exploration agent accumulates less wealth. After the regime change, the wealth level for the high-exploration agent is higher than that for the low-exploration agent, due to adjustments made by the AI agents during the regime change.

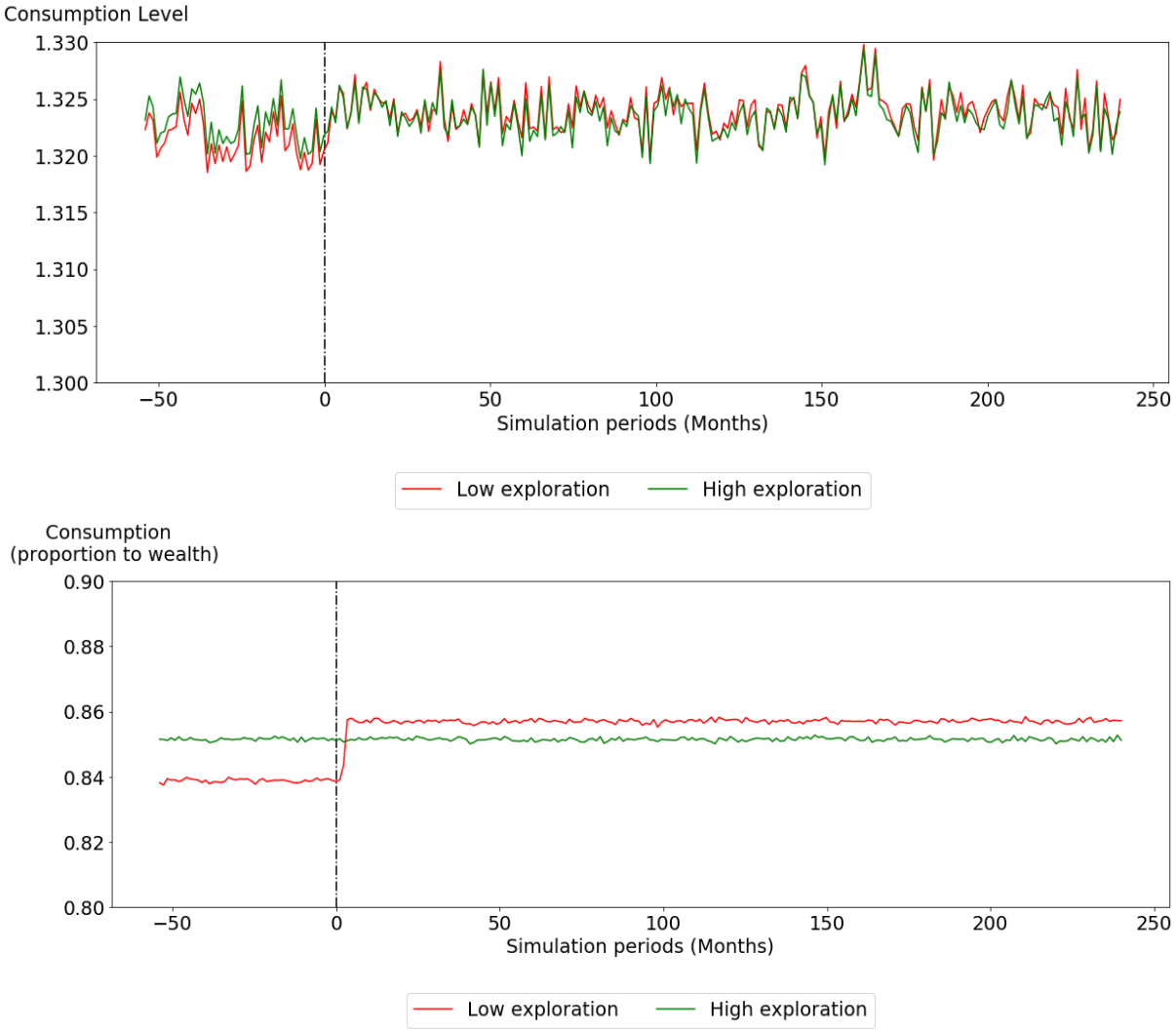
I plot the AI agents' consumption, storage, and liquidity holding decisions to better understand their adaptive behavior and the consequential aggregate dynamics presented in this subsection.

4.5.3 Consumption, Storage, and Liquidity Holding

I plot the AI agents' consumption, storage, and liquidity holding decisions during the regime change in Figures 4.3, 4.4, and 4.5.

In Figure 4.3, the plots commence 50 months prior to the regime shift. The regime change is denoted as period 0 on the x-axis. Before the regime shift, i.e., to the left of simulation period 0, Figure 4.3 shows that the high-exploration AI agent consumes about 85% of its wealth, whereas the low-exploration agent consumes roughly 84% of its wealth. Their consumption levels are roughly similar, with the high-exploration agent's consumption being modestly higher than that of the low-exploration agent. Once the regime shifts, I find that the high-exploration agent maintains a similar level of consumption, whereas the low-exploration agent increases its consumption level by around 0.1%.

Figure 4.3: Consumption of AI Agents during a Regime Change

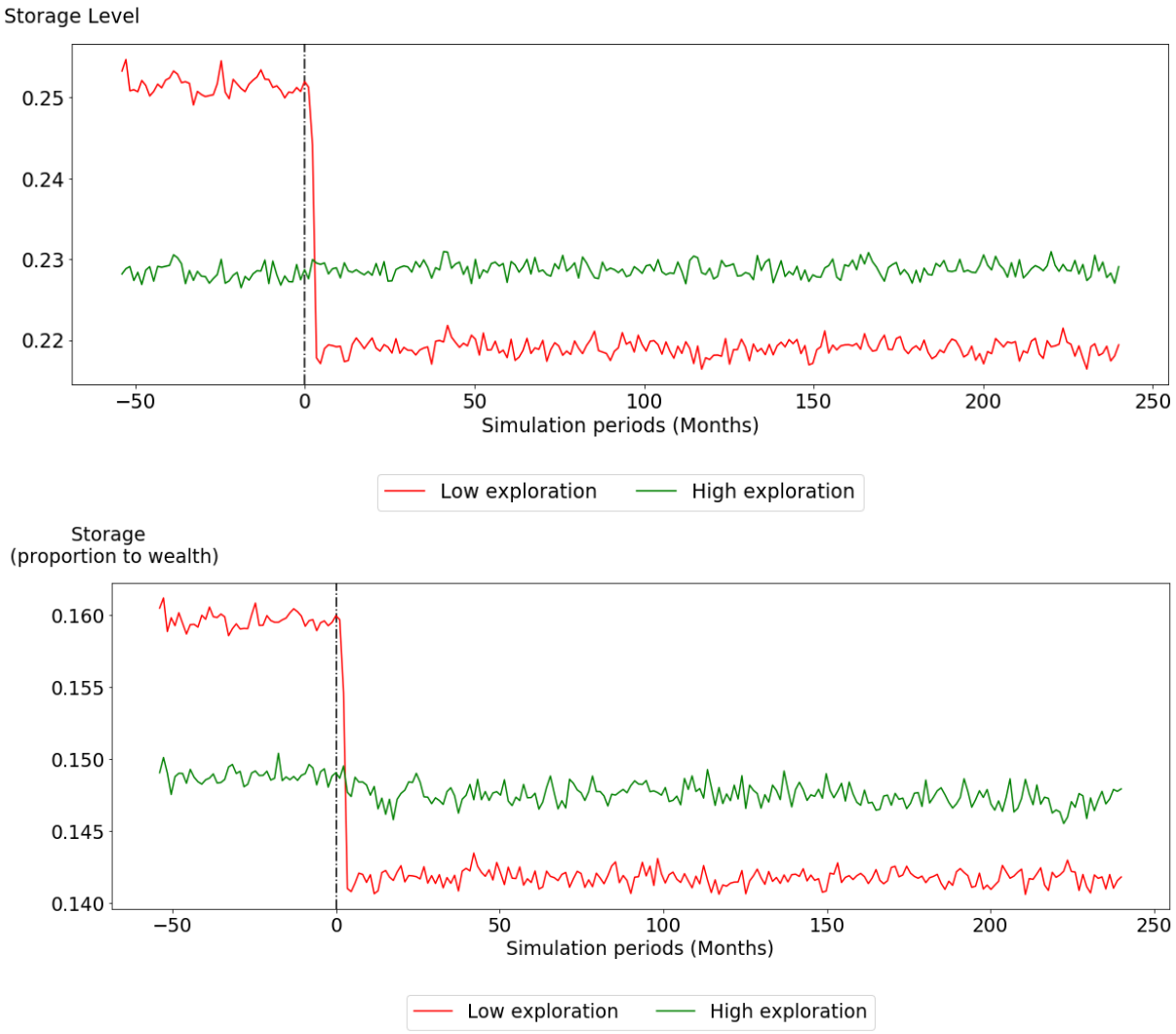


Notes:
1. The x-axis depicts simulation periods in months.
2. The y-axis represents the consumption level in the top panel, and the proportion of consumption to wealth in the bottom panel, where wealth at period t is defined as $y_t + s_t^{\alpha}$, encompassing income and prior storage.
3. A vertical dashed line at period 0 indicates the occurrence of the regime change. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% daily growth rate.
4. The red line illustrates the variable of interest for the low-exploration AI agent, while the green line represents the same variable for the high-exploration AI agent.

In this environment, in addition to storage, AI agents can also choose to keep real resources as real balances to reduce transaction costs during consumption. Given these two types of choices, high and low exploration agents behave differently in both regimes.

Figures 4.4 and 4.5 display the AI agents' two allocation choices - storage and liquidity holding. In the initial regime, the high-exploration agent stores less but accumulates more real balance than the low-exploration AI agent. Once the regime shifts, the high-exploration AI agent maintains a similar level of storage and adapts by reducing its demand for real balance. The low-exploration AI agent, however, reduces both its storage and demand for real balance.

Figure 4.4: Storage of AI Agents during a Regime Change

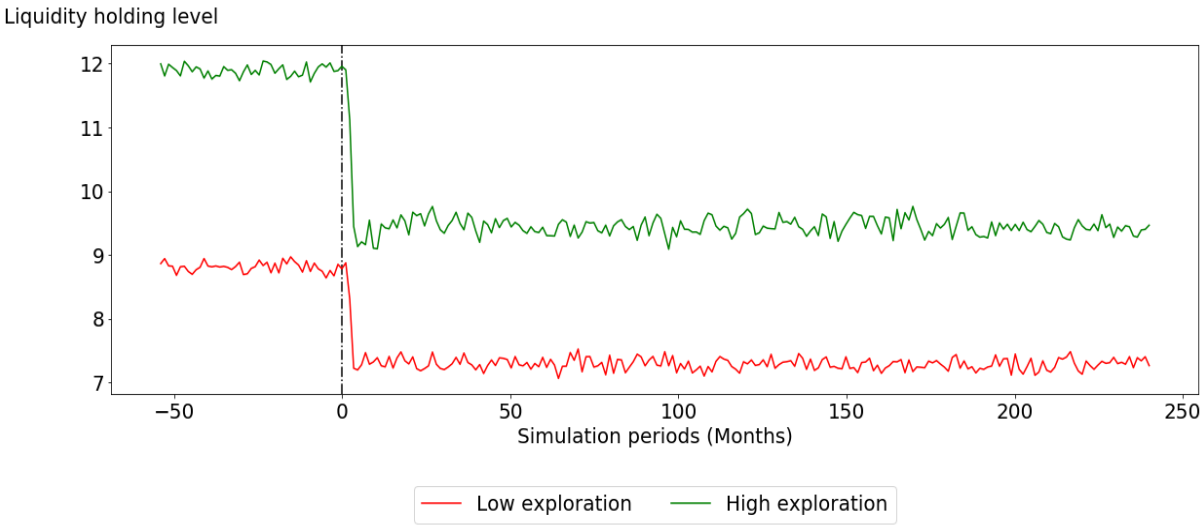


Notes:
1. The x-axis depicts simulation periods in months.
2. The y-axis represents the savings/storage level in the top panel, and the proportion of savings to wealth in the bottom panel, where wealth at period t is defined as $y_t + s_t^{\alpha}$, encompassing both income and previous storage.
3. A vertical dashed line at period 0 indicates the occurrence of the regime change. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% daily growth rate.
4. The red line illustrates the variable of interest for the low-exploration AI agent, while the green line represents the same variable for the high-exploration AI agent.

More specifically, I find that the high-exploration agent maintains a storage level of 0.23, whereas the low-exploration agent maintains a storage level of 0.25. Moreover, in this regime, the high-exploration agent maintains a liquidity holding level of 12, whereas the low-exploration agent maintains a liquidity holding of around 9.

Once the regime shifts, both agents adapt by reducing their demand for real balances. The high-exploration agent reduces its liquidity holdings by 25%, resulting in a new level of 9, while the low-exploration agent decreases its liquidity holdings by 22%, reaching a level of 7. Furthermore, while the high-exploration agent maintains its storage position, the low-exploration agent reduces its storage from 0.25 to 0.22. This reduction in storage contributes to the observed decline in aggregate wealth following the regime shift.

Figure 4.5: Real balance



Notes:
1. The x-axis represents simulation periods in months.
2. The y-axis displays the level of liquidity holding by the AI agents.
3. A vertical dashed line at period 0 signifies the occurrence of the regime change. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% annual growth rate.
4. The red line represents the variable of interest for the low-exploration AI agent, while the green line depicts the same variable for the high-exploration AI agent.

Through this simulation exercise, I find that the high-exploration agent consistently maintains a higher level of real balance than the low-exploration agent. However, regarding the decision on storage, it varies depending on the regime the agent is in. In the low-inflation regime, the high-exploration agent saves less than the low-exploration agent, whereas in the high-inflation regime, the high-exploration agent saves more.

4.5.4 AI vs RE agent(s)

It's crucial to emphasize that although an RE agent performs well in a relatively stable environment, AI agents adept in adapting to changes within their environment.

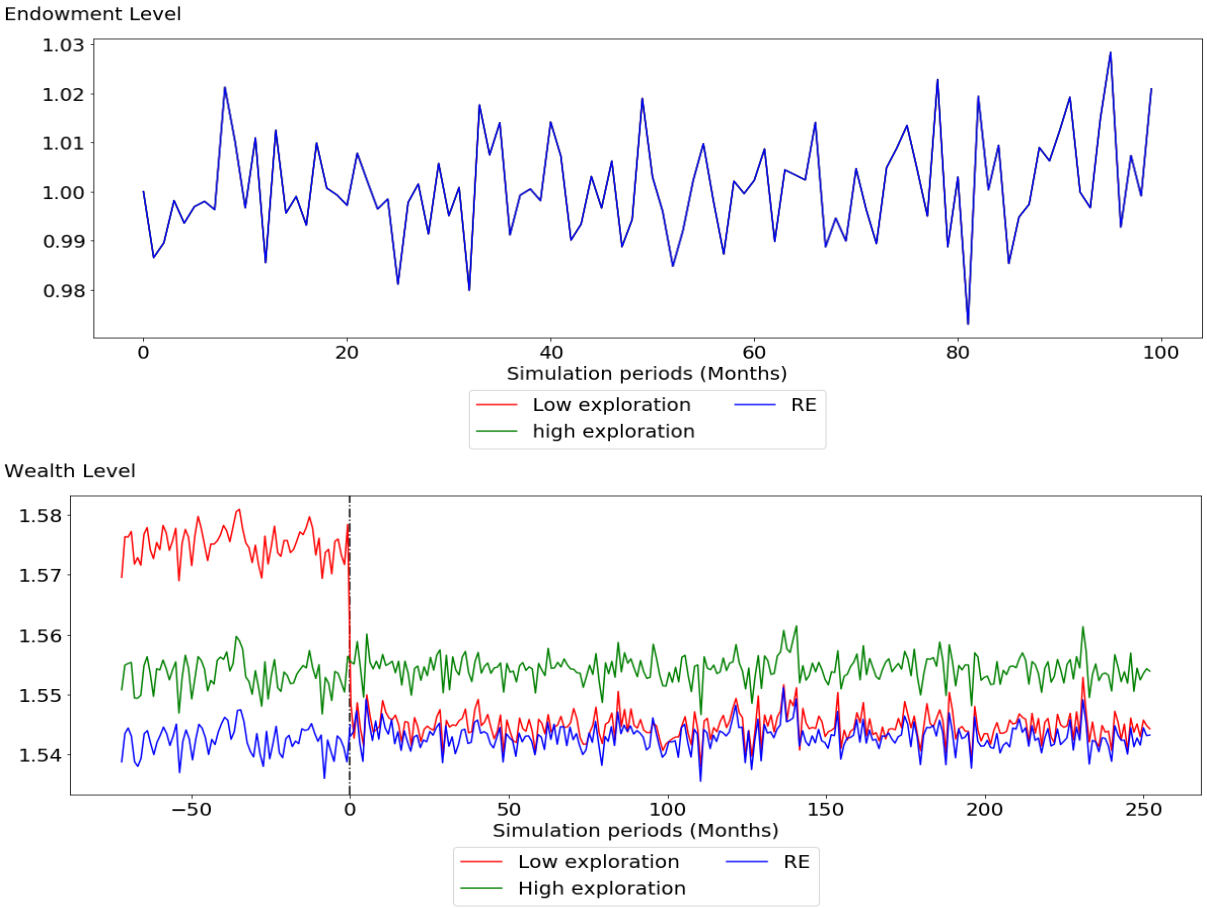
This adaptability is primarily attributed to the exploration mechanisms inherent in deep RL algorithms, setting them apart from the rigid framework of RE agents. As a result, AI agents are capable of adjusting their behaviors in response to environmental shifts, and their beliefs never fully align with those of an RE agent, sidestepping the limitations posed by the Lucas Critique.

To demonstrate this distinction, I present the behavior of an RE agent within the same environmental conditions across the two regimes in Figures 4.6 and 4.7.[10] Please note that I solve for the RE equilibrium in two regimes separately, then piece the two simulations together, labeling this combined sequence as the RE agent's behaviour. This approach is taken to sidestep the argument over whether the RE agent is aware of the regime change or if it occurs unexpectedly to them. Additionally, I include the performance of AI agents with both low and high exploration levels, as introduced in the preceding section. This comparative illustration underscores the differences between AI agents and the RE agent.

The top panel of Figure 4.6 depicts the endowment process for both AI and RE agents, illustrating that they operate in an identical environment with the same shock process. As all agents operate under the same conditions, only one line is observable in the figure.

[10]The RE equilibrium is solved following the methodology outlined in Schmitt-Grohé and Uribe [2004], utilizing a second-order approximation to the policy function.

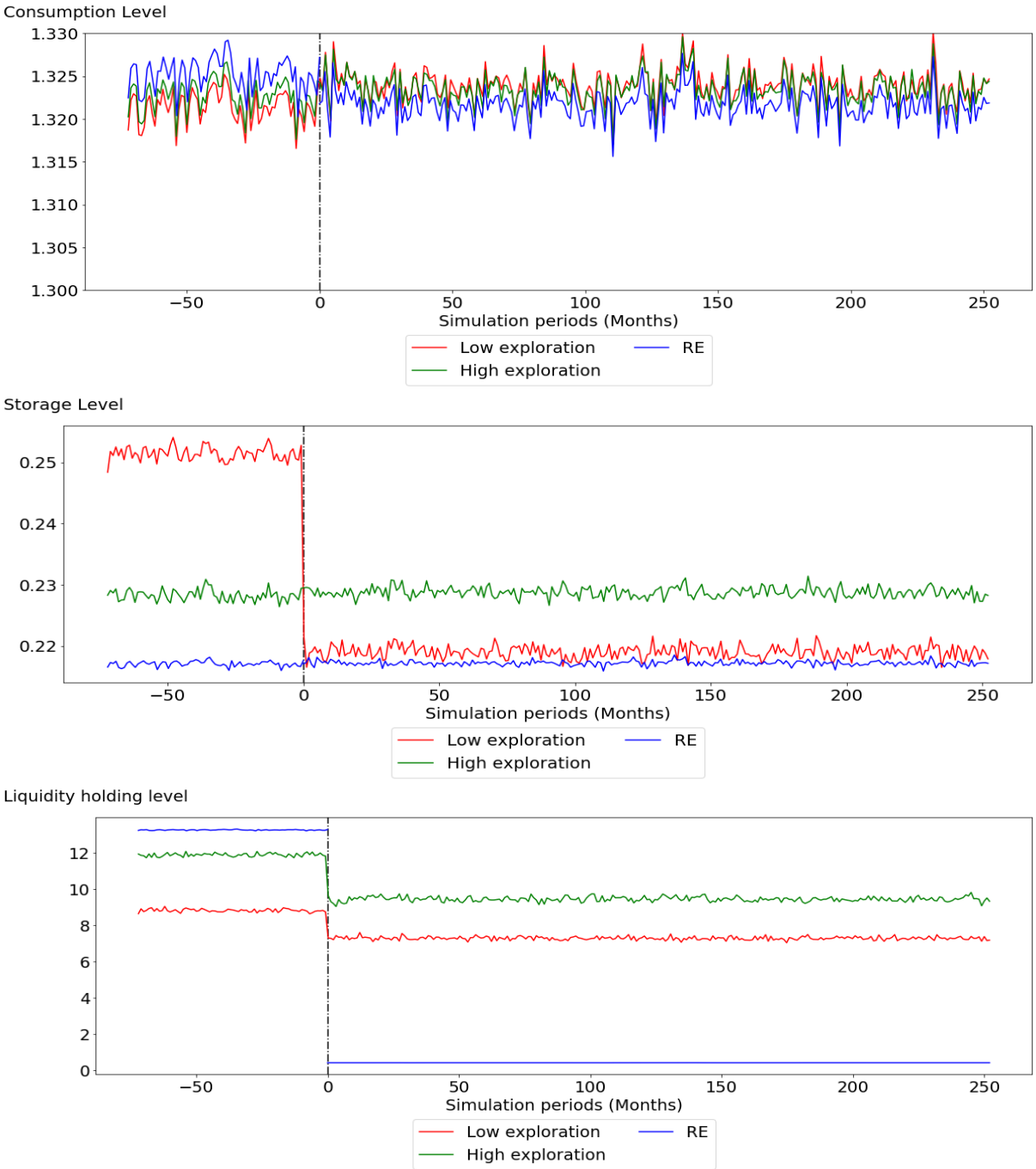
Figure 4.6: AI vs RE Agents Before and After a Regime Change



Notes:

1. The x-axis depicts simulation periods in months.
2. The y-axis represents endowment level on the top panel, and wealth on the bottom panel.
3. A vertical dashed line at period 0 indicates the occurrence of the regime change. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% daily growth rate.

Figure 4.7: AI vs RE Agents Before and After a Regime Change



Notes:

1. The x-axis depicts simulation periods in months.
2. The y-axis represents consumption level, storage level, and liquidity holding level, from the top panel to the bottom panel, respectively.
3. A vertical dashed line at period 0 indicates the occurrence of the regime change. This regime change involves an acceleration of the nominal money supply from a 0% to a 1% daily growth rate.

Figure 4.7 presents the consumption, storage, and real balance paths of all agents. It becomes evident that, across both regimes, the consumption levels of all agents display similarities, yet they are not identical. However, AI agents exhibit a tendency to save more than the RE agent across both regimes.

Furthermore, there is a distinct variation in the liquidity holding levels among all agents. More specifically, the liquidity holding levels of the AI agents are lower than those of the RE agent in the initial regime. Once the regime shifts, AI agents reduce their liquidity holding level and consistently maintain an average level of approximately 8. In contrast, the liquidity holdings of the RE agent are significantly lower, at a level around 0.4.

Moreover, given their higher saving levels compared to the RE agent, AI agents accumulate more wealth, as shown in the bottom panel of Figure 4.6. In particular, the low-exploration agent accumulates around 0.1% more wealth than the RE agent in the new regime, whereas the high-exploration agent accumulates around 0.7% more wealth than the RE agent.

In this section, I conduct a regime change experiment to illustrate the crucial role of exploration in influencing AI agents' adaptability. I also compare AI agents' simulation paths with those of the RE agent to highlight their differences. I find that the AI agents can adapt to a regime shift by adjusting their consumption, storage, and real balance decisions, leading them to save more than an RE agent in both the initial and subsequent regimes. This results in at least a 0.1% increase in wealth accumulation for the AI agents compared to the RE agent's performance. However, regarding liquidity holding levels, the AI agents' levels are found to be in between the range that is bounded by the RE agent's liquidity holding level in both regimes.

4.5.5 Discussion

The findings in the previous simulations demonstrate a departure from the processes observed in the naive adaptive expectations and accelerationist controversy literature, as AI agents in this study showcase the ability to adapt their beliefs in response to an accelerating money supply.

The representative AI agent changes its behaviors by becoming aware of policy changes through interactions with the economic environment. By making explorative decisions based on the current state and observing the subsequent state and associated reward signal, the agent adjusts its policy to maximize long-term rewards when the current decision-making strategy no longer generates high rewards. This adaptive behavior enables the agent to respond effectively to changes in monetary policy targets.

This aligns with the findings of Cavallo et al. [2017] in their experiments, which demonstrate that private agents are more likely to adjust their inflation expectations in response to price changes in supermarkets rather than relying solely on observed inflation statistics or central bank announcements regarding inflation target adjustments. The direct impact of supermarket price changes on consumers' perceived welfare makes them more influential in shaping agents' inflation expectations.

It is important to emphasize that exhibiting adaptive behaviors driven by their exploratory nature also entails that the AI agents will not fully converge to the steady state values.

It is also worth noting that there are several limitations and areas to explore in implementing deep RL algorithm in economics as a way to model human behaviors. A few important points to consider are:

- Slow Learning: Deep RL algorithms have faced criticism for their relatively slow learning and updating processes. The gradual nature of learning in these algorithms may impede AI agents' ability to attain close to the optimal belief within a single generation, especially in complex environments. As explained by Botvinick et al. [2019], the slow learning primarily results from incremental parameter adjustment and weak inductive bias within the algorithm. However, given the rapid evolution of this field, many new deep RL algorithms have been proposed to mitigate this issue and accelerate the learning process. For instance, inspired by Gershman and Daw [2017], deep RL algorithms incorporating episodic memory are under development, enabling AI agents to learn from the experiences gathered by others and potentially reducing the number of required simulation periods.

- Utility/Reward Design: Implementing the regime change experiment presents a significant challenge: designing the change as a minor shift in the nominal balance supply makes it difficult for AI agents to notice and adjust. This difficulty stems partly from their exploration mechanisms and partly from the lack of major shifts in rewards. Future studies should focus more on improving this reward signal mechanism, ensuring it aligns with the economic intuition of utility and effectively signals AI agents to adjust their beliefs.

- Estimation: Incorporating the exploration mechanism helps integrate theoretical human-like behaviors, leading to noticeable differences between agents' behaviors. However, a crucial unaddressed aspect is aligning parameters with real data or actual human behaviors. Addressing this would make the quantitative implications of exploration more tangible and greatly enhance their applicability in policy analysis.

4.6 Summary

In this chapter, I conducted simulation experiments to explore the adaptability of a representative AI agent in the context of a structural change, drawing inspiration from the accelerationist controversy literature. The primary goal was to showcase the effectiveness of the deep RL framework in a scalable general equilibrium model, enabling the creation of an artificial agent capable of adapting to structural changes due to its explorative nature. This contribution enriches the existing literature on general equilibrium models, particularly in understanding the behavior of agents with bounded rationality when faced with structural shifts.

To set up the economic environment, I employed a transaction cost of money demand model, introducing a monetary policy regime change that transitioned the nominal money supply process from a constant supply to an accelerating nominal money supply with a daily growth rate of 1%. Through this regime change, I demonstrated the AI agent's adaptability to the new monetary dynamics, particularly evident in its consumption-saving and liquidity-holding decisions.

Upon implementing the regime change, both AI agents promptly adjusted their behaviors but in terms of different allocation choices. Exploration not only impacts the levels of different decisions made by AI agents but also influences which decisions the agent adjusts during a regime shift, given that the AI agent has to make two decisions each period. The low-exploration AI agent adjusted its storage position as well as the liquidity-holding decision, whereas the high-exploration AI agent mainly adjusted its decision on liquidity holding.

The high-exploration agent consistently demands a higher level of real balance than the low-exploration agent in both regimes. Additionally, the agents' decisions on consumption and storage also differ. Initially, the high-exploration agent accumu-

lates 33% more real balances than the low-exploration agent. After the regime shift, the high-exploration agent maintains a similar level of consumption and storage but drastically reduces its real balance holdings by 25%. Conversely, the low-exploration agent decreases its storage by 12%, reduces its liquidity holdings by 22%, and modestly increases its consumption level.

Lastly, I showed that the agents' beliefs do not align with those of the RE agent due to their exploratory nature. However, the suboptimal belief (as measured by comparing it with the RE belief) is compensated for by the AI agents' ability to adapt and adjust to a regime change, which is closer to how humans learn and make decisions in real life.

The findings from this study underscore the utility of the deep RL framework as a valuable tool for modeling artificial agents within an economy undergoing structural changes. The incorporation of the exploration mechanism provided a theoretical, human-like approach to learning and adapting to changes. To enhance the reliability of quantitative predictions for policymakers, future research should focus on anchoring the exploration parameter through real data or actual human behaviors.

Appendix A

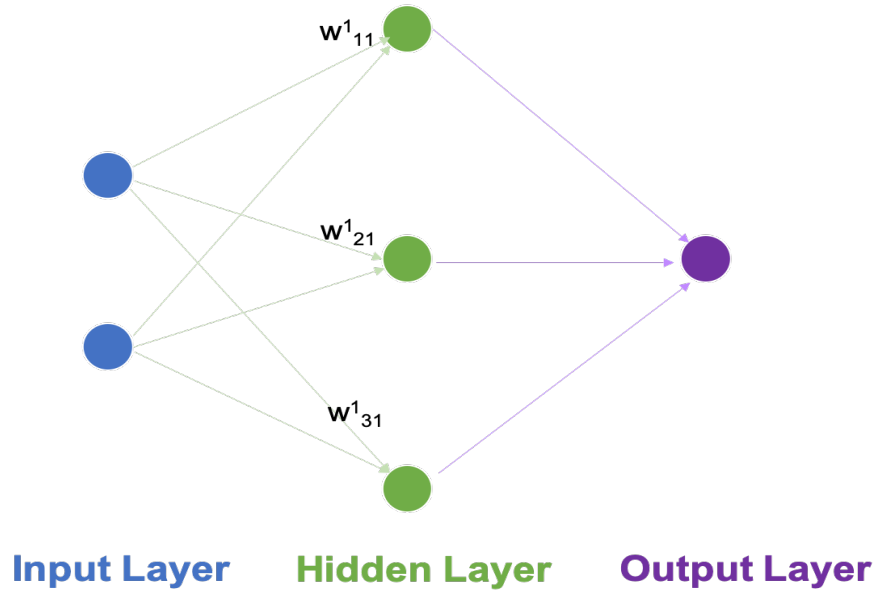
Deep Reinforcement Learning: Emerging Trends in Macroeconomics and Future Prospects

A.1 How do ANNs learn?

Approximating a function using a neural network requires the following steps:

- Determine the size of the input and output variables of the function, which serve as the inputs and outputs of the neural network.
- Specify the neural network's architecture by choosing a structure, such as a feedforward neural network in my case, and selecting the number of layers and nodes in each layer.
- Initialization: Establish the method for initializing the neural network's weights, which in this thesis involves random initialization.

Figure A.1: A Feedfoward Network with One Hidden Layer



Source: author's own construction

Figure A.1 illustrates an example feedforward network with a single hidden layer. Each circle in the diagram represents a neuron (or node) within this ANN. The first

column denotes the input layer, comprising two nodes in blue. The middle column represents the hidden layer, consisting of three nodes in green. Lastly, the output layer is featured in the last column, consisting of one node in purple. The arrows indicate the flow of information or data. In a feedforward network, data moves in a forward direction, from the input layer to the output layer. The weights, denoted as w, must be learned during the training process of an ANN. The superscripts on the weights indicate the layer to which they belong. For instance, w_{21}^1 represents the weight connecting the first neuron in the input layer (the first layer) to the second neuron in the hidden layer (the second layer).

The first node in the hidden layer serves as an example to demonstrate information flow within a node. Assuming that the nodes in the input layer output x_1 and x_2, respectively, the first node in the hidden layer receives information from the previous layer as $w_{11}^1 x_1 + w_{21}^1 x_2$ and applies an overall bias. The output of this node is then represented as $\sigma(w_{11}^1 x_1 + w_{21}^1 x_2 + bias)$, where $\sigma()$ denotes an activation function, which can take various forms such as the sigmoid, logistics, and tanh functions. One commonly used non-linear activation function is the rectified linear unit (ReLU).

How do ANNs learn and update their parameters? They employ the following procedures. Given a training dataset and an ANN model, the data is passed forward through the neural network to obtain an output or prediction. This output is then compared to a target value or goal. The loss between the predicted value and the target value is calculated. To ensure that the neural network learns and updates its parameters, including the weights of each neuron and biases, a means of connecting each weight and the loss is necessary. Backpropagation is an algorithm used to establish such connections. Specifically, it finds partial derivatives of the loss function with respect to each weight and bias by applying chain rules.

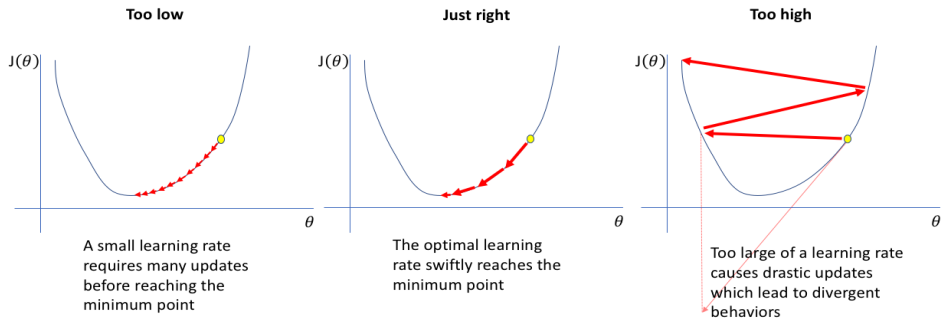
In literature, these partial derivatives are known as gradients. Using these gradients,

parameters can be updated through gradient methods, such as gradient descent. In the process of gradient descent, to update each weight of the network, the procedure is as follows.

$$w'_{ij} = w_{ij} - \eta \frac{\partial Error}{\partial w_{ij}}, \tag{1.1}$$

where w_{ij} represents weight of the j^{th} neuron to the i^{th} neuron in the next layer. $\frac{\partial Error}{\partial w_{ij}}$ is the partial derivative of the loss function with respect to the weight, and it states how much error the weight w_{ij} contributed. The new weight w'_{ij} is a combination of the current weight w_{ij} and some weighted term $\eta \frac{dError}{dw_{ij}}$. η denotes learning rate, a hyperparameter. It represents how quickly weights are updated in response to the error contribution term. The higher the learning rate, the quicker the update is.

Figure A.2: Learning Rates



Source: jeremyjordan

To clearly illustrate the purpose of η, figure A.2 shows that when the learning rate is too high, it causes a divergence and fails to reach the appropriate weights; with a

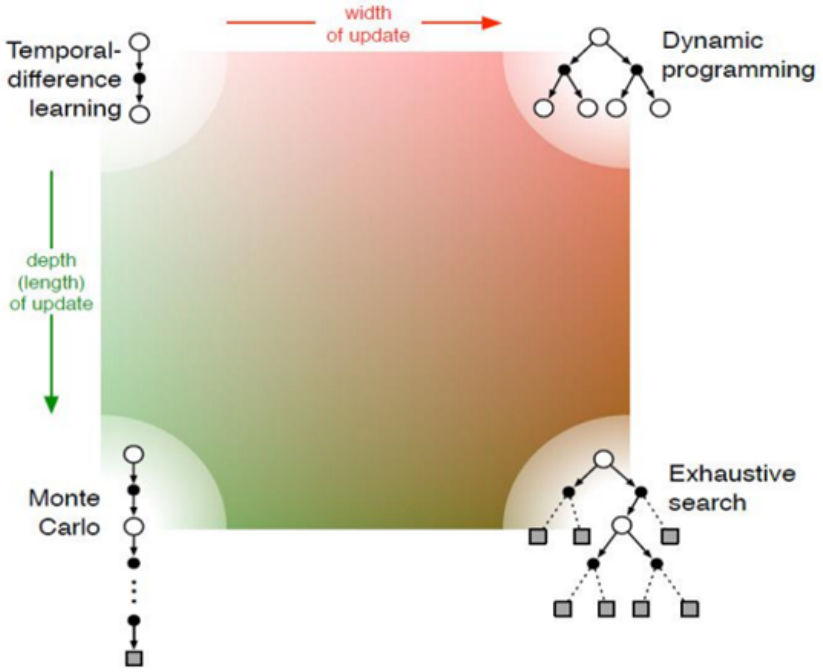
very low learning rate, the learning process takes a long time before it reaches the optimal solution.

The objective of gradient descent is to minimise a loss function. At the point where the loss is minimised, the gradient is 0. Thus, gradient descent requires weights adjustment so as to move along the line and to the minimum point, which is similar to what figure A.2 illustrates.

A.2 Comparing RL with Other Solution Methods for MDPs

Figure A.3 compares RL methods with the existing methods of optimal control. It has two dimensions. Along the vertical dimension, it shows the depth of updating, and the degree of bootstrapping. RL methods usually update at each step, in contrast to Monte Carlo method that requires a full depth of history before updating the target value. TD learning refers to the update of target value with the current new information, while Monte Carlo learning refers to the update of target value with the entire trajectory. Exhaustive search methods refer to the update of target value with all possible trajectories.

Figure A.3: Comparison of RL and Other Methods



Source: Sutton and Barto [2018]

Along the horizontal dimension is the width of updating. It shows whether the method is based on a sample trajectory of agent or expected updates. Expected updates mean that the method requires a fully specified transition dynamics, such as dynamic programming. RL methods, however, do not require all the transition information, and its methods are usually based on a sample trajectory.

RL possesses an advantage over dynamic programming algorithms in its capability to learn directly from raw data, eliminating the need for an explicit model of the environment. Furthermore, it surpasses Monte Carlo algorithms by learning without having to wait for an episode to conclude.

Appendix B

AI for Economic Agent Behaviors: Modeling Adaptability and Exploration in Dynamic Environments

B.1 Derivation of the Analytical Solution to the Stochastic Optimal Growth Model

I first derive the analytical solution and the value function for the case when the stochastic process follows $z_t = e^{\lambda + \epsilon_t}$.

Given the Bellman equation:

$$v(k_t, z_t) = \max_{k_{t+1} \in \Gamma(k_t)} \{log(z_t k_t^{\alpha} - k_{t+1}) + \beta E_t v(k_{t+1}, z_{t+1})\} \tag{1.1}$$

Guess the value function to be the form $v(k,z) = A + Blog(k) + Dlog(z)$, and substitute to the right hand side of the Bellman equation

$$\begin{aligned} v(k_t, z_t) &= \max_{k_{t+1}\in\Gamma(k_t)} \{log(z_t k_t^\alpha - k_{t+1}) + \beta E_t(A + Blog(k_{t+1}) + Dlog(z_{t+1}))\} \\ &= \max_{k_{t+1}\in\Gamma(k_t)} \{log(z_t k_t^\alpha - k_{t+1}) + \beta(A + Blog(k_{t+1}) + DE_t log(z_{t+1}))\} \end{aligned}$$

Given that $z_{t+1} = e^{\lambda+\epsilon_{t+1}}$, where $\epsilon_{t+1} \sim \mathcal{N}(0, 0.1)$, $E_t log(z_{t+1}) = \lambda$

$$v(k_t, z_t) = \max_{k_{t+1}\in\Gamma(k_t)} \{log(z_t k_t^\alpha - k_{t+1}) + \beta(A + Blog(k_{t+1}) + D\lambda)\} \tag{1.2}$$

The F.O.C:

$$\frac{\partial v(k_t, z_t)}{\partial k_{t+1}} = 0 \rightarrow -\frac{1}{z_t k_t^\alpha - k_{t+1}} + \beta\frac{B}{k_{t+1}} = 0 \tag{1.3}$$

$$k_{t+1} = \frac{\beta B}{1+\beta B} z_t k_t^\alpha \tag{1.4}$$

Apply the envelope theorem

$$\frac{B}{k_t} = \frac{\alpha z_t k_t^{\alpha-1}}{z_t k_t^\alpha - k_{t+1}} \rightarrow B = \frac{\alpha z_t k_t^\alpha}{z_t k_t^\alpha - k_{t+1}} \tag{1.5}$$

k_{t+1} can then be derived as

$$k_{t+1} = \alpha\beta z_t k_t^\alpha \tag{1.6}$$

If the guessed form of the value function is the solution then it must satisfy

$$A + Blog(k_t) + Dlog(z_t) = log(z_t k_t^{\alpha} - \alpha\beta z_t k_t^{\alpha}) + \beta(A + Blog(\alpha\beta z_t k_t^{\alpha}) + D\lambda) \quad (1.7)$$

$$A = \frac{1}{1-\beta}(log(1-\alpha\beta) + \frac{\alpha\beta}{1-\alpha\beta}log\alpha\beta + \frac{\beta\lambda}{1-\alpha\beta}) \quad (1.8)$$

$$B = \frac{\alpha}{1-\alpha\beta} \quad (1.9)$$

$$D = \frac{1}{1-\alpha\beta} \quad (1.10)$$

The value function is thus

$$v^*(k,z) = \frac{1}{1-\beta}(log(1-\alpha\beta) + \frac{\alpha\beta}{1-\alpha\beta}log\alpha\beta + \frac{\beta\lambda}{1-\alpha\beta}) + \frac{\alpha}{1-\alpha\beta}logk + \frac{1}{1-\alpha\beta}logz \quad (1.11)$$

For the case when z_t follows an autoregressive process, i.e. $z_t = e^{\lambda + \rho ln z_{t-1} + \epsilon_t}$, the analytical solution is as follows.

Equation 1.2 becomes:

$$v(k_t, z_t) = \max_{k_{t+1} \in \Gamma(k_t)} \{log(z_t k_t^\alpha - k_{t+1}) + \beta(A + Blog(k_{t+1}) + D\lambda + D\rho log(z_t)\} \tag{1.12}$$

Following the same derivation procedure, we have the same policy function:

$$k_{t+1} = \alpha\beta z_t k_t^\alpha \tag{1.13}$$

The value function is,

$$\begin{aligned} v^*(k, z) = \frac{1}{1-\beta}\left[log(1-\alpha\beta) + \frac{\alpha\beta}{1-\alpha\beta} log\alpha\beta + \frac{\beta\lambda}{(1-\alpha\beta)(1-\beta\rho)}\right] \\ + \frac{\alpha}{1-\alpha\beta} logk + \frac{1}{(1-\alpha\beta)(1-\beta\rho)} logz \end{aligned} \tag{1.14}$$

B.2 Neural Network Architectures and Parameters

In all of my experiments, the baseline ANN employs a feedforward architecture. I provide parameters and structures of the ANNs used in my experiments, including the number of hidden layers, the size of each layer, the chosen activation functions, along with the learning rate and batch size for ANN updates, in Table B.1.

Table B.1: ANN Architectures and Parameters

ANN Specifications	**Chapter 3** Networks	**Chapter 4** Networks
Number of Hidden Layers	2	2
Number of Nodes (per layer)	32	64
Activation functions	ReLU	ReLU
Learning Rate	Actor Network 1e-4 Critic Network 1e-3	Actor Network 1e-5 Critic Network 1e-4
Batch Size	128	256

I focus more on the exploration feature and how it impacts AI agents' adaptability. Therefore, I choose most neural network parameters based on existing practical guidelines[1], without rigorous cross-validation. I utilize the commonly used activation function, ReLU. The architecture, including the number of hidden layers and nodes, depends on the complexity of the underlying function and the size of the state and action vectors. As described in both chapters, the models are relatively simple with one or two action variables, and less than five state variables; therefore, I use two

[1]Some useful guides include:

- Activation functions in neural networks: `https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6`
- Configuring minibatch size: `https://machinelearningmastery.com/gentle-introduction-mini-batch-gradient-descent-configure-batch-size/`
- Understanding learning rates in deep learning: `https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/`

.

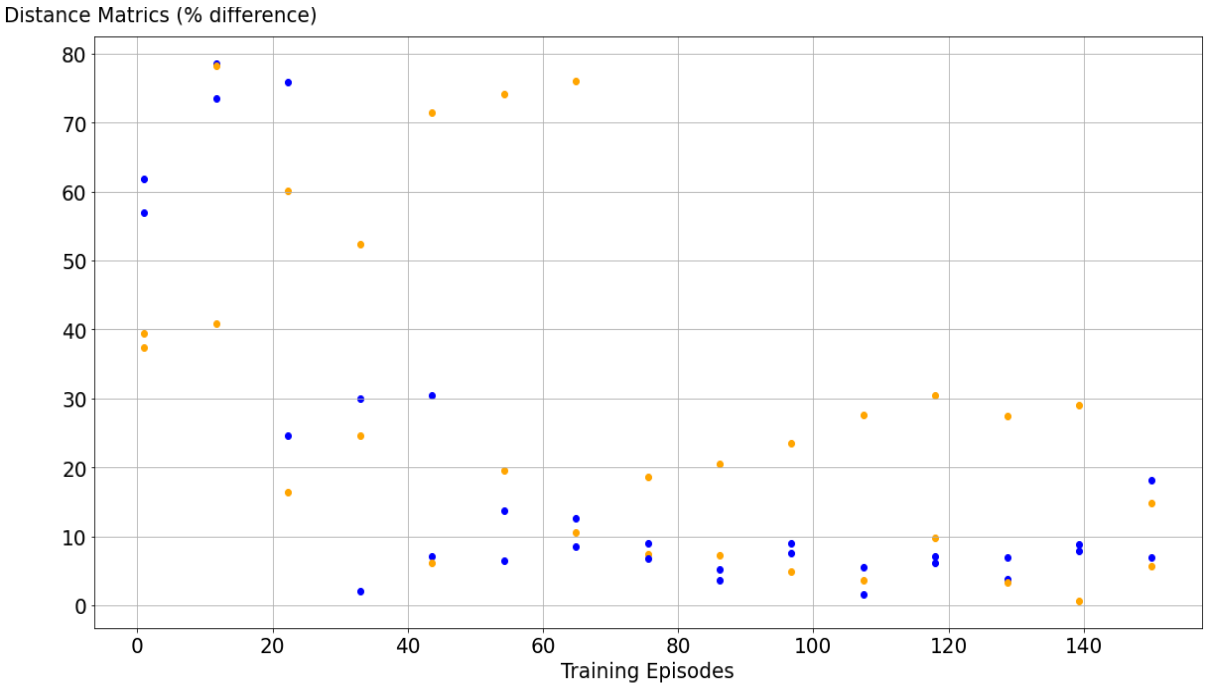
hidden layers with 32 and 64 nodes, respectively. The batch size largely depends on the memory requirements of the GPU or CPU hardware; typically, it is a power of two, such as 64, 128, 256, or 1024.

B.3 Additional Results: Random Initial Conditions during Training

Figure B.1 shows the distance metrics for AI agents beginning their training from random initial conditions, with blue dots replicating the data from Figure 3.3. An additional layer of orange dots has been introduced to represent the distance metrics between a RE agent and AI agents trained under these random initial conditions. Here, 'random initial conditions' refer to initial endowments that are randomly selected from a normal distribution.

This visualization indicates that starting training with random initial conditions may hinder the learning process, as evidenced by the increased distance in the policy functions between an RE agent and an AI agent. This finding suggests a valuable avenue for future research to explore the precise impact of initial conditions on the efficiency of the learning process.

Figure B.1: Distance Metrics



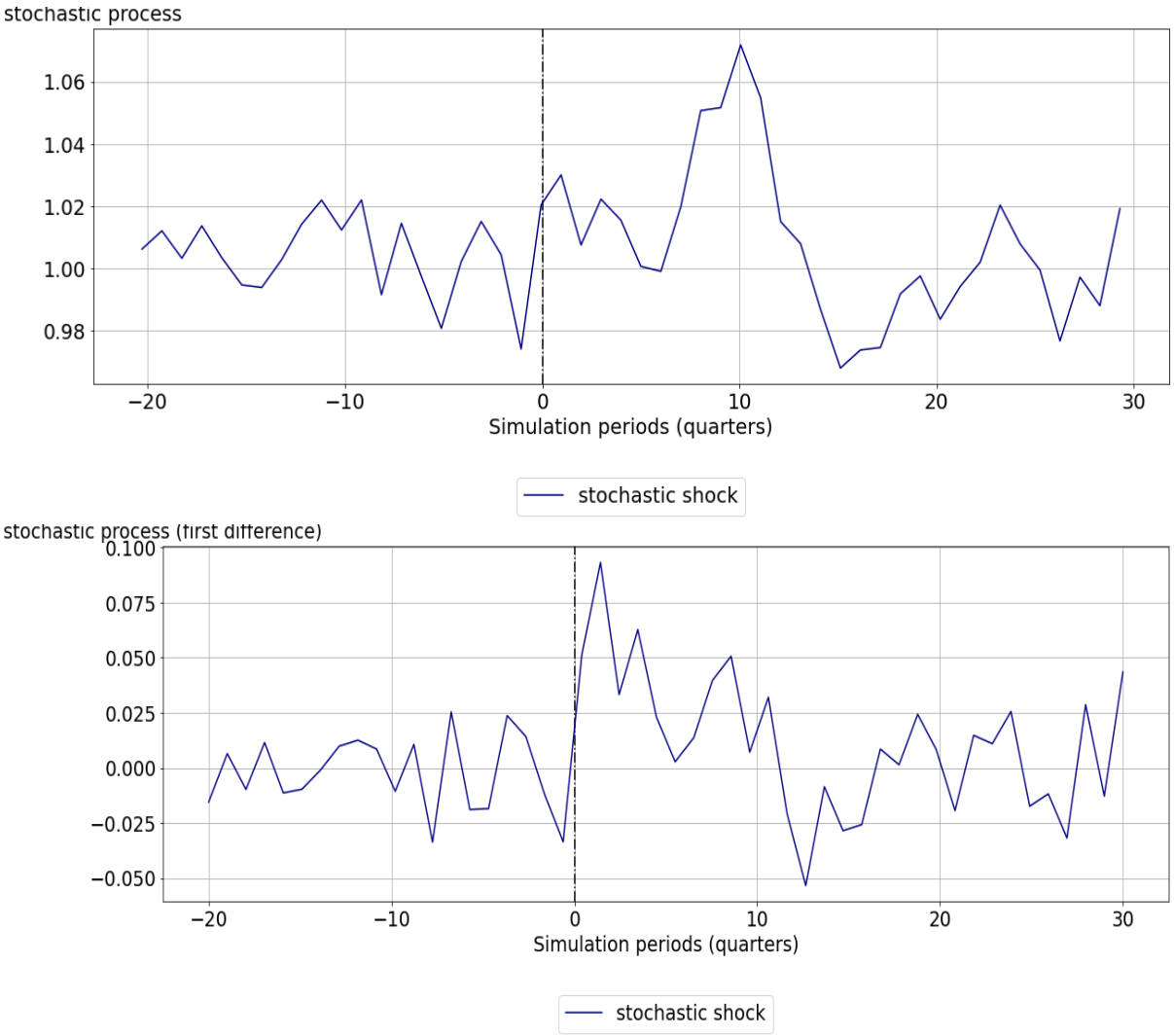
Notes:
1. y-axis represents the percentage distance according to Equation (4.14), i.e., $d_e = \frac{1}{G}\sum_{g=1}^{G}\frac{|k_g^* - k_g|}{k_g^*}$.
2. x-axis represents how long the AI agent has been living and updating its policy function. Each episode encompasses 5000 steps of updating policy and value functions.
3. A downward trend can be observed as the AI agent learns more in the economy.

B.4 Additional Results: Zero Mean for the Shock Process

In the model specification, I defined the stochastic process as $z_t = e^{0.1+\epsilon_t}$. In this section, I introduce additional results obtained by setting the shock location parameter to 0, i.e., $z_t = e^{\epsilon_t}$. The permanent change involves a shift from $z_t = e^{\epsilon_t}$ to $z_t = e^{0.7\ln(z_{t-1})+\epsilon_t}$.

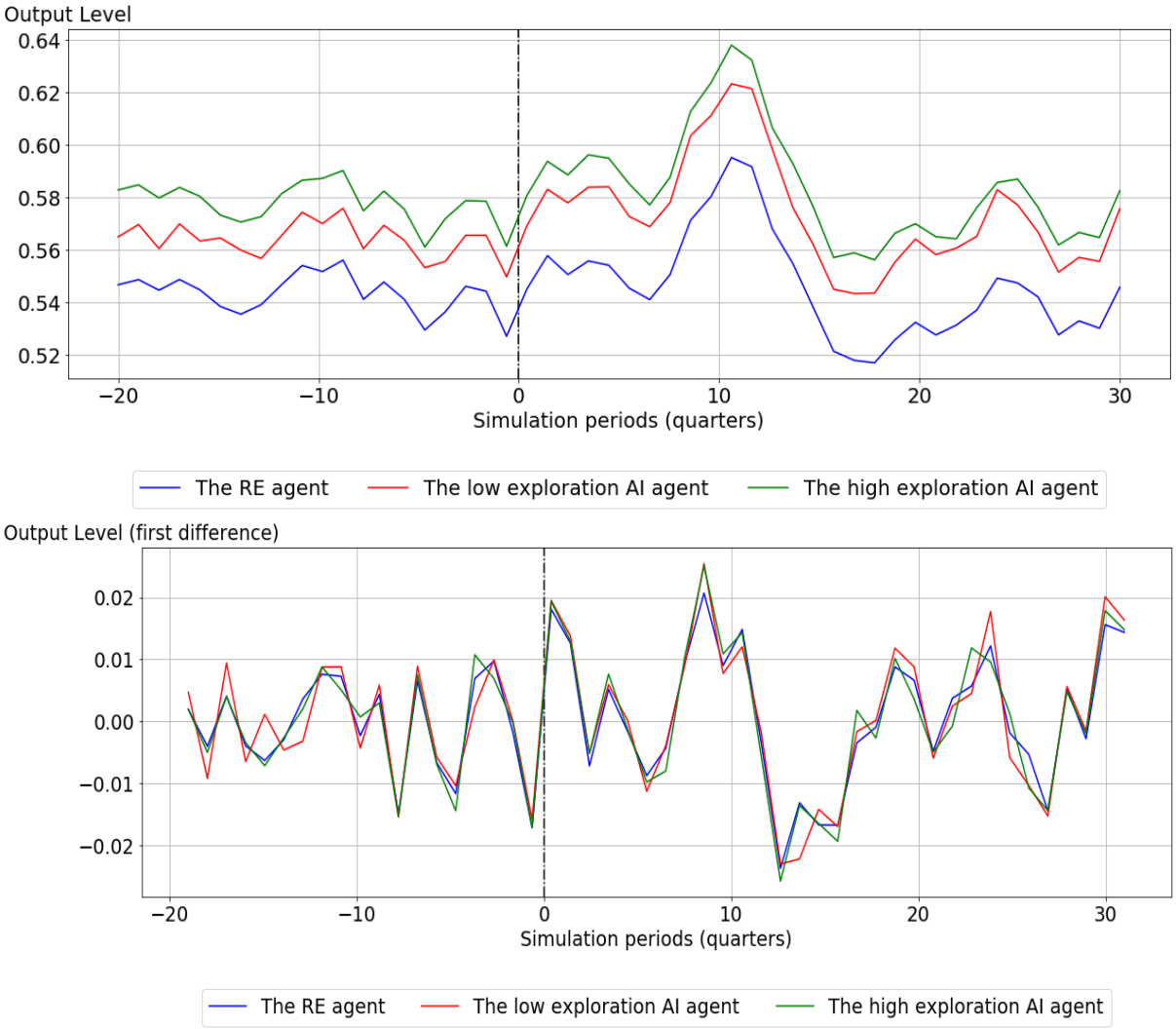
A similar conclusion can be drawn with the setup of a zero shock location parameter for the shock process: AI agents tend to save more than the RE agent. Furthermore, an AI agent that engages in greater exploration tends to save more than an agent that explores less.

Figure B.2: The Stochastic Process (in level and first difference)



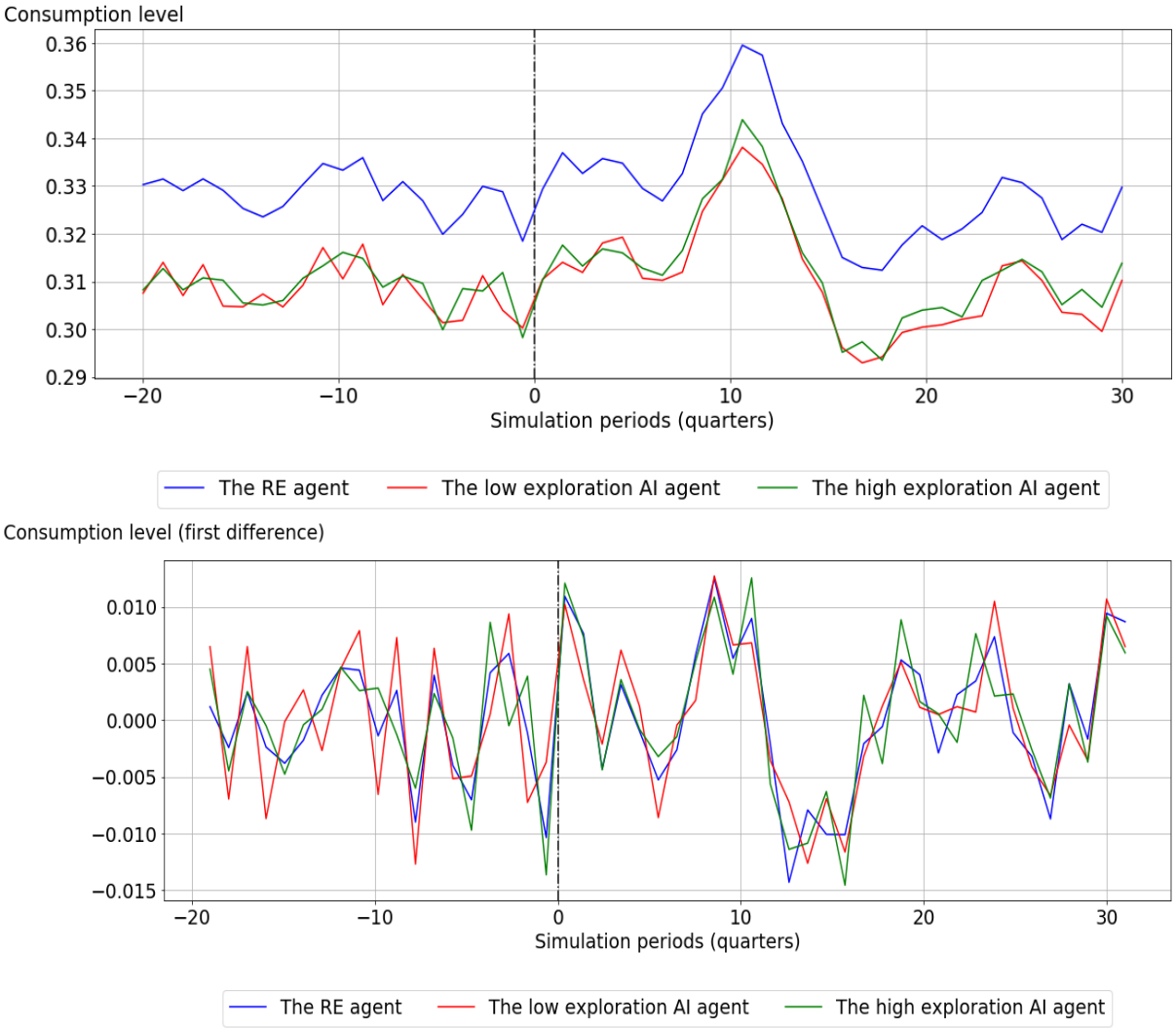
Notes:
1. The x-axis depicts simulation periods in quarters.
2. At marked period 0, a stochastic shock transition occurs from drawing from an i.i.d process to an AR(1) process.
3. The i.i.d process is represented as follows: $z_t = e^{\epsilon_t}$
4. The AR(1) process is represented as follows: $z_t = e^{0.7 \ln z_{t-1} \epsilon_t}$
5. The y-axis represents the stochastic process on the top panel and the first difference transformation of the same stochastic process on the bottom panel.

Figure B.3: Output (in level and first difference)



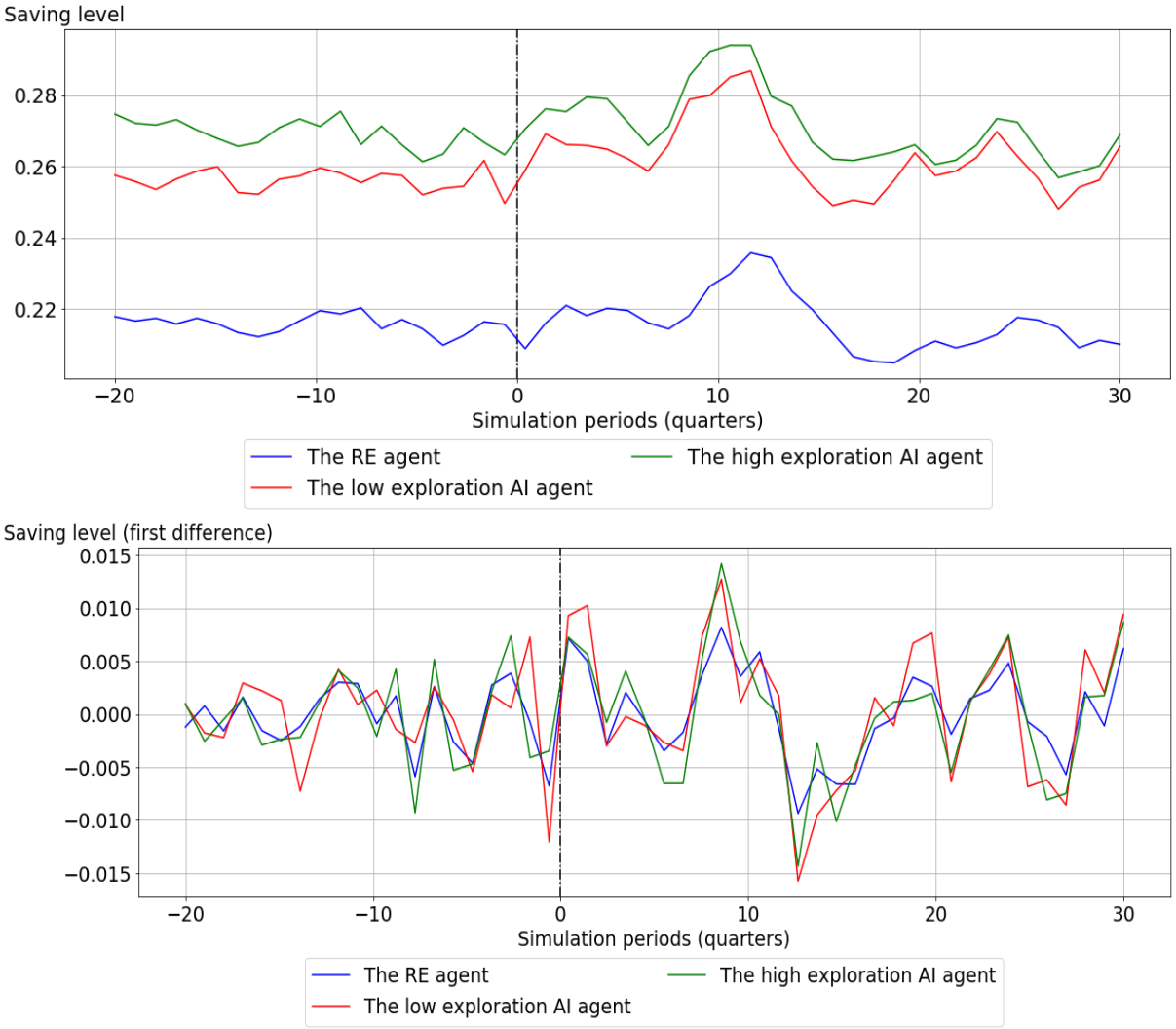
Notes:
1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the output level on the top panel and its first difference transformation on the bottom panel.

Figure B.4: Consumption (in level and first difference)



Notes:
1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the consumption level on the top panel and its first difference transformation on the bottom panel.

Figure B.5: Savings (in level and first difference)



Notes:

1. The x-axis denotes simulation periods in quarters.
2. The y-axis indicates the savings level on the top panel and its first difference transformation on the bottom panel.

Bibliography

Tanay Agrawal. *Hyperparameter optimization in machine learning: make your machine learning and deep learning models more efficient*. Springer, 2021.

Susan Amin, Maziar Gomrokchi, Harsh Satija, Herke van Hoof, and Doina Precup. A survey of exploration methods in reinforcement learning. *arXiv preprint arXiv:2109.00157*, 2021.

W Brian Arthur. Designing economic agents that act like human agents: A behavioral approach to bounded rationality. *The American economic review*, 81(2): 353–359, 1991.

Julian Ashwin, Paul Beaudry, and Martin Ellison. The unattractiveness of indeterminate dynamic equilibria. 2021.

Tohid Atashbar and Rui (Aruhan) Shi. Ai and macroeconomic modeling: Deep reinforcement learning in an rbc model. *IMF Working Paper Series No. 2023/040*, 2023.

Susan Athey. *The impact of machine learning on economics*, pages 507–547. University of Chicago Press, 2018.

Manoj Atolia, Santanu Chatterjee, and Stephen J Turnovsky. How misleading is linearization? evaluating the dynamics of the neoclassical growth model. *Journal of Economic Dynamics and Control*, 34(9):1550–1571, 2010.

Yu Bai, Chi Jin, Huan Wang, and Caiming Xiong. Sample-efficient learning of stackelberg equilibria in general-sum games. *Advances in Neural Information Processing Systems*, 34:25799–25811, 2021.

Angela Barriga, Adrian Rutle, and Rogardt Heldal. Automatic model repair using reinforcement learning. In *MoDELS (Workshops)*, pages 781–786, 2018.

Andrew G Barto and Satinder Pal Singh. On the computational economics of reinforcement learning. In *Connectionist Models*, pages 35–44. Elsevier, 1991.

Andrew G Barto and Richard S Sutton. Reinforcement learning in artificial intelligence. In *Advances in Psychology*, volume 121, pages 358–386. Elsevier, 1997.

Richard Bellman. A markovian decision process. *Journal of mathematics and mechanics*, pages 679–684, 1957.

Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

Lena Mareen Boneva, R Anton Braun, and Yuichiro Waki. Some unpleasant properties of loglinearized solutions when the nominal rate is zero. *Journal of Monetary Economics*, 84:216–232, 2016.

Mathew Botvinick, Sam Ritter, Jane Wang, Zeb Kurth-Nelson, Charles Blundell, and Demis Hassabis. Reinforcement learning, fast and slow. *Trends in Cognitive Sciences*, 23, 04 2019. doi: 10.1016/j.tics.2019.02.006.

Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolo, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

Pablo S Castro, Ajit Desai, Han Du, Rodney Garratt, and Francisco Rivadeneyra. Estimating policy functions in payment systems using reinforcement learning. *Available at SSRN 3743017*, 2020.

Alberto Cavallo, Guillermo Cruces, and Ricardo Perez-Truglia. Inflation expectations, learning, and supermarket prices: Evidence from survey experiments. *American Economic Journal: Macroeconomics*, 9(3):1–35, July 2017. doi: 10.1257/mac.20150147. URL https://www.aeaweb.org/articles?id=10.1257/mac.20150147.

Arthur Charpentier, Romuald Elie, and Carl Remlinger. Reinforcement learning in economics and finance. 2020.

Mingli Chen, Andreas Joseph, Michael Kumhof, Xinlei Pan, Rui Shi, and Xuan Zhou. Deep reinforcement learning in a monetary model. *Rebuilding Macroeconomics Working Paper Series*, (58), 2021.

Andrei Cheremukhin. Personalized ad recommendation systems with deep reinforcement learning. 2018.

Matias Covarrubias. Dynamic oligopoly and monetary policy: A deep reinforcement learning approach. 2022. URL https://www.matias-covarrubias.com/jobmarketpaper.

Michael Curry, Alexander Trott, Soham Phade, Yu Bai, and Stephan Zheng. Finding general equilibria in many-agent economic simulations using deep reinforcement learning. *arXiv preprint arXiv:2201.01163*, 2022.

Francesco D'Acunto, Ulrike Malmendier, Juan Ospina, and Michael Weber. Exposure to grocery prices and inflation expectations. *Journal of Political Economy*, 129(5):1615–1639, 2021. doi: 10.1086/713192. URL https://doi.org/10.1086/713192.

Ishita Dasgupta, Jane Wang, Silvia Chiappa, Jovana Mitrovic, Pedro Ortega, David Raposo, Edward Hughes, Peter Battaglia, Matthew Botvinick, and Zeb Kurth-Nelson. Causal reasoning from meta-reinforcement learning. *arXiv preprint arXiv:1901.08162*, 2019.

Zihan Ding and Hao Dong. Challenges of reinforcement learning. *Deep Reinforcement Learning: Fundamentals, Research and Applications*, pages 249–272, 2020.

Wei Du and Shifei Ding. A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications. *Artificial Intelligence Review*, 54: 3215–3238, 2021.

Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, 2021.

Vedran Dunjko, Jacob M Taylor, and Hans J Briegel. Advances in quantum reinforcement learning. In *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 282–287. IEEE, 2017.

BWAC Farley and W do Clark. Simulation of self-organizing systems by digital computer. *Transactions of the IRE Professional Group on Information Theory*, 4(4):76–84, 1954.

Thomas G. Fischer. Reinforcement learning in financial markets - a survey. 2018.

Drew Fudenberg and David K Levine. Continuous time limits of repeated games with imperfect public monitoring. *Review of Economic Dynamics*, 10(2):173–192, 2007.

Maxime Gasse, Damien Grasset, Guillaume Gaudron, and Pierre-Yves Oudeyer. Causal reinforcement learning using observational and interventional data. *arXiv preprint arXiv:2106.14421*, 2021.

Samuel J. Gershman and Nathaniel D. Daw. Reinforcement learning and episodic memory in humans and animals: An integrative framework. *Annual Review of Psychology*, 68(1):101–128, 2017. doi: 10.1146/annurev-psych-122414-033625.

URL https://doi.org/10.1146/annurev-psych-122414-033625. PMID: 27618944.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

St John Grimbly. Causal reinforcement learning: A primer, 2020. URL https://stjohngrimbly.com/causal-reinforcement-learning/.

Pablo Hernandez-Leal, Bilal Kartal, and Matthew E Taylor. A very condensed survey and critique of multiagent deep reinforcement learning. In *Proceedings of the 19th international conference on autonomous agents and multiagent systems*, pages 2146–2148, 2020.

Edward Hill, Marco Bardoscia, and Arthur Turrell. Solving heterogeneous general equilibrium economic models with deep reinforcement learning. *arXiv preprint arXiv:2103.16977*, 2021.

Natascha Hinterlang and Alina Tänzer. Optimal monetary policy using reinforcement learning. 2021.

Neal Hughes. Applying reinforcement learning to economic problems. *mimeo Australian National University*, 2014.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Andrei Jirnyi and Vadym Lepetyuk. A reinforcement learning approach to solving incomplete market models with aggregate uncertainty. *Available at SSRN 1832745*, 2011.

Dazzle Johnson, Gang Chen, and Yuqian Lu. Multi-agent reinforcement learning for real-time dynamic production scheduling in a robot assembly cell. *IEEE Robotics and Automation Letters*, 7(3):7684–7691, 2022.

Artem Kuriksha. An economy of neural networks: Learning from heterogeneous experiences. *arXiv preprint arXiv:2110.11582*, 2021.

David Laredo, Yulin Qin, Oliver Schütze, and Jian-Qiao Sun. Automatic model selection for neural networks. *arXiv preprint arXiv:1905.06010*, 2019.

Yuming Li, Pin Ni, and Victor Chang. Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing*, 102(6):1305–1322, 2020.

Timothy Lillicrap, Jonathan Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv e-prints*, (arXiv), 2015.

Tao Liu, Zehan Tan, Chengliang Xu, Huanxin Chen, and Zhengfei Li. Study on deep reinforcement learning techniques for building energy consumption forecasting. *Energy and Buildings*, 208:109675, 2020.

R.E.Jr Lucas. Expectations and the Neutrality of Money. *Journal of Economic Theory*, 4:103–124, 1972.

Robert E. Lucas. Econometric policy evaluation: A critique. *Carnegie-Rochester Conference Series on Public Policy*, 1:19–46, 1976. ISSN 0167-2231. doi: https://doi.org/10.1016/S0167-2231(76)80003-6. URL `https://www.sciencedirect.com/science/article/pii/S0167223176800036`.

Laudenbach-F. Askarani M.F. Rortais F. Vincent T. Bulmer J.F. Miatto F.M. Neuhaus L. Helt L.G. Collins M.J. Madsen, L.S. and A.E. Lita. Quantum computational advantage with a programmable photonic processor. *Nature*, 606(7912): 75–81, 2022.

Ulrike Malmendier. Exposure, experience, and expertise: Why personal histories

matter in economics. Working Paper 29336, National Bureau of Economic Research, October 2021. URL http://www.nber.org/papers/w29336.

Ulrike Malmendier and Stefan Nagel. Learning from Inflation Experiences *. *The Quarterly Journal of Economics*, 131(1):53–87, 2016. ISSN 0033-5533. doi: 10.1093/qje/qjv037. URL https://doi.org/10.1093/qje/qjv037.

Marvin Minsky. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1): 8–30, 1961.

Marvin Lee Minsky. *Theory of neural-analog reinforcement systems and its application to the brain-model problem*. Princeton University, 1954.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. In *NIPS Deep Learning Workshop*. 2013.

Amir Mosavi, Pedram Ghamisi, Yaser Faghan, Puhong Duan, and Shahab Shamshirband. Comprehensive review of deep reinforcement learning methods and applications in economics. Mar 2020. doi: 10.20944/preprints202003.0309.v1. URL http://dx.doi.org/10.20944/preprints202003.0309.v1.

Thanh Thi Nguyen, Ngoc Duy Nguyen, and Saeid Nahavandi. Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications. *IEEE transactions on cybernetics*, 50(9):3826–3839, 2020.

Ann Nowé, Peter Vrancx, and Yann-Michaël De Hauwere. Game theory and multiagent reinforcement learning. *Reinforcement Learning: State-of-the-Art*, pages 441–470, 2012.

Randall C O'reilly and Yuko Munakata. *Computational explorations in cognitive neuroscience: Understanding the mind by simulating the brain*. MIT press, 2000.

Parag C Pendharkar and Patrick Cusatis. Trading financial indices with reinforcement learning agents. *Expert Systems with Applications*, 103:1–13, 2018.

Aravind Rajeswaran, Igor Mordatch, and Vikash Kumar. A game theoretic framework for model based reinforcement learning. In *International conference on machine learning*, pages 7953–7963. PMLR, 2020.

Julio J Rotemberg and Michael Woodford. An optimization-based econometric framework for the evaluation of monetary policy. *NBER macroeconomics annual*, 12:297–346, 1997.

Thomas Sargent. *Bounded Rationality in Macroeconomics*. Oxford University Press, 1993.

Thomas J. Sargent. A note on the "accelerationist" controversy. *Journal of Money, Credit and Banking*, 3(3):721–725, 1971. ISSN 00222879, 15384616. URL `http://www.jstor.org/stable/1991369`.

Thomas J Sargent and Neil Wallace. Rational expectations and the dynamics of hyperinflation. *International Economic Review*, 14(2):328–50, 1973. URL `https://EconPapers.repec.org/RePEc:ier:iecrev:v:14:y:1973:i:2:p:328-50`.

Stephanie Schmitt-Grohé and Martın Uribe. Solving dynamic general equilibrium models using a second-order approximation to the policy function. *Journal of Economic Dynamics and Control*, 28(4):755–775, 2004. ISSN 0165-1889. doi: https://doi.org/10.1016/S0165-1889(03)00043-5. URL `https://www.sciencedirect.com/science/article/pii/S0165188903000435`.

Wolfram Schultz, Peter Dayan, and P Read Montague. A neural substrate of prediction and reward. *Science*, 275(5306):1593–1599, 1997.

Egemen Sert, Yaneer Bar-Yam, and Alfredo J Morales. Segregation dynamics with

reinforcement learning and agent based modeling. *Scientific reports*, 10(1):11771, 2020.

Diya Shang, Hua Sun, and Qingliang Zeng. A reinforcement-algorithm framework for automatic model selection. In *IOP Conference Series: Earth and Environmental Science*, volume 440, page 022060. IOP Publishing, 2020.

Rui (Aruhan) Shi. Learning from zero: How to make consumption- saving decisions in a stochastic environment with an ai algorithm. *CESifo Working Paper No. 9255*, 2021a. https://ssrn.com/abstract=3907739.

Rui (Aruhan) Shi. Can an ai agent hit a moving target. *arXiv preprint arXiv*, 2110, 2021b.

Herbert A. Simon. *Behavioural Economics*, pages 1–9. Palgrave Macmillan UK, London, 2016. ISBN 978-1-349-95121-5. doi: 10.1057/978-1-349-95121-5_413-1. URL https://doi.org/10.1057/978-1-349-95121-5_413-1.

Christopher Sims. A simple model for study of the determination of the price level and the interaction of monetary and fiscal policy. *Economic Theory*, 4(3):381–99, 1994. URL https://EconPapers.repec.org/RePEc:spr:joecth:v:4:y:1994:i:3:p:381-99.

Bing Song, Gang Xiong, Songmin Yu, Peijun Ye, Xisong Dong, and Yisheng Lv. Calibration of agent-based model using reinforcement learning. In *2021 IEEE 1st International Conference on Digital Twins and Parallel Intelligence (DTPI)*, pages 278–281. IEEE, 2021.

Richard Sutton and Andrew Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.

Richard S Sutton and Andrew G Barto. Toward a modern theory of adaptive networks: expectation and prediction. *Psychological review*, 88(2):135, 1981.

Gerald Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.

Victor Uc-Cetina, Nicolas Navarro-Guerrero, Anabel Martin-Gonzalez, Cornelius Weber, and Stefan Wermter. Survey on reinforcement learning for language processing. *Artificial Intelligence Review*, pages 1–33, 2022.

William Yang Wang, Jiwei Li, and Xiaodong He. Deep reinforcement learning for nlp. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, pages 19–21, 2018.

Christopher John Cornish Hellaby Watkins. Learning from delayed rewards. 1989.

Marco Wiering and Jürgen Schmidhuber. Hq-learning. *Adaptive behavior*, 6(2): 219–246, 1997.

Peter Wittek. *Quantum machine learning: what quantum computing means to data mining*. Academic Press, 2014.

Shaojun Wu, Shan Jin, Dingding Wen, and Xiaoting Wang. Quantum reinforcement learning in continuous action space. *arXiv preprint arXiv:2012.10711*, 2020.

Yufeng Zhan and Jiang Zhang. An incentive mechanism design for efficient edge learning by deep reinforcement learning approach. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 2489–2498. IEEE, 2020.

Ji Zhang, Jingkuan Song, Lianli Gao, Ye Liu, and Heng Tao Shen. Progressive meta-learning with curriculum. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(9):5916–5930, 2022.

Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control*, pages 321–384, 2021.

Yao Zhang and Qiang Ni. Recent advances in quantum machine learning. *Quantum Engineering*, 2(1):e34, 2020.

Stephan Zheng, Alexander Trott, Sunil Srinivasa, Nikhil Naik, Melvin Gruesbeck, David C Parkes, and Richard Socher. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332*, 2020.

Feiyun Zhu, Peng Liao, Xinliang Zhu, Jiawen Yao, and Junzhou Huang. Cohesion-driven online actor-critic reinforcement learning for mhealth intervention. In *Proceedings of the 2018 ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics*, pages 482–491, 2018.

Qijun Zhu and Chun Yuan. A reinforcement learning approach to automatic error recovery. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, pages 729–738. IEEE, 2007.

Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2020.